S.G.R.S.I

Sistema de Gestión de Recursos y Soporte de Informática

Ipsum

Rol	Apellido	Nombre	C.I	Correo electrónico
Coordinador	Machado	Emidio	5.776.504-3	emidiomachadogarcia@gmail.com
Sub-Coordinador	Pereyra	Facundo	5.405.169-9	pereyrafacundo126@gmail.com
Integrante 1	Borges Monichon	Lucas Mateo	5.562.596-2	lucadborges1@gmail.com

Docentes:
Lopez, Leandro (Programación Full Stack)
Medeiros, Marcela (Ingenieria en Informática)
Falcón, Enzo (UTULab)
Flores, Pablo (Adm. Sistemas Operativos)
Saule, Eric(Ciberseguridad)

Tercera entrega

Fecha de entrega:
23/10/2026

Índice

Índice....1
Introducción del problema a abordar....6
Propuesta de solución....7
Definición clara del producto....8
Aplicación de herramientas de análisis....9
Preguntas hechas al cliente:....10
Herramientas o tecnologías a utilizar....11
Nota de conformación de la empresa....12
Organización del equipo:....12
Estrategia inicial de difusión....12
Redes Sociales — Instagram (Difusión Visual y Muestra del Producto)....13
Simulación de difusión del sistema (via Instagram)....14
Prueba del prototipo con usuarios y registro del FeedBack....15
Reevaluación del problema original....16
Confirmación de la necesidad inicial....16
Evolución del alcance del problema....16
Replanteamiento hacia la solución actual....16
Justificación de errores y casos no incluido....16
Resumen de Cumplimiento por Módulos Principales....17
Matriz Comparativa Detallada: Letra Proyecto vs. Implementación Ipsum....17
Factibilidades del sistema....19
Factibilidad Técnica....19
Factibilidad Operativa....20
Factibilidad Económica....21
Factibilidad Legal....22
Analisis FODA....23
Fortalezas....23
Oportunidades....24
Debilidades....24
Amenazas....24
Análisis F.O.D.A. ponderado....26
Análisis económico y métricas....30
Métricas del software....30
Análisis costo/beneficio....31
Comparativa de metricas:....32
Especificación de requerimientos....34
Requerimientos Funcionales (RF)....35
Requerimientos no Funcionales (RNF)....35
Modelo de Desarrollo....37
Modelo de desarrollo: actualización (tercera entrega)....40

Fundamentación y Características del Modelo Incremental....40
Ciclo de Desarrollo de Cada Incremento....41
Coordinación del Equipo (Prácticas Inspiradas en Scrum)....41
Resumen de Beneficios Clave Aplicados al Proyecto....41
Lógica y flujo del sistema....42
Evaluación y justificación de la metodología de trabajo....43
Evaluación del Aplicado de la Metodología....43
Definición de Roles....43
Uso de Artefactos y Herramientas....43
Justificación de la Metodología Elegida....44
Modelo Ambiental....44
Propósito del Sistema:....45
Lista de acontecimientos....46
Modelo de comportamiento....47
Diagrama de Análisis de Dominio....48
Análisis de Dominio....48
Identificación y Alcance de Conceptos del Dominio....48
Especificación de Clases, Atributos y Comportamientos....49
Jerarquía de Actores: Usuario, Docente, Técnico y Administrador....49
Espacios e Inventario: Aula y Activo....51
Ciclo Operativo Diario: Reporte Diario y Detalle ReportePC....51
Mesa de Ayuda, Reservas y Custodia: Incidencia, Solución, Solicitud y Préstamo...51
Multiplicidades y Semántica de Negocio....52
Trazabilidad con Requerimientos y Casos de Uso....53
El modelo de dominio actúa como puente directo entre las especificaciones del cliente y la implementación técnica de la arquitectura en tres capas:....53
Diagrama de Casos de Uso....54
I. Los Usuarios....54
II. Las Funciones....54
III. Las flechas punteadas con <<include>> (“paso obligatorio”)....55
Desglose de casos de uso....56
Diagrama de secuencias....73
Desglose del Diagrama de Secuencia....74
Dinámica por Módulos del Sistema....75
Diagrama de Estados....76
Reporte de Incidencias (Ticket)....76
Reporte Diario....77
Cuenta de Usuario....78
Solicitud de Laboratorio / Peticiones....81
Diagrama de Clases....83
Desglose del diagrama de clases....84
Casos de prueba....85
Gestion de riesgos....104
Planificación y Estrategias de Mitigación:....105

Riesgos de Disponibilidad Laboral y Académica....105
Riesgos de Personal y Estructura.... 105
Riesgos Tecnológicos y de Base de Datos....105
Riesgos de Estimación y Requerimientos.... 106
Evaluación de funcionamiento grupal y mejoras.... 107
Estándares de codificación....108
Repositorio utilizado.... 110
Ficha Técnica del Repositorio....110
Características, Ventajas y Desventajas....110
Comparativa de Mercado y Justificación.... 112
Gestion de ramas....113
Modelo de Ramificación (Git Flow / Feature Branching).... 113
Ciclo de Vida de Integración y Políticas de Merge.... 114
Convenciones de commit y resolución de issues....115
Prefijos de Conventional Commits y Ejemplos Reales.... 115
Vinculación y Cierre Automático de Issues....116
Criterios de Agrupación y Calidad del Historial....116
Versionado.... 117
Estándar Semantic Versioning (SemVer 2.0.0)....117
Cronología y Trazabilidad de Releases en Ipsum.... 118
Estructura de archivos y directorios del sistema....119
Árbol Jerárquico del Proyecto....120
Justificación de la Arquitectura y DocumentRoot.... 122
Delimitación Cliente / Servidor y Descripción por Capas.... 122
Persistencia.... 123
Diagramas de Entidad-Relación (DER y MER).... 123
Modelo Relacional Inicial (0FN) y Diagnóstico de Anomalías.... 124
Proceso Formal de Normalización (1FN → 2FN → 3FN)....126
Esquema Relacional Final Normalizado en 3FN....127
Matriz de Prevención de Anomalías....129
Punteo de Operaciones en Álgebra Relacional....129
Consultas DQL (Data Query Language)....130
Sentencias DML (Data Manipulation Language)....131
API REST....137
Propósito, Arquitectura y Protocolo.... 137
Catálogo de Recursos, Endpoints y Métodos HTTP.... 137
Catálogo de Recursos, Endpoints y Métodos HTTP.... 138
Introducción al servidor.... 140
Organización de la tercera entrega.... 140
Objetivos y alcance.... 141
Objetivo general....141
Alcance de la implementación....142
Entorno de instalación y arquitectura.... 142
Sistema operativo....142

Arquitectura del servidor....143
Script final de administración del servidor....144
Estructura modular....144
Consulta la estructura de los scripts....145
Función de la biblioteca common....147
Ejecución del menú principal....150
Manejo de errores....150
Administración de usuarios y grupos....152
Géstion de usuarios....152
Gestion de Grupos....153
Seguridad en la administración de cuentas....153
Gestión de respaldos....153
Objetivo de la estrategia de respaldo....153
Script de respaldo integral....154
Ejecución de respaldo....155
Verificación del archivo de respaldo....155
Programación automática....155
Conservación de respaldos....156
Gestión de red....157
Verificación de interfaces de red....157
Administración de bases de datos....159
Función de MariaDB....159
Consulta del servicio....159
Conexión desde PHP....160
Copias y recuperación de bases de datos....160
Administración del firewall....161
Objetivo de firewall....161
Puertos requeridos....161
Gestión de registros y logs del sistema....162
Importancia de los registros....162
Registros del sistema....162
Registros de Apache....163
Despliegue de SGRSI con Apache y PHP-FPM....163
Ubicación de la aplicación....163
Configuración del sitio de Apache....164
Funcionamiento de PHP-FPM....165
Comprobacion HTTP y acceso a la aplicación....166
Configuración y verificacion de MariaDB para SGRSI....166
Seguridad del servidor y acceso remoto mediante SSH....167
Acceso remoto....167
Principio de mínimo privilegio....167
Control de acceso al directorio de respaldos....168
Gestión de contenedores y alcance del despliegue....168
Arquitectura actualmente utilizada....169

Servidor de respaldo y contingencia..... 169
Objetivo del servidor secundario..... 169
Ejecución de respaldo..... 169
Verificación del archivo..... 170
Recuperación de los archivos..... 170
Recuperación de MariaDB..... 170
Pruebas de recuperación..... 170
Conclusiones del servidor..... 171
Riesgos y pruebas de seguridad..... 173
Plan de pruebas pendiente de ejecución..... 174
Plan de mejoras y cierre..... 175
Correcciones de prioridad alta..... 175
Mejoras complementarias..... 175
Continuidad en la entrega final..... 176
Conclusión..... 176
Referencias Bibliográficas..... 177
Libros y textos académicos..... 177
Material de clase..... 178
Anexo - Actas..... 179
Anexo..... 180

Introducción del problema a abordar

El equipo de Coordinación de Informática del Instituto Tecnológico de Informática (ITI) administra un área compuesta por múltiples talleres y laboratorios. En la actualidad, el control del estado y la verificación de estos espacios se realizan mediante un registro en papel (formularios de chequeo manuales). Aunque este método tradicional ha permitido mantener una operativa básica además de abarcar todos los problemas, se quiere llegar a un trabajo más digital para bajar el consumo de papel, mejorar la organización entre los profesores y el sector de Coordinación. El manejo físico de formularios ocasiona los siguientes problemas:

- **Acumulación de Papeles de reportes diarios:** El volumen diario de formularios generados sin incidencias o con estas satura el espacio físico de almacenamiento. Esto dificulta el orden y la conservación a largo plazo de la información de los salones.
- **Retraso en la Ayuda:** Cuando un docente detecta una rotura o falla en un equipo durante el registro diario la incidencia queda atrapada en el registro físico hasta que el formulario sea procesado: Esto puede generar retrasos debido a que también pueden haber pérdidas de estos registros.
- **Imposibilidad de analizar datos operativos:** El formato físico anula cualquier capacidad de procesar estadísticas básicas (por ejemplo, identificar qué salones tienen mayor tasa de fallas, cuáles son los componentes más propensos a romperse o el tiempo promedio de resolución), bloqueando la toma de decisiones basada en datos para la mejora de infraestructura.

Propuesta de solución

Para solucionar la problemática planteada, se digitalizará la gestión de recursos informáticos, creación de reportes (formularios diarios) y soporte técnico dentro de la institución del ITI. Se propone el desarrollo de un Sistema de Gestión de Recursos y Soporte de Informática (S.G.R.S.I) accesible mediante una aplicación web.

Esta solución permitirá centralizar en una única plataforma el registro y los seguimientos de incidencias técnicas y las solicitudes de servicios. De esta manera, ya no serán necesarios los procesos manuales, mejorando la organización, el acceso y la obtención de la información.

El sistema contará con diferentes roles de usuarios:

- **Usuario solicitante (Docente):** Podrá crear tickets diarios o de incidencias y solicitudes de servicios, además de consultar el estado de sus tickets teniendo la posibilidad de editarlos o borrarlos con un tiempo límite de 10 minutos.
- **Usuario técnico (Asistente):** Podrá gestionar las incidencias, actualizar estados y documentar soluciones.
- **Usuario Administrador (Coordinador):** Tendrá acceso a la creacion de usuarios, modificación y edición de estos, acceso a reportes con sus respectivas soluciones y métricas del sistema.

El sistema funcionará a través de una intranet y contará con autenticación de usuario garantizando la seguridad de la información, también implementará una interfaz intuitiva y accesible, permitiendo una experiencia de uso sencillo para todos los usuarios.

Con esta solución se plantea optimizar los tiempos de respuesta ante incidencias, facilitar el trabajo del personal de soporte y proporcionar información confiable para la planificación y mejora continua de los recursos informáticos del instituto.

Definición clara del producto

¿Qué es el producto?

El S.G.R.S.I. (Sistema de Gestión y Relevamiento de Software), desarrollado por Ipsum presenta un sistema creado como proyecto académico para gestionar software y requerimientos, con una base de datos para poder almacenar información y la utilización de Git para controlar las versiones del proyecto.

Características principales y componentes del sistema:

Relevamiento y documentación: Cuenta con documentación la cual describe las necesidades del sistema y ayudan a identificar, analizar y organizar los requisitos necesarios para su desarrollo.

Base de datos: Cuenta con una base de datos organizada para almacenar la información del sistema, junto con scripts SQL que permiten consultar , modificar y gestionar las tablas y sus datos.

Control de versiones y desarrollo: Utiliza Git para organizar el desarrollo del sistema, trabajar con distintas ramas según la etapa del proyecto y marcar las versiones estables para facilitar el seguimiento de los cambios realizados.

Automatización y puesta en marcha: Cuenta con scripts y herramientas que facilitan la configuración del entorno de desarrollo y permiten iniciar el sistema de forma más rápida y sencilla.

Propósito del sistema:

El objetivo principal de Ipsum con el SGRSI es mejorar y organizar la recopilación de información y la gestión de datos del software, ofreciendo una herramienta documentada y fácil de implementar, tanto en el ámbito académico como profesional.

Aplicación de herramientas de análisis

Al momento de analizar la situación presentada en la institución del ITI y principalmente en la Coordinación de Informática, se utilizaron tres herramientas para poder llegar adecuadamente a esta problemática planteada las cuales fueron: entrevista, análisis FODA y un diagrama de flujo.

Se realizó una entrevista con el cliente para poder identificar la manera en la que trabajan, que problemas abarcan diariamente con la elaboración de reportes técnicos y que les gustaría mejorar. A través del análisis FODA, el equipo pudo organizarse mejor con la información extraída, permitiendo comprender la realidad, notando la unidad del formulario ya existente y a su vez se fueron exponiendo las debilidades del formato en el cual se proyectaba dicha idea. Los formularios al estar hechos de papel se podían romper, perderse o traspapelarse, sin contar los altos costes de impresión, siendo el principal problema.

Una vez enfocados en la consigna y el problema planteado, se elaboró un diagrama de flujos para así poder visualizar paso a paso sobre el cómo manejar cada solicitud, cada reporte de alguna falla hasta que se solucionen permitiendo así ver donde se generan conflictos y retrasos.

Se llegó a la conclusión de que el sistema debe ser capaz de gestionar incidencias y recursos de forma centralizada, permitiendo el seguimiento del estado de cada tarea y diferenciar los roles de los usuarios para que cada usuario tenga acceso a la información y herramientas que necesita según su función.

Preguntas hechas al cliente:

- *"Cuando se necesita ingresar stock ¿Esto lo hacen a través de tablas por excel o en qué formato lo hacen?"*
    - *"Y si se necesita validar si hay algún ítem en el stock ¿Cómo lo hacen actualmente? ¿Les gustaría que esto se haga de alguna forma en particular?"*
- *"Al momento de hacer un ticket de instancia ¿Que diferenciaría entre un ticket hecho por un profesor u otro y un estudiante?"*
    - *"En el manejo del sistema día a día ¿Los estudiantes serán usuarios en algún módulo del sistema?"*
    - *"En el módulo de prestaciones ¿Como se manejan actualmente y como les gustaría que se maneje en el sistema?"*
    - En el caso de que se preste algo y se registre en el sistema ¿Cómo le gustaría asegurar que lo prestado sea devuelto y como le gustaría que le recuerde al prestador sobre este préstamo?
    - ¿Qué pasa si alguien pierde lo que se le prestó? ¿El sistema debe marcarlo como 'Perdido' o simplemente se elimina?

Preguntas realizadas	Objetivo del dato	Requerimiento derivado
"¿Cómo les gustaría que el sistema recuerde un préstamo?"	Definir alertas y evitar que no se devuelvan items	El sistema enviará una notificación al usuario horas antes.
"¿Los estudiantes serán usuarios del sistema?"	Definir roles y permisos	Creación de un módulo de "Acceso Estudiante" con permisos restringidos
"¿Cuando necesitan ingresar stock como lo hacen?"	Saber medio de control de stock	El sistema almacenará el stock y maneja a este

Herramientas o tecnologías a utilizar

Con el objetivo de determinar las tecnologías más adecuadas para el desarrollo del sistema de gestión de recursos de informática (S.G.R.S.I), se realizaron diversas pruebas de concepto que permitieron evaluar herramientas para el desarrollo, almacenamiento de datos y utilización del sistema.

- **Ubuntu Server:** Se realizó la instalación de Ubuntu server como sistema operativo para el servidor del sistema. Se evaluó su estabilidad, facilidad al administrarlo y si gran compatibilidad con la mayoría de entornos informáticos.
    - **Dificultades encontradas:** Configuración inicial de la red y complicaciones en la configuración del servidor.
    - **Solución aplicada**: Configuración de una IP estática e investigación por parte del equipo para su uso y preparación.

- **Visual Studio Code (VS Code):** Se escogió este editor de código para desarrollar el programa gracias a su facilidad de uso, compatibilidad con múltiples lenguajes y una gran disponibilidad de extensiones.
- **Git y GitHub:** Se realizaron pruebas para el control de versiones y trabajo colaborativo mediante Git y GitHub.
    - **Dificultades encontradas:** Conflictos al subir cambios realizados por diferentes integrantes del equipo.
    - **Solución aplicada:** Uso de pull para validar cada cambio antes de importarlos al proyecto principal.
- **HTML, CSS, JavaScript, Sql y PHP:** Se utilizaron estos lenguajes de programación para realizar la interfaz que interactúa con el servidor, el programa y el usuario.
    - **Dificultades encontradas:** Organización con la estructura del código y planificación del diseño de la página.
    - **Solución aplicada:** Se estableció un reglamento de organización de software y estándares de codificación y utilización de herramientas como Figma para establecer estándares de visualización para la página.

Nota de conformación de la empresa



Montevideo, 16/04/2026

Ipsum
Enzo Falcón
UTULAB
Instituto Tecnológico de Informática
Presente.

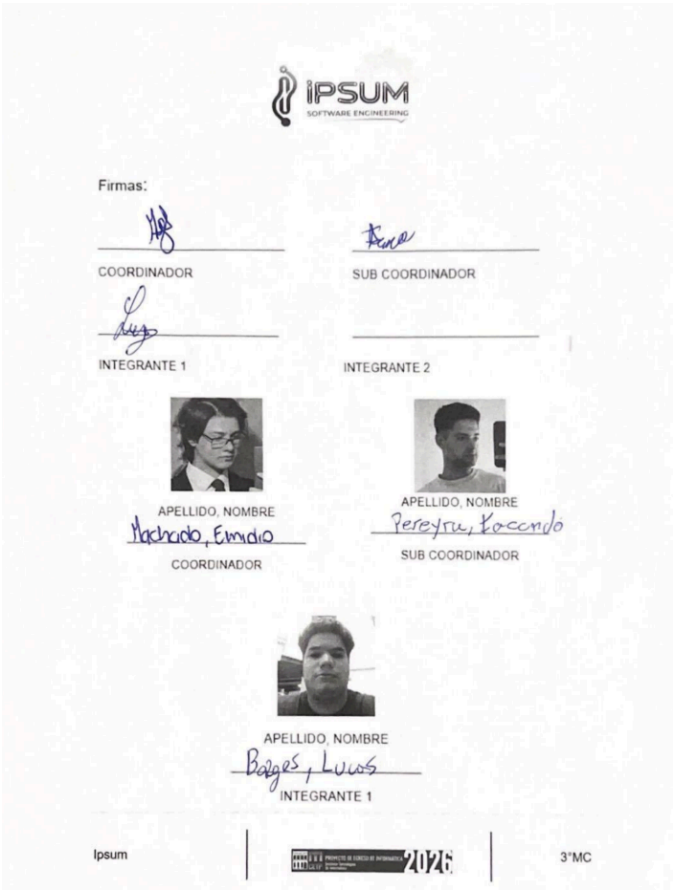
A continuación, los alumnos de 3°MC del turno matutino del Instituto Tecnológico de Informática nos presentamos ante usted, con el fin de informar la creación del grupo Ipsum..

Los correspondientes integrantes con sus roles son los siguientes:

A continuación, se detalla dicha integración y roles del grupo:

ROL	C.I	APELLIDO	NOMBRE	E-MAIL
Coordinador	5776504-3	Machado	Emidio	emidiomachadogarcia@gmail.com
Subcoordinador	5405169-9	Pereyra	Facundo	pereyrafacundo126@gmail.com
Integrante 1	5562596-2	Borges	Lucas	lucadborges1@gmail.com

Por contacto al correo: ipsum.emp@gmail.com



Firmas:

COORDINADOR

SUB COORDINADOR

INTEGRANTE 1

INTEGRANTE 2

APELLIDO, NOMBRE
Machado, Emidio
COORDINADOR

APELLIDO, NOMBRE
Pereyra, Facundo
SUB COORDINADOR

APELLIDO, NOMBRE
Borges, Lucas
INTEGRANTE 1

Ipsum | 2026 | 3°MC

Organización del equipo:

Emidio Machado (Coordinador)

- Mayor participación en la organización del equipo.
- Participación en el desarrollo del sistema.
- Participación en la elaboración de los informes y documentación.

Facundo Pereyra (Sub-coordinador):

- Mayor participación en la elaboración del servidor del sistema.
- Participación en la elaboración de los informes y documentación.

Lucas Borges (Integrante):

- Mayor participación en el desarrollo del sistema.
- Elaboración de actas de reuniones.
- Menor participación en la elaboración de los informes y documentación.

Estrategia inicial de difusión

Correo electrónico (Comunicación interna y oficial)

El correo empresarial del equipo será el medio formal para enviar avisos directos a los involucrados dentro del instituto.

- **Acción 1 (Lanzamiento e Invitación):** Envío de un correo de presentación del **S.G.R.S.I.** a los profesores y a la Coordinación de Informática del ITI, explicando la transición del formato físico/papel al sistema digital.
- **Acción 2 (Guías Rápidas y Manuales):** Envío de un "Paso a Paso" en formato digital (PDF o enlace) para que los docentes sepan cómo iniciar sesión, crear un ticket de reporte de fallas y consultar el estado de sus solicitudes.

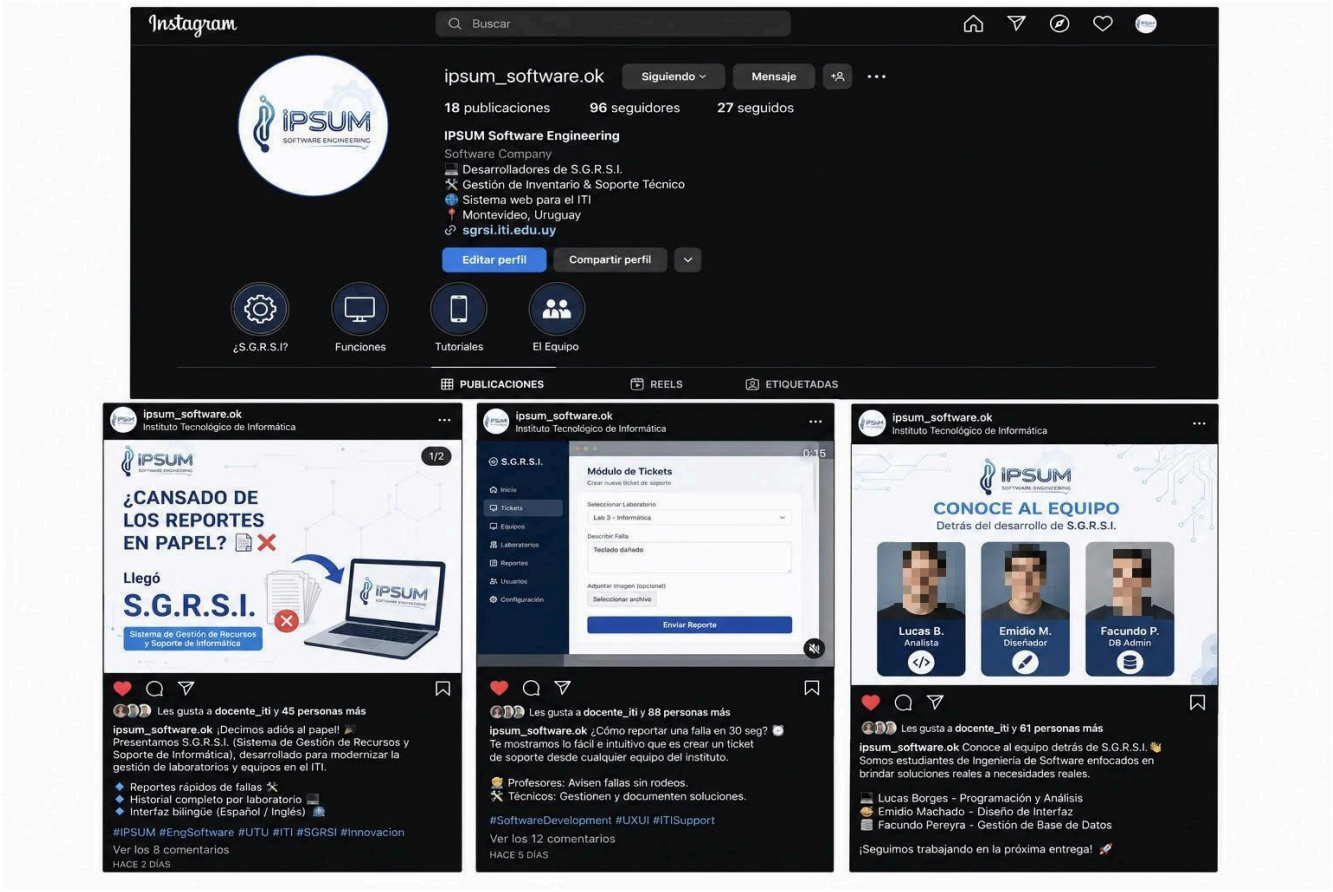
  Acción 3 (Novedades y Actualizaciones): Comunicados breves sobre nuevas funcionalidades o actualizaciones del sistema a medida que se implementen en entregas futuras.

Redes Sociales — Instagram (Difusión Visual y Muestra del Producto)

Instagram será la vitrina pública del proyecto para mostrar el funcionamiento, el diseño y el impacto del software de manera dinámica.

- **Acción 1 (Publicidad del Sistema - Screenshots y Demostraciones):**
  - Publicación de imágenes y *reels* cortos mostrando las principales pantallas del sistema: la pantalla de *login*, el módulo de *tickets*, el panel de inventario y los gráficos estadísticos.
- **Acción 2 (Muestra de Beneficios / Caso de Uso):**
  - Infografías comparativas: *"Antes (Planillas de papel y retrasos) vs. Ahora (S.G.R.S.I. digitalizado y sin consumo de papel)"*.
  - Destacar las principales fortalezas del sistema: diseño adaptativo (móviles y monitores), interfaz bilingüe (español e inglés) y gestión por roles
- **Acción 3 (Anuncios / Historias Destacadas):**
  - Crear secciones de Historias Destacadas tituladas: **"¿Qué es S.G.R.S.I.?"**, **"Funcionalidades"**, **"Tutoriales"** y **"El Equipo"**.
  - Uso de publicaciones periódicas durante la etapa de desarrollo para mostrar el avance del proyecto y generar expectativa antes del despliegue final.

Simulación de difusión del sistema (via Instagram)



Prueba del prototipo con usuarios y registro del FeedBack

I. Ejecución de la sesión

Usuario evaluado: Mikaela Nuñez**.**

- **Entorno de prueba:** Interacción con el prototipo del sistema para cargar reportes diarios y reportes de incidencias técnicas.

II. Documentación del FeedBack recibido

- **Opiniones (Usabilidad y diseño):**
  - *"Me gusta que no haya tanto ruido visual."*
  - *"La combinación de colores, diseños de botones y acceso a los módulos es cómodo al uso."*

- **Dificultades Detectadas / Oportunidades de Mejora:**
  - *"Podría aparecer una función de tiempo límite en poder editar un reporte."* Identifica la necesidad de proteger la integridad del historial de reportes una vez enviados.

III. Ajustes y Soluciones Implementadas en la Versión Actual

A partir de la observación con la usuaria y sus comentarios, se agregaron las siguientes características funcionales en el sistema:

Sistema de Permisos y Tiempos de Edición:

  - **S**e programó un temporizador límite de 10 minutos para que el docente pueda editar o corregir sus incidencias diarias tras el envío.
  - Se bloquea la edición de incidencias en cuanto pasan a estado *'En proceso'* o *'Resuelto'*.
- **Mejoras en Formularios para Profesores:**
  - Incorporación de nuevos campos académicos (*Turno*, *Horarios manuales*, *Grupo*, *Asignatura* e *Identificación editable de PCs*).
- **Gestión y Control Administrativo:**
  - Fichas tipo formulario para la revisión de registros.
  - Obligatoriedad de adjuntar documentación técnica antes de cerrar un ticket.
- **Soporte de Persistencia:**
  - Registro e historial filtrado en tiempo real de la zona horaria de Uruguay.

Reevaluación del problema original

Confirmación de la necesidad inicial

Se reafirma la problemática identificada en la entrega 1 respecto a la falta de un sistema centralizado para la gestión de recursos e incidencias en los salones informáticos.

Evolución del alcance del problema

Las pruebas con usuarios reales demostraron que el problema no era simplemente "digitalizar formularios". El desafío real radica en la integridad, trazabilidad y validez de la información reportada. Se identificó qué permitir la edición de los reportes de forma indefinida generaba desconfianza en el historial de fallas y en los tiempos de respuesta técnica.

Replanteamiento hacia la solución actual

El problema evolucionó de un simple "registro de datos" a un **sistema de gestión bajo reglas de negocio estrictas**. La solución debió incorporar mecanismos de seguridad operativa como la restricción de edición, el bloqueo automático de incidencias en procesos o resueltos, la obligación de adjuntar documentación técnica de cierre y la sincronización horaria en tiempo real

Justificación de errores y casos no incluido

- En la versión anterior de la tabla cometimos un error al calcular los costos: indicamos una duración de ocho meses, pero los subtotales correspondían a cuatro meses. Por eso figuraban $U 234.000 por integrante y $U 702.000 en total. Al revisar la tabla, corregimos el cálculo: $U 58.500 mensuales × 8 meses = $U 468.000 por integrante, lo que da un total de $U 1.404.000 para los tres integrantes.
- **Sistema bilingüe:** Por motivos de tiempo y debido a realizar cambios este no se pudo implementar.

ALCANCE DEL PROYECTO:

SOLICITADO VS. IMPLEMENTADO

Resumen de Cumplimiento por Módulos Principales

Mediante una arquitectura desacoplada en tres capas (MVC), un modelo relacional normalizado en 3FN y un backend robusto en PHP 8.5 con MySQL/MariaDB y API REST, el sistema implementado por Ipsum da respuesta a todas las áreas requeridas en la letra:

Módulo / Dimensión	Estado en Sistema	Nivel de Cobertura y Cumplimiento
Gestión de Inventario y Activos	**Implementado y Auditado**	100% — Normalizado en 3FN con catálogo de aulas y puestos PC asignados.
Mesa de Ayuda (Tickets)	**Implementado y Operativo**	100% — Ciclo Pendiente → En Proceso → Resuelto con diagnóstico obligatorio.
Solicitudes de Servicio y Aulas	**Implementado y Automatizado**	100% — Agenda interactiva con algoritmo de no solapamiento temporal y API REST.
Toma de Decisiones y Métricas	**Implementado con KPIs**	100% — Dashboard analítico dinámico con tasas de resolución, tiempos y fallas por aula.
Control de Acceso y Seguridad	**Implementado y Blindado**	100% — RBAC estricto, blindaje CSRF, consultas preparadas PDO y hash BCrypt.

Matriz Comparativa Detallada: Letra Proyecto vs. Implementación Ipsum

A continuación, se detalla ítem por ítem el requerimiento de la letra del proyecto frente a la evidencia arquitectónica, de base de datos, lógica de negocio y controladores implementada en el sistema Ipsum:

Ítem / Requerimiento	Alcance Pedido (Letra ITI)	Alcance Implementado (Ipsum)	Estado y Evidencia Técnica
Gestión de Inventario (ABM y Catalogación)	Alta, baja y modificación de activos tecnológicos (ABM), catálogo de recursos y su condición física.	Normalización relacional en 3FN mediante tabla catálogo 'aulas' (código, nombre, tipo, activa) y detalle transaccional de puestos 'reporte_diario_pc'.	Cumplido. Tablas: aulas, reporte_diario_pc. Clase: AulaModelo.php.
Trazabilidad de Equipos y Auditoría	Historial de asignaciones ('quién tuvo qué equipo y	Mapeo nominal e inmutable en 'reporte_diario_pc' (pc + nombre_alumno) asociado	Cumplido. Tablas: reportes_diarios, reporte_diario_pc.

	cuándo') para auditorías ante pérdidas o daños.	por transacción al docente, aula, turno, horario y fecha de clase.	DAO: ReporteDiarioDAO.php.
Control de Préstamos y Devoluciones	Registro de préstamos y devoluciones con disponibilidad de recursos en tiempo real.	Módulo de reservas y solicitudes de equipamiento y espacios con validación de estados (pendiente, aprobada, rechazada) y control de uso.	Cumplido. Tabla: solicitudes_laboratorio. DAO: SolicitudLaboratorioDAO.php.
Mesa de Ayuda y Ciclo de Tickets	Módulo de incidencias con ciclo de vida: Pendiente, En Proceso y Resuelto.	Módulo completo de tickets con estados formales, asignación técnica, filtrado por aula/tipo y bloqueo de edición tras su atención.	Cumplido. Tabla: reportes_incidencias. Controladores: coordinacion/ y profesor/.
Base de Conocimiento y Diagnóstico	Registro de diagnósticos, notas técnicas e historial de intervenciones previo al cierre.	El sistema exige obligatoriamente documentar la solución técnica y acción correctiva antes de cerrar cualquier ticket de incidencia.	Cumplido. Campo: solucion (TEXT). Controlador: resolverIncidencia.php.
Solicitudes de Servicio y Laboratorios	Canal formal para planificación de recursos, preparación de laboratorios y software antes de clases.	Módulo interactivo donde docentes solicitan aula, fecha, horario y motivo, con dictamen administrativo (aprobación/rechazo) y respuesta técnica.	Cumplido. Tabla: solicitudes_laboratorio. Endpoint: /api/solicitudes-laboratorio.
Calendarización y Gestión Temporal	Organización de tareas preventivas y reactivas según demanda real.	Algoritmo de cálculo de superposición horaria en tiempo real que previene colisiones de agendas entre solicitudes aprobadas en una misma sala.	Cumplido. Método: exigirDisponibilidad(). Endpoint: /api/.../disponibilidad.
Toma de Decisiones (Métricas y Reportes)	Dashboard con métricas: equipos más fallados, tiempos promedio de resolución y uso de laboratorios.	Panel analítico para Administrador con KPIs dinámicos: tasa de resolución, tiempo promedio, distribución por tipo de fallo y ranking de averías por sala.	Cumplido. Clase: MetricasModelo.php. Endpoint: /api/metricas.
Control de Acceso por Roles (RBAC)	Roles diferenciados con permisos específicos: Administrador, Técnico/Soporte y Solicitante.	Esquema RBAC estricto en sesión PHP y API REST con tres perfiles: administrador, coordinador (técnico) y profesor (solicitante), más blindaje CSRF.	Cumplido. Helper: sesion.php. Tokens CSRF + control de rutas.
Infraestructura y Despliegue de Servidor	Servidor Linux empresarial, scripts de administración modular, firewall, respaldos y contingencia.	Ubuntu Server 26.04 LTS con Apache 2.4, PHP 8.5 FPM y MariaDB 11.8. Menú modular (/scripts/admin_servidor.sh) y cron semanal (/respaldo.sh).	Cumplido (Nativo). Scripts bash modulares en /scripts/modules/.

Factibilidades del sistema

Esta sección presenta el análisis de viabilidad y el estudio de factibilidades correspondientes al desarrollo e implementación del Sistema de Gestión de Recursos y Soporte de Informática (S.G.R.S.I.). El análisis se estructura bajo los lineamientos metodológicos de la ingeniería de software, contemplando las dimensiones técnica, operativa, económica y legal.

Factibilidad Técnica

En primera instancia, el sistema operará bajo la modalidad de intranet dentro del Instituto Tecnológico de Informática. La arquitectura del software adoptará un paradigma de Programación Orientada a Objetos (POO) estructurado rigurosamente en tres capas independientes: Presentación, Lógica del Negocio y Datos. Esta separación garantiza la mantenibilidad y la escalabilidad del producto a largo plazo, permitiendo realizar adaptaciones modulares.

Respecto a la persistencia de los datos, el equipo se encuentra técnicamente capacitado para aplicar las reglas de normalización relacional hasta la Tercera Forma Normal (3FN). Este procedimiento asegura la eliminación de la redundancia informativa y previene anomalías durante las operaciones de inserción, actualización y borrado de registros de activos fijos (computadoras, impresoras y proyectores).

La seguridad informática constituye un pilar crítico dada la naturaleza web de la plataforma. Para mitigar los riesgos identificados en el análisis de amenazas (como la inyección de código malicioso o el secuestro de credenciales administrativas), se implementará un esquema robusto de validación y sanitización de datos en el servidor, complementado con la encriptación asimétrica o hashing de contraseñas (mediante algoritmos

como BCrypt) almacenadas fuera del repositorio público de control de versiones. Por lo tanto, considerando que las tecnologías requeridas se alinean con el programa académico vigente, el proyecto es técnicamente factible.

Factibilidad Operativa

Los datos obtenidos a través de las entrevistas estructuradas realizadas al Equipo de Coordinación del ITI confirman una necesidad institucional real de sustituir los registros independientes en planillas de cálculo tradicionales por una base de datos centralizada. El sistema proveerá una trazabilidad completa de los activos del centro logístico e informático. Para asegurar el correcto flujo operacional, el sistema implementará un control de acceso basado en roles bien definidos:

- **Administrador:** Gestión de altas, bajas y modificaciones de usuarios, y visualización de paneles estadísticos avanzados para la toma de decisiones estratégicas basadas en métricas de fallas.
- **Usuario Técnico:** Habilitado para la atención de incidentes reportados, control operativo de préstamos y la cumplimentación obligatoria de la documentación de soluciones previas al cierre formal de cualquier ticket de servicio.
- **Usuario Solicitante (Docentes):** Acceso restringido a interfaces intuitivas y de diseño responsivo orientadas exclusivamente a la emisión de reportes de averías y el seguimiento en tiempo real del estado de sus solicitudes.

Cabe destacar que, debido a que el alcance del despliegue inicial es básico y no incluye integraciones automáticas con servicios de mensajería externa (SMS o correo electrónico) ni aplicaciones móviles nativas, la operatividad dependerá de la consulta manual de los usuarios a través del navegador web. No obstante, dado el soporte multilingüe (español

e inglés) y la usabilidad del software, se concluye que el sistema posee una alta factibilidad operativa.

Factibilidad Económica

Para Marzo de 2026, la remuneración media de un Programador Junior en el sector tecnológico uruguayo se estima en un salario neto aproximado de $U 45.000 mensuales, lo que representa un costo empresarial aproximado de $U 58.500 mensuales al incorporar las cargas sociales y de seguridad social básicas. El desglose financiero de los recursos humanos se detalla en la Tabla 1.

Rol Asignado	**Integrante del Equipo**	**Costo Mensual Empresa (UYU)**	**Duración Estimada**	**Subtotal (UYU)**
Programador Junior/Analista	Lucas Borges	$U 58.500	8 meses	$U 468.000
Programador Junior/ Diseñador	Emidio Machado	$U 58.500	8 meses	$U 468.000
Programador Junior / DB	Facundo Pereyra	$U 58.500	8 meses	$U 468.000

Admin				
Inversión Total de Personal	**Equipo de Desarrollo**	**$U 175.500**	**8 meses**	**$U 1.404.000**

Por otra parte, los costos de infraestructura de hardware y licencias de software son nulos nulos ($U 0), puesto que el equipo utiliza estaciones de trabajo personales previamente amortizadas, herramientas de entorno de desarrollo integrado gratuitas (Visual Studio Code) y sistemas de gestión de bases de datos de código abierto. Asimismo, el despliegue del servidor de pruebas e intranet utilizará la infraestructura de red preexistente del Instituto Tecnológico de Informática.

La relación costo-beneficio demuestra que la inversión teórica de $U 702.000 se compensa con creces mediante los beneficios intangibles y tangibles para el instituto, tales como la reducción drástica en la pérdida física de dispositivos tecnológicos prestados, la optimización de los tiempos de respuesta del personal técnico y el incremento en la vida útil de los equipos debido al mantenimiento preventivo planificado. El proyecto es económicamente viable.

Factibilidad Legal

El análisis de factibilidad legal determina la conformidad del S.G.R.S.I. con el marco jurídico y normativo vigente en la República Oriental del Uruguay, garantizando que el tratamiento de datos y los derechos de autor operen bajo la legalidad.

En primer lugar, el almacenamiento de registros que asocian datos personales (nombres, roles, números de identificación o correos de estudiantes y docentes) a tickets de

fallas informáticas inscribe al sistema dentro de los términos de la Ley N° 18.331 de Protección de Datos Personales y Acción de "Habeas Data" de Uruguay. Para cumplir con las disposiciones de la Unidad Reguladora y de Control de Datos Personales (URCDP), el sistema incorporará medidas de seguridad técnicas avanzadas (encriptación de credenciales de usuario y control de accesos restringido) destinadas a evitar la filtración, alteración o el acceso no autorizado a la base de datos institucional.

En segundo lugar, en lo que respecta a la propiedad intelectual, los derechos morales del código fuente pertenecen exclusivamente a sus desarrolladores: Lucas Borges, Emidio Machado y Facundo Pereyra, al amparo de la Ley N° 9.739 de Propiedad Literaria y Artística de Uruguay y sus correspondientes modificaciones. El equipo acordará los términos de licenciamiento (pudiendo optar por una licencia de código abierto de tipo MIT o una cesión de uso no exclusiva) para permitir que el Instituto Tecnológico de Informática aloje, ejecute y mantenga la plataforma web en su intranet sin generar conflictos de derechos de autor corporativos. El proyecto se declara plenamente factible desde el punto de vista legal.

Analisis FODA

Fortalezas

- Centralización de procesos: El sistema sustituye registros manuales y planillas independientes por una herramienta única, mejorando significativamente la trazabilidad de la información.
- Roles Definidos: Estructura de permisos clara para Administradores, Técnicos, Docentes y Estudiantes, lo que asegura que cada usuario acceda solo a lo que le corresponde.

- Enfoque en Calidad Técnica: La exigencia de normalización de datos (3FN), seguridad de lado del servidor y manejo de errores profesional garantiza un sistema estable y escalable.
- Toma de Decisiones Basada en Datos: Es más sencilla la obtención de datos permitiendo optimizar el rendimiento y mejorar el estado de los equipos.
- Base de Conocimiento: El requisito de documentar soluciones antes de cerrar tickets permite crear un historial de soluciones para futuras fallas.
- Multilingüismo: Interfaz disponible en español e inglés, lo que aumenta la accesibilidad y el alcance del software.

Oportunidades

- Necesidad Real del Instituto: El proyecto responde a una carencia actual de digitalización en la Coordinación de Informática, lo que asegura que el producto final tenga utilidad inmediata.
- Posibilidad de expansión o integración futura con otros sistemas de la institución.
- Crecimiento de la cultura de digitalización en el sector educativo.

Debilidades

- **Alcance del Despliegue:** El despliegue es básico o parcial, lo que podría limitar el rendimiento en entornos de alta demanda.
- **Ausencia de Notificaciones:** Actualmente, el sistema no contempla alertas automáticas para los usuarios, dependiendo de que estos entren manualmente a consultar el estado.

Amenazas

- **Seguridad y Ciberataques:** Al ser una plataforma web conectada a la red institucional, está expuesta a intentos de inyección de código o robo de credenciales si las validaciones no son estrictas.

- **Dependencia del Servidor:** Al operar como una intranet, si la red institucional o el servidor de respaldo fallan, la gestión de soporte vuelve a ser manual temporalmente.

Análisis F.O.D.A. ponderado

El análisis F.O.D.A. identifica fortalezas, oportunidades, debilidades y amenazas del proyecto. Cada elemento tiene un peso, una calificación del 1 al 5 y una ponderación calculada, multiplicando el peso por la calificación.

Fortalezas	**Peso**	**Calificación 1-5**	**Ponderación**
Código base bien estructurado: El código está organizado por carpetas y archivos separados por función.	0.15	4	0.60
Pruebas unitarias: Uso de herramientas para comprobar que el código funcione correctamente	0.10	3	0.30
Trabajo en equipo: Buena organización con Git y tareas bien repartidas.	0.10	4	0.40
Escalabilidad: La organización del sistema facilita agregar nuevas funcionalidades y realizar mejoras en un futuro	0.15	3	0.45
Alta cohesión: los módulos	0.15	4	0.60

Fortalezas	Peso	Calificación 1-5	Ponderación
agrupan funciones relacionadas con una responsabilidad específica.			
Bajo acoplamiento entre módulos: la separación de responsabilidades reduce dependencias y facilita realizar cambios.	0.15	3	0.45
Reutilización: los componentes y utilidades comunes pueden aprovecharse en distintas partes del proyecto, evitando repetir código.	0.20	4	0.80
Subtotal Fortalezas:			3.60/5

Oportunidades	Peso	Calificación	Ponderación
Plataformas gratuitas en la nube: Disponibilidad de servicios gratuitos para publicar el sistema.	0.40	4	1.60
Devolución del profesor: Comentarios que ayudan a corregir y mejorar el	0.35	3	1.05

Oportunidades	Peso	Calificación	Ponderación
proyecto			
Herramientas modernas: Disponibilidad de herramientas que facilitan el desarrollo y la seguridad del sistema.	0.25	3	0.75
Subtotal Oportunidades:			3.40/5

Debilidades	Peso	Calificación 1-5	Ponderación
Código pendiente de mejorar: Algunas partes del sistema necesitan ser revisadas y mejoradas.	0.40	2	0.80
Documentación desactualizada: Algunos documentos y diagramas no están al día con los cambios del sistema.	0.30	2	0.60
Limite de tiempo ajustado: Margen muy corto para corregir errores críticos previos a la fecha de entrega.	0.30	1	0.30

Debilidades	Peso	Calificación 1-5	Ponderación
Subtotal Debilidades:			1.70/5

Amenazas	Peso	Calificación	Ponderación
Diferencias entre entornos: El sistema puede funcionar de forma distinta en la computadora y en el servidor.	0.40	3	1.20
Fallos de última hora en integraciones de APIs: Dependencia de servicios externos inestables.	0.35	2	0.70
Conflictos de horario del equipo: Sobrecarga académica con otras materias en la semana de entrega.	0.25	3	0.75
Subtotal Amenazas:			2.65/5

Análisis estratégico derivado:

Estrategia FO (Maximizar Fortalezas y Oportunidades): Aprovechar la buena organización del código y el trabajo en equipo para utilizar las plataformas gratuitas

disponibles y publicar el sistema en la nube. De esta manera, se busca facilitar su funcionamiento y aprovechar las herramientas disponibles para mejorar el proyecto.

Estrategia DO (Minimizar Debilidades y Maximizar Oportunidades): Aprovechar los comentarios de los profesores y las herramientas disponibles para corregir los errores y mejorar ciertos aspectos del sistema. Así, se busca dejar el proyecto en mejores condiciones antes de realizar las pruebas finales y su publicación.

Estrategia FA (Maximizar Fortalezas y Minimizar Amenazas): Aprovechar las pruebas realizadas y la buena organización del sistema para detectar y corregir errores antes de publicarlo. De esta manera, se busca reducir los problemas que puedan surgir al instalarlo en el servidor y asegurar que funcione correctamente.

Estrategia DA (Minimizar Debilidades y Amenazas): Organizar el tiempo disponible y priorizar las funciones más importantes del sistema para evitar problemas antes de la fecha de entrega. Además, contar con alternativas en caso de que algún servicio externo falle, asegurando que las funciones principales del sistema sigan funcionando correctamente.

Análisis económico y métricas

Métricas del software

1) **Tamaño del sistema:**
    - **Pantallas y formularios:** Registro de usuarios, login, crear reportes diarios y reportes de incidencias y cerrar incidencias .
    - **Tabla de datos:** Cuatro bases de datos principales (Usuarios, Aulas, Reportes Diarios y Reportes de Incidencias).
    - **Consultas y reportes:** Tablas para ver historiales, paneles por rol (Profesor, Asistentes, Admin) y gráficos con estadísticas.

2) **Esfuerzo y tiempo:**
    - Llevó 960 horas de trabajo en total.
    - Repartido entre los tres integrantes, son 320 horas por persona a lo largo de 8 meses.

Análisis costo/beneficio

1) **Costos generales:**
    - **Sueldos del personal:** En Uruguay un programador junior gana aproximadamente $U 58.000 con aportes y por persona siendo tres el desarrollo costaría unos $U 702.000 en el plazo de 8 meses.
    - **Programas y herramientas:** $U 0 (se usó Studio Code, MySQL, Apache y Git, todo gratis y de código abierto).
    - **Equipos e internet:** $U 0 (se usaron las computadoras del equipo y la red que ya tiene el ITI).
    - **Gasto de mantenimiento futuro:** Unos $U 85.000 al año (para pequeñas revisiones y luz/servidor).

Costo total de entrada $U 702.000

2) **Beneficios**
    - **Ahorro del tiempo del personal:** Docentes, asistentes y coordinador no pierden tiempo buscando qué máquina se rompió o a quien avisarle.
    - **Salas funcionando más rápido:** Los equipos no quedan semanas sin funcionamiento sin que nadie se entere.

Comparativa de metricas:

Criterio	Métrica de Tamaño (Líneas de Código - LOC)	Métrica de Función (Puntos de Función - FP)
¿Qué mide?	El volumen físico de texto/código escrito (aprox. **5.300 líneas** en SGRSI).	La cantidad de funciones y utilidades que entrega al usuario (aprox. **101 PF** en SGRSI).
Dependencia tecnológica	**Totalmente dependiente:** Si el sistema se hiciera en Java, C o PHP, la cantidad de líneas cambiaría drásticamente para hacer lo mismo.	**Independiente del lenguaje:** Mide la funcionalidad real (hacer un login o registrar una incidencia vale los mismos puntos sin importar el lenguaje).
Momento de cálculo	Solo se puede medir con certeza **al final del proyecto**, cuando ya está todo programado.	Se puede calcular **desde el inicio**, apenas se tienen los requerimientos y casos de uso listos.
Facilidad de comprensión	Fácil de entender para los programadores, pero poco útil para el cliente o directores del instituto.	Muy claro para directores y usuarios, ya que habla de "formularios", "reportes" y "tablas".
Productividad	Penaliza la eficiencia: un código más corto, limpio y moderno parece "más chico".	Premia la eficiencia: mide el valor entregado sin inflar líneas innecesarias.

Especificación de requerimientos

El sistema S.G.R.S.I es la plataforma web orientada mayormente a la intranet que se centrará mayormente en la gestión de activos tecnológicos y podrá manejar las actividades de soporte técnico del instituto; buscando implementar todo en una misma herramienta de que optimice el trabajo.

El sistema actuará como una herramienta de registro, organización y seguimiento, sin reemplazar la función humana del soporte técnico enfocados en:

- Creación de reportes diarios y reportes de incidencias.
- Gestión de solicitudes de servicio para planificación de recursos.
- Poder tomar decisiones mediante métricas.

Funcionalidad:	Estado	Descripción
Reporte de fallas		Permite avisar si algo se rompió y ver si ya lo están arreglando.
Historial de cada equipo		Saber quién usó cada aparato antes y qué arreglos se le hicieron.
Permisos por usuario		El administrador puede hacer todo, el técnico arregla y el usuario solo avisa fallas.
Diferentes idiomas		El programa se puede usar tanto en español como en inglés.
Buscador rápido		Una barrita para encontrar rápido un equipo o una falla vieja por su nombre.
Resúmenes y gráficos		Ver dibujos o tablas que muestran qué equipos se rompen más seguido.
Avisos al celular o email		El sistema no manda

		mensajes ni correos automáticos todavía.
Aplicación para descargar		No es una App de celular; se usa solo entrando por el navegador de internet.
Arreglo automático		El sistema solo ayuda a organizar; el técnico sigue siendo quien arregla el equipo.

Requerimientos Funcionales (RF)

RF - 01: Reporte de fallas: El sistema permitirá registrar incidencias técnicas asociadas a un equipo.

RF - 02: Estadísticas y reportes: El sistema permitirá generar reportes de incidencias técnicas.

RF - 03: Control de Acceso por Roles: El programa tendrá distintos roles de usuarios.

RF - 04: Autenticación y Registro: El sistema permitirá a los usuarios registrarse e iniciar sesión mediante usuario y contraseña.

RF - 05: Consulta de estado: El sistema podrá verificar el estado de los equipos.

RF - 06: Soporte multilingüe: El sistema permitirá cambiar el idioma de la interfaz entre español y inglés.

RF - 07: Editar: El programa permite editar tickets ya creados.

RF - 08: Borrar: El programa permite eliminar tickets ya creados.

Requerimientos no Funcionales (RNF)

RNF - 01: Tipo de Plataforma: El sistema opera como una intranet.

RNF - 02: Diseño Responsivo: Debe funcionar correctamente en móviles, tabletas y monitores.

RNF - 03: Seguridad y protección de datos: Bloqueo de código malicioso en formularios para proteger credenciales.

RNF - 04: Protección de Credenciales: Contraseñas encriptadas y configuración fuera del repositorio.

RNF - 05: Orden de base de datos (3FN): Organización sin información repetida para evitar errores al actualizar.

RNF - 06: Arquitectura del Sistema: Desarrollo bajo POO y estructurado en tres capas.

RNF - 07: Usabilidad: Interfaz intuitiva y de fácil navegación.

RNF - 08: Accesibilidad de Red: Acceso desde cualquier equipo conectado a la red del ITI.

RNF - 09: Gestión de errores y comunicación:El sistema deberá mostrar mensajes de error claros al usuario y registrar los errores para facilitar su diagnóstico.

Modelo de Desarrollo

La empresa decidió utilizar el Modelo Cascada como metodología de desarrollo iniciar para el proyecto S.G.R.S.I.(Sistema de Gestión de Recursos y Soporte de informática). El modelo cascada fue seleccionado debido a que presenta una estructura lineal y secuencial,permitiendo organizar el desarrollo del sistemas en etapas claramente definidas,Entre sus principales características se destacan:

- Planificación inicial detallada.
- Documentación exhaustiva en cada etapa.
- Definición clara de qué requerimientos.
- Definición clara de requerimientos.
- Control y seguimientos ordenado del proyecto.
- Facilidad para gestionar avances y entregas.

La elección de este modelo se fundamenta en que el proyecto posee requisitos relativamente estables y bien definidos desde las primeras etapas, lo que permite realizar una planificación completa antes de comenzar el desarrollo técnico.

Para la primera entrega del proyecto se solicita específicamente una planificación inicial detallada, documentación técnica y organización formal del proceso de desarrollo, aspectos en los cuales el modelo cascada resulta especialmente adecuado.

El modelo se dividirá en las siguientes etapas:

- Etapa 1:Análisis de Requisitos,
- Etapa 2:Diseño.
- Etapa 3:Implementación.

Etapa 4:Verificación

Etapa 5:Mantenimiento.

Sin embargo, además de utilizar el Modelo Cascada como metodología principal,la empresa consideró apropiado adoptar algunas características de otros modelos de desarrollo

debido a determinados requerimientos establecidos en la letra del proyecto. No obstante cabe aclarar que las características que utilicemos de otras metodologías no quita que nuestro ciclo de vida seleccionado sea cascada ya que las características de las otras metodologías sólo serán usadas como herramientas de organización del trabajo y seguimiento del proyecto.

Uno de los enfoques complementarios adoptados corresponde al modelos Scrum,específicamente mediante el uso de reuniones semanales similares a los sprints,las cuales serán utilizadas para:

- Compartir avances del proyecto.
- Organizar la distribución de tareas
- Realizar seguimiento del trabajo realizado.
- Mejorar la comunicación y coordinación del equipo.

La incorporación de estas prácticas permite una mejora en la organización interna y un seguimiento más constante del progreso del proyecto.
Asimismo,la empresa también adoptará ciertas características del Modelo Incremental,debido a la necesidad de presentar avances funcionales en distintas entregas parciales y posteriores instancias de corrección.

Este enfoque permitirá realizar entregas de valor temprano para recibir retroalimentación por parte de los docentes y aplicar mejoras progresivas al sistema durante el desarrollo del proyecto .

Las principales características adoptadas del Modelo Incremental son:

- Construcción progresiva:cada incremento añadirá nuevas funcionalidades al sistema existente.

- Flexibilidad adaptativa:permitirá realizar cambios y ajustes basados en la retroalimentación continua obtenida en cada entrega.

- Iteraciones funcionales:el sistema evolucionará progresivamente a través de múltiples entregas y correcciones.

De esta manera,la empresa utilizará el Modelo Cascada como base principal de organización y planificación,complementando con prácticas de scrum y características del Modelo Incremental para lograr un desarrollo más organizado,flexible y adaptado a las necesidades del proyecto y de esta primera entrega solicitada.

Modelo de desarrollo: actualización (tercera entrega)

Para el desarrollo del S.G.R.S.I. (Sistema de Gestión de Recursos y Soporte de Informática), la empresa optó por utilizar el Modelo Incremental como metodología principal de desarrollo. Esta elección responde a la necesidad de construir el software de forma progresiva, evitando la espera de tener el producto final para realizar las primeras entregas y permitiendo presentar versiones funcionales desde las fases tempranas.

Fundamentación y Características del Modelo Incremental

El modelo permite organizar los requerimientos según su nivel de prioridad y avanzar por partes completas que pueden ser utilizadas y evaluadas de inmediato por los usuarios. Entre sus aspectos centrales destacan:

Planificación inicial y entregas parciales Se realiza una planificación general del proyecto, definiendo qué funcionalidades se desarrollarán en cada incremento. Esto permite presentar avances funcionales sin necesidad de completar la totalidad del sistema para la primera entrega.

Retroalimentación constante: Cada versión entregada abre instancias de revisión y corrección. Los comentarios recibidos (por ejemplo, de docentes o evaluadores) permiten aplicar ajustes, correcciones y mejoras antes de pasar a la siguiente etapa de desarrollo.

Integración continua de funcionalidades: Cada nueva parte que se construye se integra directamente con el sistema preexistente, verificando que las nuevas opciones funcionen correctamente en conjunto con las anteriores.

Documentación en cada etapa: El desarrollo de cada incremento exige la elaboración y actualización de la documentación técnica correspondiente.

Ciclo de Desarrollo de Cada Incremento

Para llevar a cabo cada entrega parcial o incremento, el trabajo se divide de forma metódica en 5 etapas sucesivas:

1. Análisis de Requisitos: Identificación y definición de las necesidades específicas del incremento a construir.

2. Diseño: Estructuración técnica y visual de las soluciones planteadas.

3. Implementación: Codificación y programación de las funcionalidades del módulo.

4. Pruebas e Integración: Verificación del correcto funcionamiento del nuevo módulo y su acople con las funcionalidades que ya estaban operativas.

5. Entrega y Evaluación: Presentación del incremento para recibir retroalimentación, la cual sirve de base para planificar los ajustes de las siguientes fases.

Coordinación del Equipo (Prácticas Inspiradas en Scrum)

Para complementar la metodología y lograr un seguimiento dinámico, la empresa incorporó herramientas de organización interna tomadas de Scrum Se establecieron reuniones semanales orientadas a:

Compartir los avances del proyecto y coordinar la distribución de tareas entre los integrantes.

Monitorear el trabajo realizado, identificando a tiempo dificultades, bloqueos o tareas pendientes.

Fortalecer la comunicación y la alineación interna del equipo.

Resumen de Beneficios Clave Aplicados al Proyecto

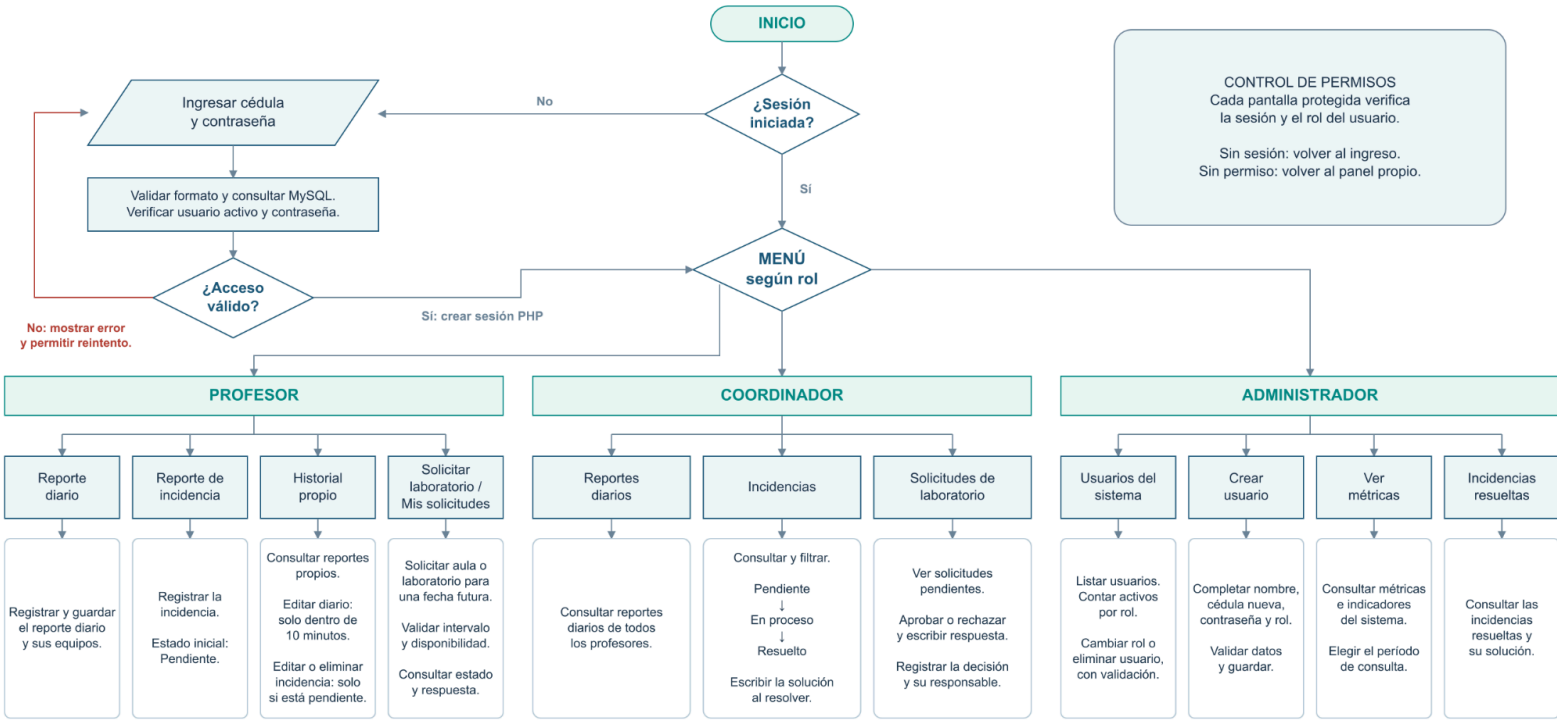
Construcción progresiva: Añade funcionalidades de forma acumulativa sobre la base ya existente.

Entregas funcionales: Proporciona versiones operativas reales para evaluar en contexto de uso.

Incorporación de ajustes: Aprovecha la retroalimentación para aplicar correcciones a lo largo del desarrollo.

Trabajo integrado y flexible: Combina la estructura del modelo incremental con el seguimiento constante de Scrum para mantener el proyecto organizado y adaptado a cada entrega solicitada.

Lógica y flujo del sistema



Evaluación y justificación de la metodología de trabajo

Evaluación del Aplicado de la Metodología

Para comprobar que estamos trabajando de manera correcta y aplicando lo planificado, realizamos un chequeo de las prácticas y herramientas del equipo:

Definición de Roles

Cada integrante tiene responsabilidades asignadas según sus fortalezas, lo que evita la superposición de tareas y mantiene la organización:

- **Lucas Borges:** Programador Junior / Analista (encargado del relevamiento y definición del sistema).
- **Emidio Machado:** Programador Junior / Diseñador (encargado de la interfaz y experiencia de usuario).
- **Facundo Pereyra:** Programador Junior / SO Admin (Encargado del funcionamiento óptimo y diseño del servidor y sistema operativo para este).

Uso de Artefactos y Herramientas

La empresa aplica correctamente los artefactos mínimos para gestionar el proyecto:

- **Repositorio en GitHub:** Se utiliza para subir el código fuente y mantener un control de versiones ordenado, garantizando que todos trabajemos sobre la última versión.
- **Diagrama de Flujo y Lógica:** Guía el desarrollo del sistema para que las pantallas y validaciones (login, tickets y formularios) se construyan respetando la lógica aprobada.
- **Minutas / Seguimiento de reuniones:** En cada encuentro semanal anotamos qué hizo cada uno y qué tareas quedan pendientes para la siguiente semana.

Conclusión de la Evaluación

El equipo está cumpliendo con las prácticas elegidas. La combinación de una planificación clara con reuniones de coordinación semanales ha permitido cumplir en tiempo y forma con la primera entrega del proyecto. Como punto a mejorar para los próximos avances, el equipo reforzará el uso de un tablero visible de tareas (tipo Kanban) dentro de GitHub para visualizar aún más rápido el estado de cada requerimiento.

Justificación de la Metodología Elegida

Para el desarrollo del S.G.R.S.I., la empresa seleccionó un enfoque híbrido que combina las prácticas de la metodología ágil Scrum y del Modelo Incremental.

Esta combinación se justifica por las siguientes razones:

- **Prácticas de Scrum (Organización del equipo):** Como somos un equipo de tres integrantes, necesitamos una comunicación constante. Incorporamos reuniones semanales cortas para ponernos al día, repartir tareas y revisar que nadie esté trabado.
- **Modelo Incremental (Adaptación a las entregas):** Este enfoque nos ayuda a construir el sistema por partes funcionales. De esta forma, podemos presentar avances en cada entrega del curso, recibir correcciones de la docente y aplicar mejoras paulatinamente.

En resumen, la metodología elegida le da a la empresa el equilibrio perfecto entre **orden y planificación** y **coordinación y entregas periódicas** (Scrum e Incremental).

Modelo Ambiental

Propósito del Sistema:

Centralizar y digitalizar la gestión de recursos tecnológicos, el inventario, los préstamos de los equipos, las incidencias técnicas y las solicitudes de servicio en el instituto, bajo la normativa legal vigente de protección de datos.

- **Entidades Externas:**
  - Usuario solicitante:

    Interactúa para reportar fallas y solicitar servicios de soporte académico.
  - Usuario técnico:

    Interactúa para registrar activos, gestionar préstamos, resolver tickets y documentar diagnósticos.
  - Usuario Administrador:

    Interactúa para supervisar métricas, gestionar usuarios y analizar reportes para la toma de decisiones.
- **Diagrama de contexto:**
  - Al centro se ubica el S.G.R.S.I. alojado en la intranet del instituto.
  - Alrededor interactúan los tres tipos de usuarios mediante flujos de datos de entrada, como la creación de tickets, solicitudes, altas de inventario, y salida como reportes, estados de solicitudes y métricas.

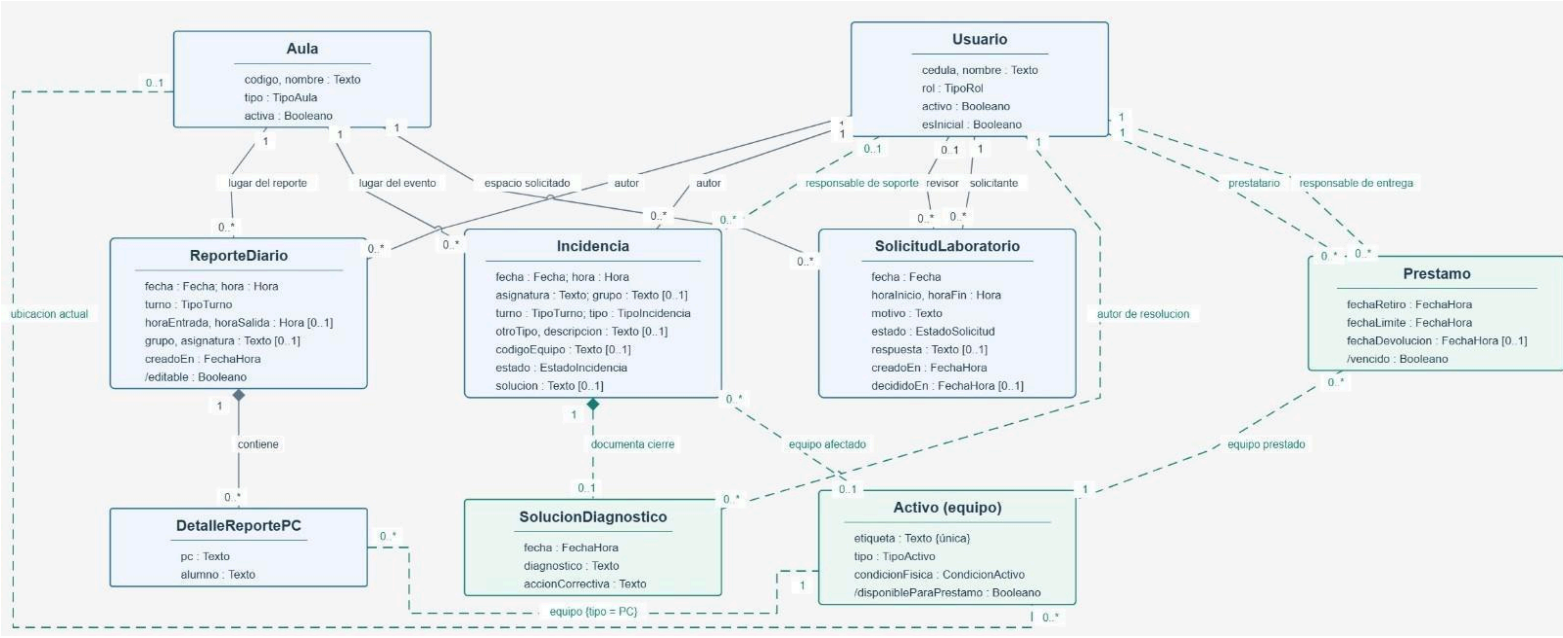
Lista de acontecimientos

- **Eventos del solicitante:**
    - El solicitante registra una incidencia técnica mediante un ticket de falla.
    - El solicitante ingresa una solicitud de servicio, como la preparación de laboratorios o instalación de software.
    - El solicitante consulta el estado de sus requerimientos en el sistema.
- **Eventos del Técnico:**
    - El técnico registra el alta, baja o la modificación de un activo tecnológico en el inventario.
    - El técnico registra el préstamo o la devolución de un equipo informático.
    - El técnico actualiza el estado de un ticket (Pendiente, En Proceso y Resuelto) y añade notas de diagnóstico.
- **Eventos del administrador:**
    - El administrador gestiona los permisos, roles y altas de nuevos usuarios institucionales.
    - El sistema procesa y genera tableros de métricas y reportes sobre el uso y estado de los equipos.

Modelo de comportamiento

- **Subsistema de inventario y trazabilidad:** Mantiene el registro de los activos tecnológicos y almacena el historial de los equipos asignados a usuarios para auditorías ante pérdidas o daños.
- **Subsistema de préstamo y devoluciones:** Revisa en tiempo real qué dispositivos hay disponibles para prestar y cuáles están en uso.
- **Subsistema de mesa de ayuda:** Gestiona los tickets de soporte desde el inicio hasta el final, preparando una lista de soluciones para que resolver los mismos problemas sea más fácil.
- **Subsistema de control de acceso y seguridad:** Auténtica y restringe las funciones del sistema según el rol del usuario (Solicitante, Técnico o Administrador), protegiendo los datos personales conforme a la normativa vigente.
- **Subsistema de toma de decisiones:** Agrupa métricas de fallas, tiempos de respuesta y uso de laboratorios para optimizar la infraestructura del instituto.

Diagrama de Análisis de Dominio



Análisis de Dominio

Identificación y Alcance de Conceptos del Dominio

El Sistema de Gestión de Recursos y Soporte de Informática (S.G.R.S.I.) articula la gestión física y de soporte técnico del Instituto Tecnológico de Informática (ITI). Para garantizar trazabilidad total y cobertura de requerimientos (RF-01 a RF-08) y casos de uso, se identifican los siguientes conceptos:

Concepto de Dominio	Naturaleza	Responsabilidad en el Negocio Institucional
Usuario	Clase Base Abstracta	Identidad institucional, autenticación única, asignación de permisos y control del ciclo de vida de credenciales.
Docente / Técnico / Admin	Especializaciones	Jerarquía que modela a solicitantes en aula, personal de mantenimiento correctivo y jefatura de coordinación.
Aula (Espacio Físico)	Recurso Locativo	Talleres y laboratorios donde se imparten clases, se asignan equipos y se registran eventos pedagógicos.
Activo (Equipo)	Activo Fijo	Computadoras, proyectores e impresoras inventariados, con seguimiento de fallas y disponibilidad física.
Reporte Diario & DetallePC	Registro Operativo	Apertura y fiscalización de clases, con trazabilidad nominal del alumno por cada puesto de PC asignado.
Incidencia & Solución	Gestión de Soporte	Ticket de rotura o avería y su obligatoria documentación de diagnóstico antes del cierre formal del caso.
SolicitudLaboratorio	Planificación	Petición formal de franja horaria para talleres especiales con control de no solapamiento temporal.
Préstamo	Custodia Temporal	Entrega y retorno de periféricos y equipos portátiles con plazos definidos y alertas de vencimiento.

Especificación de Clases, Atributos y Comportamientos

Jerarquía de Actores: Usuario, Docente, Técnico y Administrador

- **Usuario (Abstracta):** id: Int, ci: String, nombre: String, apellido: String, contrasenaHash:

String, rol: TipoRol, estado: EstadoUsuario, esInicial: Boolean.

Operaciones: autenticar(credenciales): Boolean, cambiarRol(nuevoRol), desactivarCuenta().

- **Docente:** emitirReporteDiario(...), reportarIncidencia(...), solicitarLaboratorio(...).
- **Técnico:** tomarIncidencia(t), resolverIncidencia(t, solucion), registrarPrestamo(a, u), registrarDevolucion(p).
- **Administrador:** crearUsuario(...), evaluarSolicitud(s, estado), obtenerMetricasFallas(): Reporte.

Espacios e Inventario: Aula y Activo

- **Aula:** código: String, nombre: String, tipo: TipoAula [Taller, Lab, Aula], activa: Boolean.
  Operaciones: verificarDisponibilidad(fecha, horaInicio, horaFin): Boolean, obtenerEquipos(): List<Activo>.
- **Activo:** id: Int, etiqueta: String, tipo: TipoActivo [PC, Proyector, Impresora], estado: EstadoActivo [Operativo, Falla, Prestado, Baja].

Ciclo Operativo Diario: Reporte Diario y Detalle ReportePC

- **ReporteDiario:** id: Int, fecha: Date, horaCreacion: Time, horaEntrada: Time, horaSalida: Time, turno: TipoTurno, grupo: String, asignatura: String, estado: EstadoReporte.
  Operaciones: agregarAsignacionPC(activo, alumno), actualizarDatos`(datos) [Regla: ≤ 10 min]`, `consolidar()`.
- **Detalle ReportePC:** id: Int, nombreAlumno: String. Asocia un puesto de Activo específico al estudiante de la jornada.

Mesa de Ayuda, Reservas y Custodia: Incidencia, Solución, Solicitud y Préstamo

- **Incidencia:** id: Int, fechaHora: Timestamp, tipo: TipoIncidencia, descripción: Text, estado: EstadoIncidencia [Pendiente, EnProceso, Resuelto, Cancelado].
  Operaciones: `modificar(...) [≤ 10 min y Pendiente]`, `eliminar()`, asignarTecnico(t), cerrarConSolucion(solucion).
- **SolucionDiagnostico:** id: Int, fecha: Timestamp, diagnostico: Text, accionCorrectiva: Text.
- **SolicitudLaboratorio:** id: Int, fechaReserva: Date, horaInicio: Time, horaFin: Time, estado: EstadoSolicitud.
- **Prestamo:** id: Int, fechaRetiro: Timestamp, fechaLimite: Timestamp, estado: EstadoPrestamo.

Multiplicidades y Semántica de Negocio

Relación	Tipo	Multiplicidad	Regla de Negocio Asociada
Usuario — ReporteDiario	Asociación	1 a 0..*	Un docente genera sus partes diarios; cada reporte tiene un único autor auditable.
ReporteDiario — DetallePC	Composición	1 a 1..*	Existencia dependiente. Un reporte agrupa la nómina de alumnos sentados frente a cada PC.
DetallePC → Activo	Asociación	* a 1	Cada detalle referencia a una máquina física catalogada en el inventario.
Aula — ReporteDiario / Incidencia	Asociación	1 a 0..*	Todo reporte o incidente ocurre en un espacio locativo real del ITI (resuelve el campo VARCHAR suelto).
Incidencia → Solución	Composición	1 a 0..1	No se permite cerrar formalmente un ticket sin diagnóstico técnico y solución documentada.
Aula — SolicitudLaboratorio	Asociación	1 a 0..*	Reserva temporal de salas; coordinación valida que no haya solapamiento de horarios.
Activo — Préstamo	Asociación	1 a 0..*	Control de entrega y devolución de periféricos; previene extravíos y bloquea duplicados en préstamo.

Trazabilidad con Requerimientos y Casos de Uso

El modelo de dominio actúa como puente directo entre las especificaciones del cliente y la implementación técnica de la arquitectura en tres capas:

Subsistema / Módulo	Entidades Responsables	Casos de Uso Vinculados	Regla y Estado de Negocio
Gestión de Averías e Incidencias	Incidencia, SolucionDiagnostico, Activo, Técnico	Registrar Incidencia, Diagnosticar y cerrar ticket	Transición: Pendiente → En Proceso → Resuelto. Obligatoriedad de diagnóstico escrito previo al cierre.
Control Diario de Aulas	ReporteDiario, DetalleReportePC, Aula, Docente	Crear reporte diario, Consultar historial	Transición: Registrado (Editable) → Consolidado al expirar los 10 min de gracia.
Reserva de Laboratorios	SolicitudLaboratorio, Aula, Administrador	Solicitud de laboratorio / peticiones	Control de colisión de agendas: Pendiente → Aprobada / Rechazada.
Préstamo de Equipos	Préstamo, Activo, Técnico, Usuario	Registrar préstamo de equipos	Cambio de estado del activo a Prestado; emisión de alertas preventivas de entrega.
Control de Acceso y Cuentas	Usuario, Docente, Técnico, Administrador	Autenticar usuario (<<include>>), Gestionar usuarios	Protección de cuenta inicial, hashing de claves y

Subsistema / Módulo	Entidades Responsables	Casos de Uso Vinculados	Regla y Estado de Negocio
			ciclo de cuenta: Activo → Inactivo.

Diagrama de Casos de Uso

Este gráfico funciona como una representación general que muestra quiénes usan el sistema S.G.R.S.I. y qué cosas pueden hacer dentro de él.

I. Los Usuarios

Representan a los distintos grupos de personas que interactúan con la plataforma en el día a día del instituto:

- **Docentes y Administrativos (Solicitantes):** Son quienes usan los salones y laboratorios. Su objetivo principal es avisar cuando algo no funciona (crear un ticket) y verificar si ya lo están arreglando.
- **Técnicos de Soporte:** Se encargan del mantenimiento. Ellos reciben los avisos, registran qué arreglo hicieron para solucionar el problema y llevan el control de los equipos que se prestan.
- **Administrador / Coordinador:** Es el responsable del área. Tiene acceso para gestionar el catálogo completo de equipos y ver gráficos con estadísticas de cómo va el trabajo del equipo técnico.

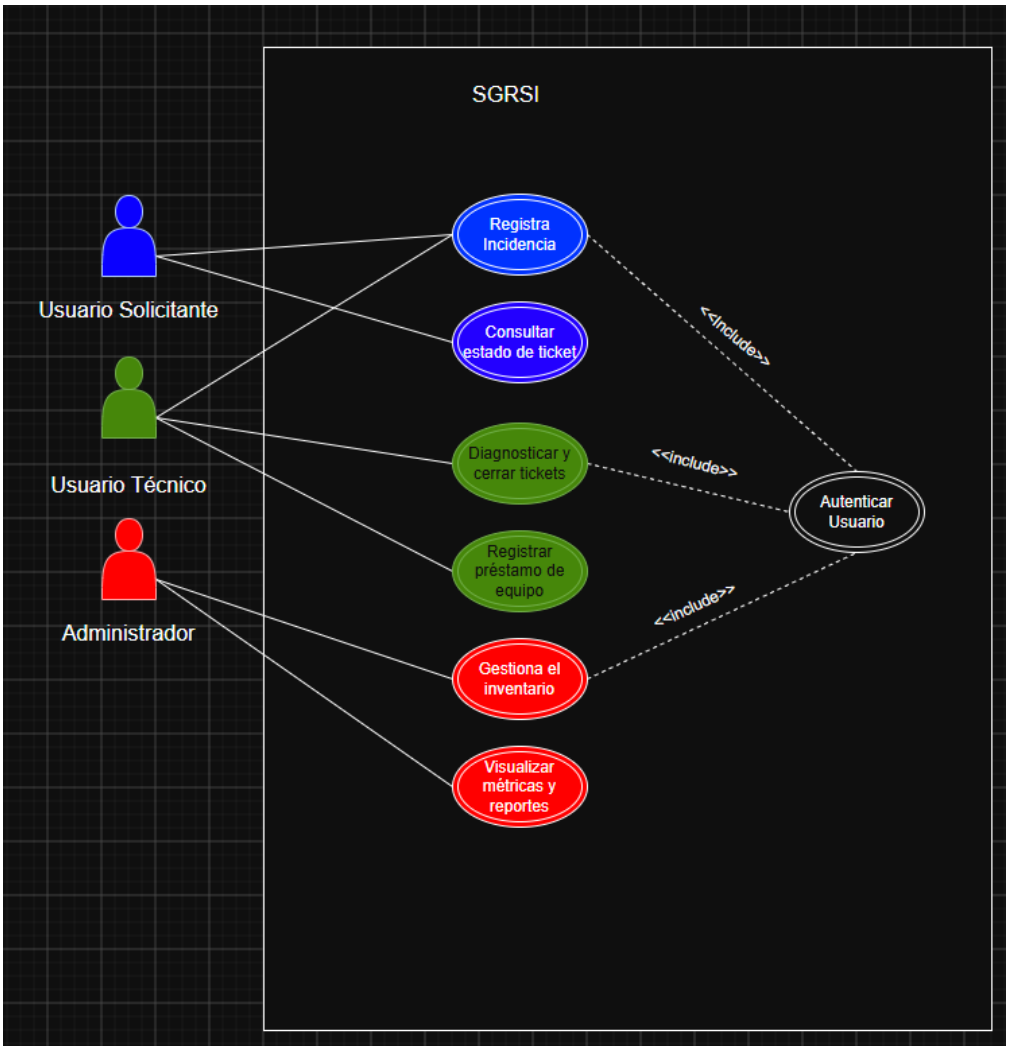
II. Las Funciones

- El rectángulo grande representa al sistema S.G.R.S.I.. Cada círculo con un texto dentro es una acción concreta que el programa les permite realizar a los usuarios.

- Las flechas que van desde una persona hacia un círculo indican las tareas que ese tipo de usuario tiene permiso para hacer.

III. Las flechas punteadas con <<include>> (“paso obligatorio”)

- Muestran que para hacer acciones importantes (como reportar una falla, cerrar un ticket o modificar el inventario), el sistema le exigirá siempre a la persona que primero ponga su usuario y contraseña. Ningún usuario desconocido puede realizar cambios en la plataforma.



Desglose de casos de uso

CU01 Iniciar sesión

Objetivo: Acceder al panel correspondiente a su rol.

Actor principal: Profesor, coordinador o administrador

Precondiciones: Usuario registrado y activo. Aplicación y base de datos disponibles. No existe una sesión autenticada.

Flujo principal

1. El usuario abre la pantalla de ingreso.

2. El sistema muestra los campos de cédula y contraseña.

3. El usuario completa sus credenciales y pulsa "Ingresar".

4. El sistema valida la solicitud, busca el usuario y verifica la contraseña y su estado activo.

5. El sistema inicia la sesión y muestra el panel asociado al rol.

Flujos alternativos y excepciones

4a. Cédula o contraseña vacías: solicita completar los datos.

4b. Cédula sin 7 u 8 dígitos normalizados: informa formato incorrecto.

4c. Usuario inexistente, inactivo o contraseña incorrecta: rechaza el ingreso.

4d. Token inválido o error de base de datos: informa el problema y vuelve al ingreso.

Postcondiciones

Existe una sesión con identificador, nombre, cédula y rol del usuario. Si falla, no se autentica.

CU02 Cerrar sesión

Objetivo: Finalizar el acceso autenticado al sistema.

Actor principal: Usuario autenticado como profesor, coordinador o administrador

Precondiciones: Sesión iniciada con cualquiera de los tres roles.

Flujo principal

1. El usuario pulsa Cerrar Sesión.

2. El sistema elimina los datos de sesión y vence la cookie.

3. El sistema devuelve al usuario a la pantalla de ingreso.

Flujos alternativos y excepciones

1a. No hay sesión activa: el sistema vuelve igualmente al ingreso.

3a. El usuario vuelve atrás o abre una ruta protegida: se le solicita ingresar nuevamente.

Postcondiciones

La sesión anterior no permite acceder a las funciones protegidas.

Reglas de esta versión

El cierre de una pestaña no es este caso de uso. sesion-pestana.js está desactivado; la cookie de sesión no garantiza cerrar el acceso al cerrar una sola pestaña.

CU03 Consultar usuarios del sistema

Objetivo: Revisar usuarios registrados y cantidades por rol.

Actor principal: Administrador

Precondiciones: Sesión de administrador. Usuarios de prueba disponibles.

Flujo principal

1. El administrador accede al panel.

2. El sistema consulta los usuarios y cuenta los activos por rol.

3. El sistema presenta nombre, cédula, rol y acciones permitidas.

4. El administrador revisa la lista y los indicadores.

Flujos alternativos y excepciones

2a. Falla la base de datos: se muestra un aviso de conexión.

3a. El usuario inicial aparece protegido, sin acciones de cambio de rol o eliminación.

1a. Otro rol solicita la ruta: se redirige a su propio panel.

Postcondiciones

El administrador conoce los usuarios y cantidades; no se modifican registros.

CU04 Crear usuario

Objetivo: Registrar una cuenta para que una persona pueda usar el sistema.

Actor principal: Administrador

Precondiciones: Sesión de administrador. Cédula nueva.

Flujo principal

1. El administrador abre Nuevo Usuario.

2. El sistema presenta nombre, cédula, contraseña y rol.

3. El administrador completa los datos y pulsa Crear Usuario.

4. El sistema valida los datos y verifica que la cédula no esté registrada.

5. El sistema guarda la cuenta activa con contraseña protegida y actualiza el panel.

Flujos alternativos y excepciones

3a. Cancela: se cierra el formulario sin guardar.

4a. Datos incompletos, cédula incorrecta o rol inválido: rechaza el alta.

4b. Cédula duplicada: informa que ya existe un usuario.

4c. Token inválido o falla de base de datos: no confirma el alta.

Postcondiciones

Existe un nuevo usuario activo con es_inicial=0 y el rol seleccionado.

Reglas de esta versión

Las cuentas las crea el administrador. Esta versión no incluye autorregistro, correo de verificación ni confirmación de contraseña.

CU05 Cambiar rol de usuario

Objetivo: Asignar un nuevo rol a una cuenta existente.

Actor principal: Administrador

Precondiciones: Sesión de administrador. Usuario destino no inicial.

Flujo principal

1. El administrador localiza al usuario en su panel.

2. Selecciona uno de los roles disponibles.

3. El sistema recibe la solicitud y valida rol, usuario y token.

4. El sistema actualiza el rol en usuarios y confirma el cambio.

Flujos alternativos y excepciones

3a. Usuario inicial: no se permite cambiar su rol.

3b. Rol o identificador inválido, o token incorrecto: rechaza la solicitud.

4a. No hay filas modificadas: el controlador muestra el aviso asociado a usuario inicial.

Postcondiciones

El rol queda actualizado en la base; los indicadores se recalculan al cargar el panel.

CU06 Eliminar usuario

Objetivo: Quitar una cuenta que puede eliminarse sin perder reportes asociados.

Actor principal: Administrador

Precondiciones: Sesión de administrador. Usuario no inicial y sin reportes vinculados.

Flujo principal

1. El administrador localiza la cuenta y pulsa Eliminar.

2. El sistema solicita confirmar la operación.

3. El administrador confirma.

4. El sistema valida la solicitud y elimina la cuenta.

5. El sistema confirma la eliminación y actualiza la lista.

Flujos alternativos y excepciones

3a. Cancela la confirmación: la cuenta permanece.

4a. Usuario inicial: se rechaza la eliminación.

4b. Usuario con reportes asociados: la base rechaza el borrado por claves foráneas.

4c. Identificador o token inválido: no se elimina.

Postcondiciones

La cuenta desaparece solo si cumple las restricciones. Los reportes existentes se conservan.

CU07 Registrar reporte diario

Objetivo: Registrar el uso del aula y las asignaciones de computadoras a alumnos.

Actor principal: Profesor

Precondiciones: Sesión de profesor. Hay aulas activas disponibles.

Flujo principal

1. El profesor abre el formulario.

2. El sistema completa la fecha y presenta aulas activas.

3. El profesor selecciona turno, horario, grupo, asignatura y aula.

4. Opcionalmente agrega filas de PC y alumno, y quita las que no corresponden.

5. El profesor pulsa Guardar Reporte.

6. El sistema guarda el reporte y sus asignaciones en una transacción y vuelve al panel.

Flujos alternativos y excepciones

4a. No agrega equipos: la versión actual permite guardar el reporte sin asignaciones.

5a. Datos obligatorios vacíos: el formulario impide enviar.

5b. Aula inválida o token incorrecto: el servidor rechaza la solicitud.

6a. Falla una inserción: se revierte la transacción y se informa el error.

5c. Cancela: vuelve al panel sin crear el reporte.

Postcondiciones

Reporte asociado al profesor y aula, con cero o más asignaciones guardadas.

CU08 Registrar incidencia

Objetivo: Comunicar una rotura, falla u otro problema en un aula.

Actor principal: Profesor

Precondiciones: Sesión de profesor y al menos un aula activa.

Flujo principal

1. El profesor abre el formulario.

2. El sistema muestra el docente de sesión y completa fecha y hora.

3. El profesor completa asignatura, grupo, turno, aula y tipo de solicitud.

4. Si selecciona Otro, especifica el tipo; puede describir el problema.

5. Pulsa Enviar Reporte.

6. El sistema registra la incidencia como pendiente y vuelve al panel.

Flujos alternativos y excepciones

4a. Tipo Otro sin especificación: el formulario exige completar el campo.

5a. Faltan datos obligatorios: se impide enviar o el servidor rechaza.

5b. Cancela: vuelve al panel sin crear la incidencia.

6a. Token inválido o falla de base de datos: no se confirma el envío.

Postcondiciones

Incidencia pendiente vinculada al profesor y aula. La descripción inicial es opcional en esta versión.

CU09 Consultar historial propio

Objetivo: Revisar sus reportes diarios e incidencias y el estado de estas.

Actor principal: Profesor

Precondiciones: Sesión de profesor. Puede haber o no reportes propios.

Flujo principal

1. El profesor abre Mis Reportes.

2. El sistema recupera únicamente sus reportes diarios e incidencias.

3. El sistema presenta tarjetas, cantidades y estados.

4. El profesor alterna Todos, Diarios e Incidencias.

5. El profesor revisa la información y las acciones disponibles.

Flujos alternativos y excepciones

2a. No hay reportes: se muestra el historial vacío.

5a. Pasaron más de 600 segundos desde creado_en del reporte diario: el servidor rechaza la edición por plazo vencido, aunque se muestre la acción Editar.

5b. Incidencia en proceso o resuelta: no se permite editar ni eliminar.

Postcondiciones

El profesor consulta únicamente sus reportes y estados; no se modifican registros.

CU10 Editar reporte diario propio

Objetivo: Corregir datos generales de un reporte recién creado.

Actor principal: Profesor

Precondiciones: Sesión de profesor. Reporte propio con antigüedad dentro de los 10 minutos.

Flujo principal

1. El profesor abre Mis Reportes y localiza el diario.

2. Pulsa Editar.

3. El sistema muestra aula, turno, horas, grupo y asignatura.

4. El profesor modifica los datos y pulsa Guardar Cambios.

5. El sistema comprueba la propiedad y el plazo, actualiza y vuelve al historial.

Flujos alternativos y excepciones

4a. Cancela: cierra el formulario sin modificar.

5a. Reporte inexistente o ajeno: rechaza la edición.

5b. Más de 600 segundos desde creado_en: rechaza por plazo vencido.

5c. Identificador, aula o token inválido: rechaza la solicitud.

Postcondiciones

Los datos generales del reporte propio quedan actualizados si se cumplen las validaciones y el plazo de 10 minutos. Las asignaciones de PC y alumno no se modifican en esta edición. Si se cancela, el reporte es ajeno o inexistente, la solicitud es inválida o pasaron más de 600 segundos desde creado_en, se conserva el reporte sin cambios.

CU11 Editar incidencia propia pendiente

Objetivo: Corregir una incidencia que todavía puede modificarse.

Actor principal: Profesor

Precondiciones: Sesión de profesor. Incidencia propia en estado pendiente.

Flujo principal

1. El profesor localiza la incidencia pendiente.

2. Pulsa Editar y el sistema abre sus datos.

3. Modifica aula, asignatura, grupo, turno, tipo o descripción.

4. Pulsa Guardar Cambios.

5. El sistema actualiza solo si la incidencia pertenece al profesor y sigue pendiente.

Flujos alternativos y excepciones

3a. Cancela: no se actualiza.

5a. Incidencia inexistente o ajena: se rechaza la edición y no se modifican datos.

5b. La incidencia ya no está pendiente: la base de datos no la modifica. Comportamiento conocido de esta versión: puede mostrarse el mensaje "actualizada correctamente" aunque ninguna fila haya cambiado. Esta discrepancia se contempla en CP11.

5c. Datos inválidos, identificador o token incorrecto: se rechaza la solicitud y no se actualiza la incidencia.

5d. Error de base de datos: no se confirma la actualización y se informa el problema.

Postcondiciones

La incidencia propia queda actualizada únicamente si permanece pendiente y los datos son válidos; conserva el estado pendiente. Si se cancela o falla una validación, no se modifica. Si dejó de estar pendiente, permanece sin cambios, aunque pueda aparecer el mensaje de éxito indicado en 5b.

CU12 Eliminar incidencia propia pendiente

Objetivo: Retirar una incidencia que aún no está en atención.

Actor principal: Profesor

Precondiciones: Sesión de profesor. Incidencia propia en estado pendiente.

Flujo principal

1. El profesor localiza la incidencia pendiente.

2. Pulsa Eliminar y confirma la pregunta.

3. El sistema verifica token, propiedad y estado.

4. El sistema elimina la incidencia y vuelve al historial.

Flujos alternativos y excepciones

2a. Cancela la pregunta: se conserva el reporte.

3a. Incidencia inexistente o ajena: informa falta de permisos.

3b. Incidencia en proceso o resuelta: rechaza la eliminación.

3c. Identificador o token inválido: no se elimina.

Postcondiciones

La incidencia pendiente desaparece del historial y de la consulta general.

CU13 Consultar reportes diarios generales

Objetivo: Revisar el uso de las aulas informado por todos los profesores.

Actor principal: Coordinador o administrador. El administrador accede por la ruta correspondiente; su panel no tiene botones hacia estas pantallas.

Precondiciones: Sesión de coordinador o administrador.

Flujo principal

1. El actor abre los registros diarios.

2. El sistema muestra reportes de todos los profesores, con aula y cantidad de equipos.

3. El actor escribe un término en el buscador si desea acotar la lista.

4. Selecciona Ver para un reporte.

5. El sistema muestra datos generales y asignaciones PC alumno; el actor cierra el detalle.

Flujos alternativos y excepciones

3a. No hay coincidencias: no aparecen filas que cumplan el término.

4a. Diario sin PCs: el detalle indica que no se registraron equipos.

1a. Profesor o visitante solicita la ruta: se restringe el acceso.

Postcondiciones

Los registros y sus asignaciones se consultan sin modificar la base.

CU14 Consultar incidencias generales

Objetivo: Revisar incidencias, sus estados y la información del problema.

Actor principal: Coordinador o administrador. El administrador accede por la ruta correspondiente; su panel no tiene botones hacia estas pantallas.

Precondiciones: Sesión de coordinador o administrador.

Flujo principal

1. El actor abre las incidencias.

2. El sistema muestra reportes de todos los profesores.

3. El actor puede buscar texto y seleccionar un estado.

4. Pulsa Ver para consultar una incidencia.

5. El sistema muestra docente, aula, tipo, descripción, estado y solución cuando existe.

Flujos alternativos y excepciones

3a. No hay coincidencias: la tabla queda sin filas coincidentes.

5a. No hay solución: el detalle omite ese apartado.

1a. Usuario sin permiso: se restringe el acceso.

Postcondiciones

El actor conoce el problema y su estado; no se modifica ninguna incidencia.

CU15 Marcar incidencia en proceso

Objetivo: Indicar que una incidencia pendiente está siendo atendida.

Actor principal: Coordinador o administrador. El administrador accede por la ruta correspondiente; su panel no tiene botones hacia estas pantallas.

Precondiciones: Sesión de coordinador o administrador. Incidencia existente pendiente.

Flujo principal

1. El actor localiza una incidencia pendiente.

2. Selecciona la acción para pasarla a En proceso.

3. El sistema valida el token y el identificador.

4. El sistema cambia el estado a en_proceso y confirma.

5. El actor revisa el estado actualizado.

Flujos alternativos y excepciones

3a. Identificador no positivo o token incorrecto: se rechaza.

4a. Falla la base de datos: se informa el error.

2a. Profesor intenta usar la ruta: se redirige a su panel.

Postcondiciones

La incidencia queda en proceso. El profesor ya no puede editarla ni eliminarla.

CU16 Documentar solución y actualizar estado

Objetivo: Registrar una solución o avance y asignar el estado correspondiente.

Actor principal: Coordinador o administrador. El administrador accede por la ruta correspondiente; su panel no tiene botones hacia estas pantallas.

Precondiciones: Sesión de coordinador o administrador. Incidencia pendiente o en proceso.

Flujo principal

1. El actor abre el formulario de solución.

2. El sistema ofrece Resuelto, En proceso y Pendiente.

3. El actor escribe la solución o avance y selecciona un estado.

4. Pulsa guardar.

5. El sistema valida que exista una descripción y actualiza solución y estado.

6. El sistema vuelve a la lista y confirma la actualización.

Flujos alternativos y excepciones

3a. Cancela: se cierra el formulario sin guardar.

5a. Solución vacía: se solicita una descripción válida.

5b. Token o identificador inválido: se rechaza la solicitud.

5c. Error de base de datos: se informa y no se confirma el cambio.

Postcondiciones

Se almacena la solución o avance y el estado elegido; si queda resuelto aparece en el panel del administrador.

CU17 Consultar incidencias resueltas

Objetivo: Revisar problemas solucionados y la solución documentada.

Actor principal: Administrador

Precondiciones: Sesión de administrador. Puede haber incidencias resueltas.

Flujo principal

1. El administrador abre su panel.

2. El sistema consulta incidencias con estado resuelto.

3. El sistema presenta la lista de reportes resueltos.

4. El administrador selecciona ver la solución.

5. El sistema muestra la información y la solución; el administrador cierra el detalle.

Flujos alternativos y excepciones

2a. No hay resueltas: se muestra el estado vacío de la sección.

4a. Cierra el detalle: vuelve a la lista sin alterar datos.

1a. Coordinador o profesor intenta abrir el panel: se restringe el acceso.

Postcondiciones

El administrador consulta las soluciones registradas sin cambiar las incidencias.

CU18 Consultar métricas del sistema

Objetivo: Analizar reportes e incidencias dentro de un período.

Actor principal: Administrador

Precondiciones: Sesión de administrador. Base de prueba con fechas y estados conocidos.

Flujo principal

1. El administrador abre Ver Métricas.

2. El sistema utiliza el período inicial de últimos 30 días.

3. El administrador puede elegir Hoy, Esta semana o un rango personalizado.

4. El sistema calcula incidencias activas, tasa de resolución, tiempo promedio y reportes diarios.

5. El sistema muestra además incidencias por aula, distribución por tipo y hasta cinco incidencias recientes.

Flujos alternativos y excepciones

3a. Rango personalizado invertido: el sistema intercambia las fechas.

3b. Fechas omitidas o formato incorrecto: utiliza valores de respaldo.

4a. Sin incidencias en el período: muestra cero activas, tasa 100 por ciento y promedio cero.

1a. Profesor o coordinador solicita la ruta: se restringe el acceso.

Postcondiciones

El administrador consulta agregados del período; no se modifican los registros.

CU19 Solicitar laboratorio o salón

Objetivo: Solicitar el uso de un aula, laboratorio o salón para una fecha y un horario determinados.

Actor principal: Profesor

Precondiciones: Sesión de profesor. Al menos un aula activa disponible para seleccionar.

Flujo principal

1. El profesor abre Solicitar laboratorio.

2. El sistema muestra las aulas activas.

3. El profesor elige el aula y la fecha; el sistema muestra la disponibilidad de ese día.

4. El profesor completa el horario de inicio y fin y el motivo o la asignatura, y envía la solicitud.

5. El sistema valida el intervalo, comprueba que la fecha y hora sean futuras, verifica la disponibilidad y valida el token CSRF.

6. El sistema guarda la solicitud como pendiente, asociada al profesor, y confirma el registro.

Flujos alternativos y excepciones

5a. Horario mal formado o fin anterior o igual al inicio: se informa el error y no se guarda la solicitud.

5b. Fecha u hora ya pasada: se rechaza la solicitud y se solicita un horario futuro.

5c. Aula ocupada por una solicitud aprobada que se superpone al horario elegido: se informa la falta de disponibilidad y no se guarda.

5d. Token CSRF inválido: se rechaza la operación y no se registra la solicitud.

6a. Error de base de datos: se informa el problema y no se confirma el registro.

Postcondiciones

Queda una solicitud pendiente a nombre del profesor, que puede consultar en Mis solicitudes. Si falla una validación o el registro, no se crea la solicitud. La decisión se realiza mediante CU21.

CU20 Consultar mis solicitudes

Objetivo: Revisar las solicitudes de laboratorio propias, su estado y la respuesta recibida.

Actor principal: Profesor

Precondiciones: Sesión de profesor. Puede haber o no solicitudes propias.

Flujo principal

1. El profesor abre Mis solicitudes.

2. El sistema recupera únicamente las solicitudes del profesor autenticado.

3. El sistema presenta el aula, la fecha, el horario y el estado de cada solicitud: pendiente, aprobada o rechazada.

4. El profesor revisa las solicitudes y la respuesta del coordinador cuando existe.

Flujos alternativos y excepciones

2a. No tiene solicitudes: el sistema muestra la lista vacía.

2b. Error de base de datos: se informa el problema y no se confirma la carga de la lista.

4a. Solicitud pendiente o sin respuesta: se presenta su estado sin una respuesta registrada.

Postcondiciones

El profesor conoce el estado de sus solicitudes y las respuestas disponibles; no se modifica ningún registro ni se muestran solicitudes de otros profesores.

CU21 Aprobar o rechazar solicitud

Objetivo: Decidir una solicitud de laboratorio y comunicar la respuesta al profesor.

Actor principal: Coordinador. El administrador también puede acceder mediante la ruta correspondiente.

Precondiciones: Sesión de coordinador o administrador. Solicitud existente en estado pendiente.

Flujo principal

1. El actor abre Solicitudes de laboratorio y consulta las pendientes.

2. El actor selecciona una solicitud y revisa sus datos.

3. El actor elige aprobar o rechazar y escribe una respuesta.

4. El sistema valida la solicitud y el token CSRF, comprueba que siga pendiente y, si se aprueba, vuelve a verificar la disponibilidad del horario.

5. El sistema guarda la decisión, la respuesta y la identidad del usuario que la tomó.

6. El sistema confirma el cambio y actualiza la lista.

Flujos alternativos y excepciones

4a. La solicitud ya fue decidida: se informa "La solicitud ya fue decidida." y no se permite una segunda decisión.

4b. Al aprobar, el horario quedó ocupado por otra solicitud aprobada que se superpone: se rechaza la operación y no se registra la aprobación.

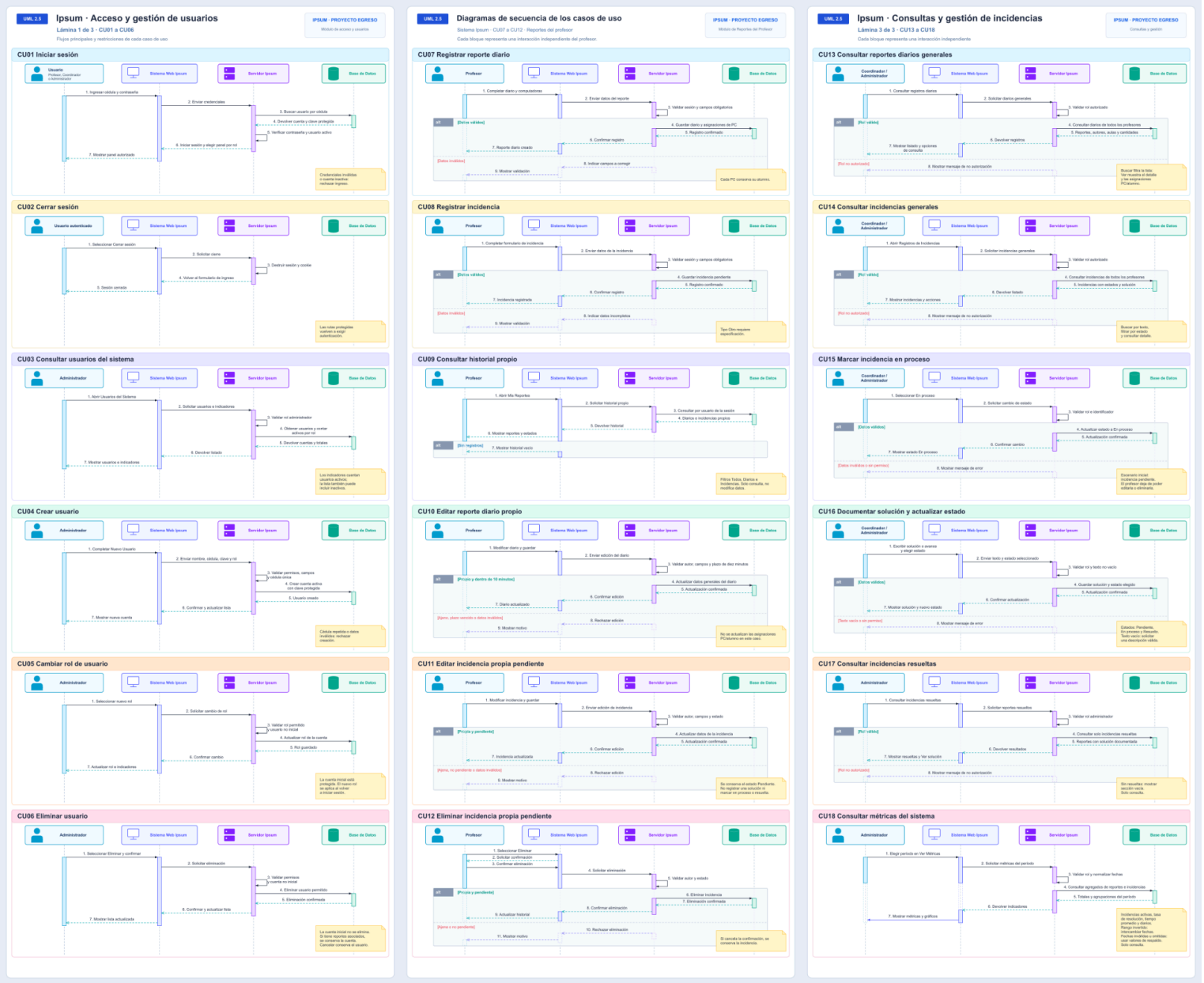
4c. Token CSRF inválido: se rechaza la operación y no se guarda la decisión.

5a. Error de base de datos: se informa el problema y no se confirma la decisión.

Postcondiciones

La solicitud pasa a aprobada o rechazada, con la respuesta y el usuario que la decidió. El profesor puede ver el resultado en CU20. Si la operación se rechaza o falla, no se guarda una nueva decisión.

Diagrama de secuencias



Desglose del Diagrama de Secuencia

El diagrama de secuencia modela cómo interactúan los usuarios con la interfaz web, los controladores y la base de datos del sistema S.G.R.S.I. ante cada caso de uso. Muestra el orden cronológico en el que se envían y responden las peticiones entre los siguientes componentes:

- **Actor (Usuario):** Docente, Asistente/Técnico o Administrador que realiza una acción en el sistema.

- **Interfaz Web (Frontend):** Pantallas y formularios accesibles desde el navegador donde el usuario ingresa o consulta datos.

- **Servicio Web (Controlador):** Lógica del servidor encargada de comprobar datos, verificar permisos y procesar solicitudes.

- **Base de Datos:** Almacena y recupera los datos oficiales (usuarios, reportes diarios, incidencias y estados).

Dinámica por Módulos del Sistema

- **Acceso y Gestión de Usuarios (CU01 al CU06):** Regula el inicio y cierre de sesión, y la administración de cuentas por el administrador. El sistema verifica que las credenciales existan en la base de datos y estén activas antes de habilitar el panel del rol correspondiente.

- **Gestión del Docente y Reportes Diarios (CU07 al CU12):** Permite al profesor editar su reporte diario dentro de un marco de 10 minutos límite posterior a su creación, y editar o eliminar sus incidencias mientras estén en estado pendiente

- **Consultas y Atención de Incidencias (CU13 al CU18):** Permite a la coordinación y administración revisar el historial general, pasar incidencias a estado `En Proceso` y resolverlas obligatoriamente con una solución técnica documentada, alimentando las métricas institucionales.

En pocas palabras, el diagrama de secuencia permite comprender el recorrido paso a paso de cada acción dentro de S.G.R.S.I., asegurando que cada operación cumpla con las validaciones requeridas de forma clara, ordenada y segura.

Diagrama de Estados

Reporte de Incidencias (Ticket)

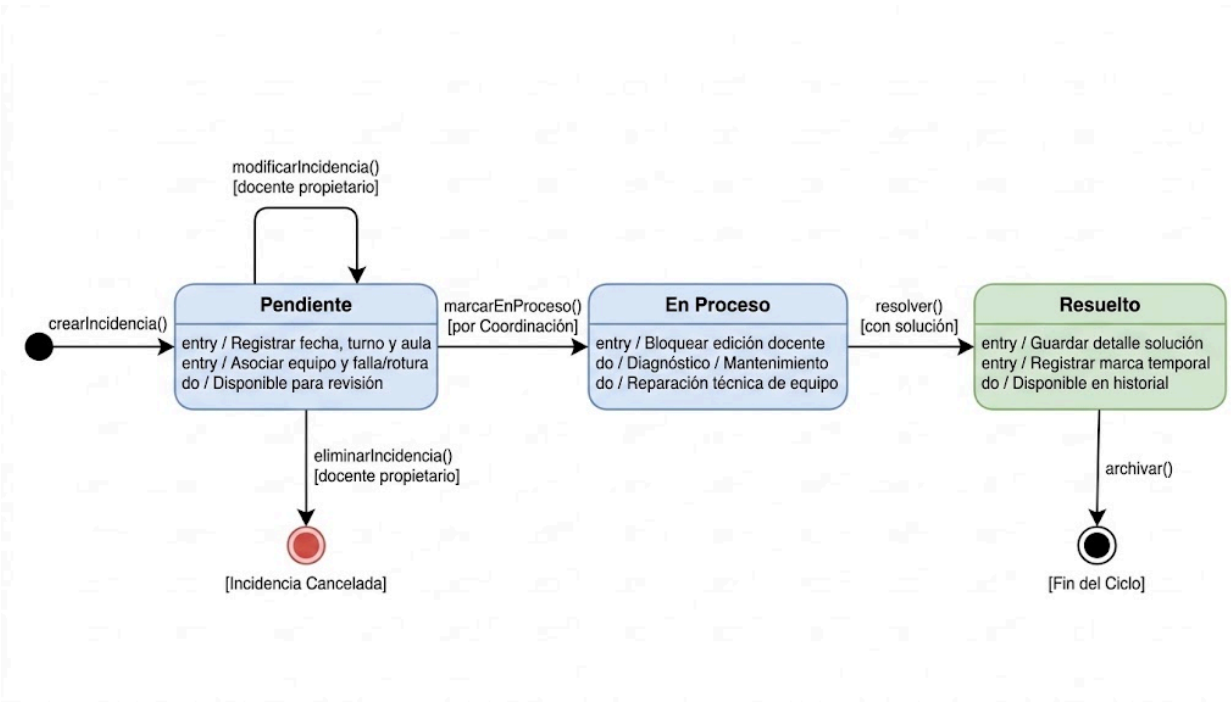


Tabla Didáctica de Estados

Estado Origen	Evento / Operación	Condición de Guarda [Regla]	Acción / Efecto	Estado Destino
Inicio	crearIncidencia()	Docente autenticado con aula y motivo	Registra ticket en BD	Pendiente
Pendiente	modificarIncidencia()	[Docente es propietario]	Actualiza datos del reporte	Pendiente (Auto-bucle)

Pendiente	eliminarIncidencia()	[Docente es propietario]	Elimina registro de BD	Final (Cancelado)
Pendiente	marcarEnProceso()	[Por Coordinación o Admin]	Bloquea edición del docente	En Proceso
En Proceso	resolver()	[Se adjunta texto de solución]	Registra solución y fecha final	Resuelto
Resuelto	archivar()	Sin condiciones	Disponible para auditoría	Final (Cierre)

Reporte Diario

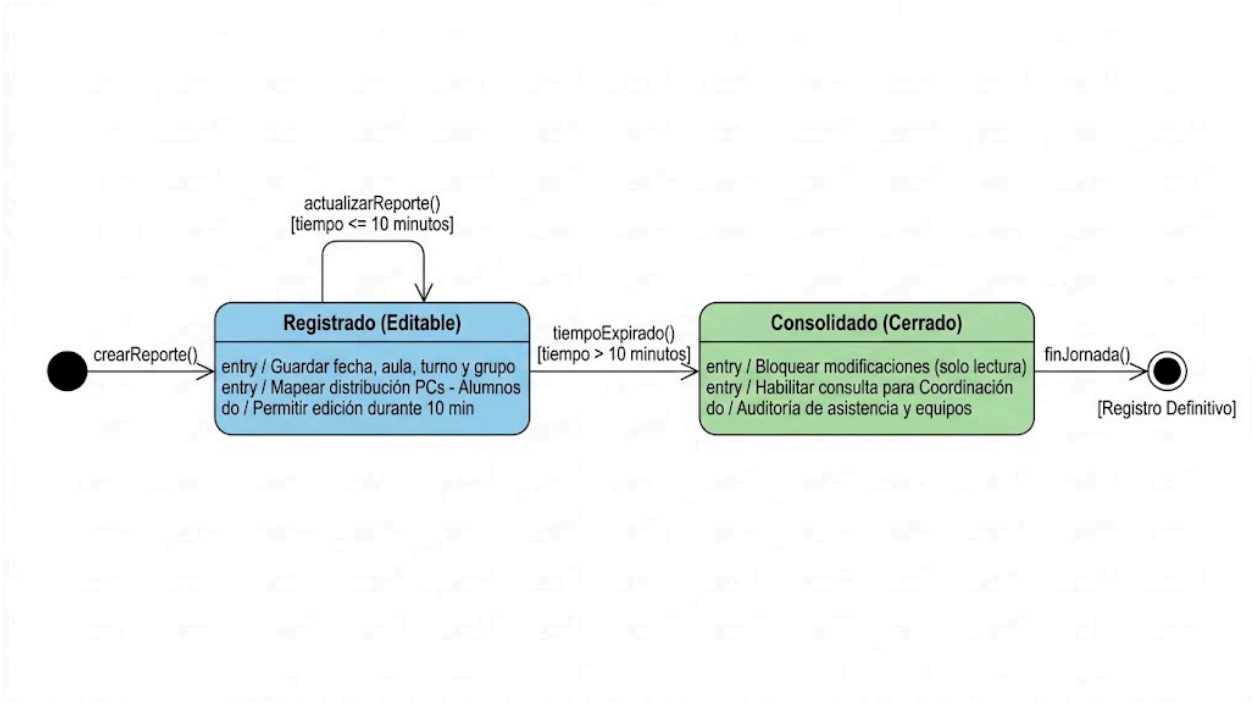


Tabla Didáctica de Estados

Estado Origen	Evento / Operación	Condición de Guarda [Regla]	Acción / Efecto	Estado Destino
Inicio	crearReporte()	Docente en aula con asignaciones	Guarda datos y marca hora creación	Registrado (Editable)
Registrado (Editable)	actualizarReporte()	[Tiempo transcurrido ≤ 10 minutos]	Permite modificar aulas y PCs	Registrado (Editable)
Registrado (Editable)	tiempoExpirado()	[Tiempo transcurrido > 10 minutos]	Bloquea edición (solo lectura)	Consolidado (Cerrado)
Consolidado (Cerrado)	finJornada()	Sin condiciones	Almacenamiento para auditoría	Final (Registro Definitivo)

Cuenta de Usuario

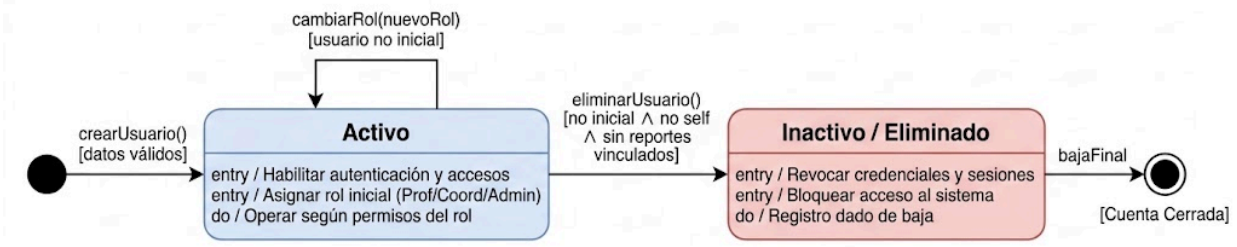


Tabla Didáctica de Estados

Estado Origen	Evento / Operación	Condición de Guarda [Regla]	Acción / Efecto	Estado Destino
Inicio	crearUsuario()	[Cédula válida ∧ Cédula no registrada]	Genera hash de contraseña y asigna rol	Activo
Activo	cambiarRol (nuevoRol)	[Usuario no es cuenta inicial protegida]	Actualiza permisos (Prof/Coord/Admin)	Activo (Auto-bucle)
Activo	eliminarUsuario()	[No inicial ∧ No cuenta propia ∧ Sin reportes]	Revoca accesos y elimina cuenta	Inactivo / Eliminado

Estado Origen	Evento / Operación	Condición de Guarda [Regla]	Acción / Efecto	Estado Destino
Inactivo / Eliminado	bajaFinal	Sin condiciones	Fin de ciclo de la cuenta	Final (Cuenta Cerrada)

Solicitud de Laboratorio / Peticiones

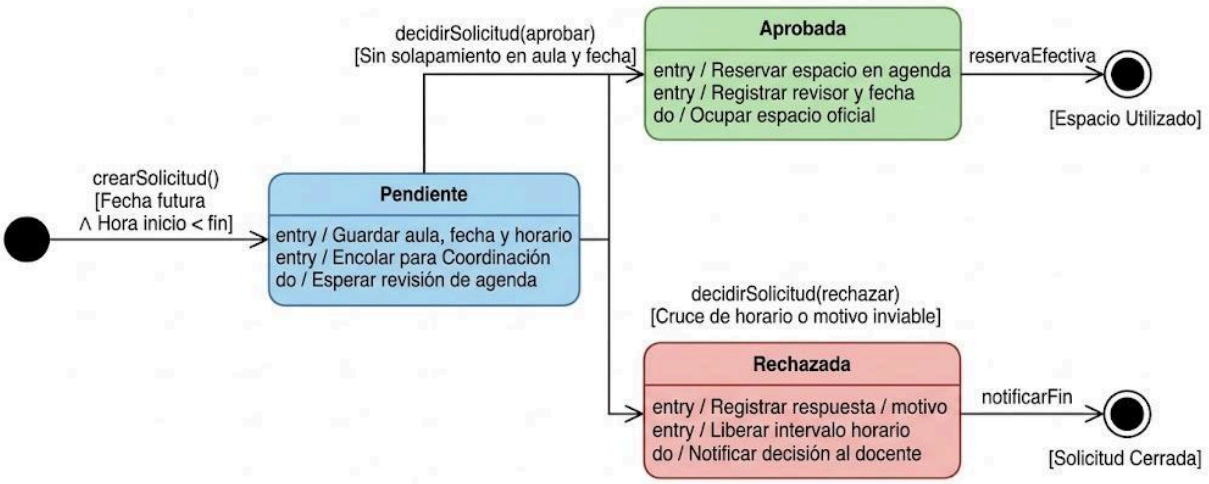
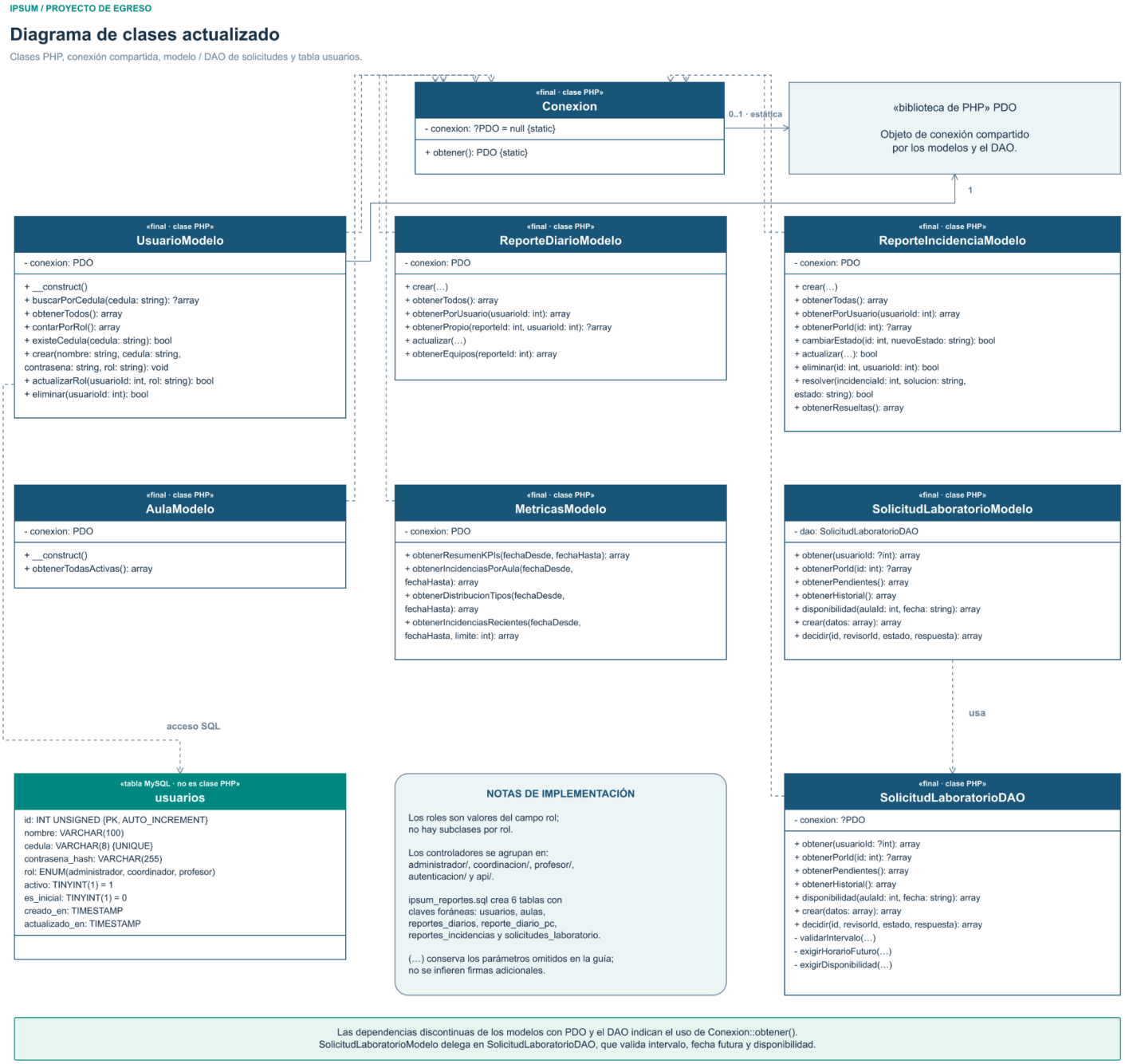


Tabla Didáctica de Estados

Estado Origen	Evento / Operación	Condición de Guarda [Regla]	Acción / Efecto	Estado Destino
Inicio	crearSolicitud()	[Fecha futura ∧ Hora inicio < fin]	Encola solicitud para revisión	Pendiente
Pendiente	decidirSolicitud (aprobar)	[Sin solapamiento	Bloquea franja horaria en agenda	Aprobada

		en aula y fecha]		
Pendiente	decidirSolicitud (rechazar)	[Cruce de horario o motivo inviable]	Guarda respuesta de denegación	Rechazada
Aprobada	reservaEfectiva	[Fecha y hora cumplidas]	Cierre administrativo de uso	Final (Espacio Utilizado)
Rechazada	notificarFin	[Docente consulta respuesta]	Fin de la gestión del pedido	Final (Solicitud Cerrada)

Diagrama de Clases



Desglose del diagrama de clases

El diagrama de clases muestra cómo está organizada la información del sistema Ipsum reportes y cómo se relacionan sus partes principales. Cada cuadro representa una parte importante del sistema y contiene los datos que guarda.

- **Usuario:** guarda la información de las personas que usan el sistema, como nombre, cédula, contraseña, rol y estado. Cada usuario tiene un identificador único.
- **Reporte Diario:** almacena los reportes que realizan los profesores cada día, con datos como fecha, turno, grupo, asignatura, aula y el usuario que creó el reporte.
- **Reporte Diario PC:** guarda las computadoras y los alumnos registrados dentro de un reporte diario. Un mismo reporte puede tener varias computadoras asociadas.
- **Reporte Incidencia:** registra los problemas o fallas encontrados en equipos o aulas, incluyendo la descripción del problema, su estado y quién lo reportó.
- **Aula:** contiene la información de las aulas, laboratorios y talleres disponibles, como su código, nombre, tipo y estado.

Las relaciones del diagrama indican que un usuario puede crear muchos reportes diarios y muchas incidencias, pero cada reporte o incidencia pertenece a un solo usuario. Además, un reporte diario puede incluir varias computadoras. En pocas palabras, el diagrama ayuda a entender cómo se organiza la información del sistema y cómo se conecta cada parte entre sí de una forma visual y sencilla.

Casos de prueba

Caso de Prueba 01: Iniciar sesión (CU01)

Objetivo: Verificar que un usuario activo ingrese con su cédula y contraseña y acceda al panel de su rol.

Precondiciones

La aplicación y la base de datos están disponibles.

Existe una cuenta activa de profesor con las credenciales indicadas.

No hay una sesión autenticada en el navegador.

Pasos:

1. Abrir la pantalla de ingreso.
2. Completar los campos de cédula y contraseña.
3. Pulsar Ingresar.
4. Comprobar que se abra el panel de profesor y se identifique al usuario.

Datos de Entrada:

Cédula: 34567890.

Contraseña: Prueba123!

Rol de la cuenta: profesor.

Resultado Esperado:

El sistema inicia una sesión con identificador, nombre, cédula y rol del usuario.

Se muestra el panel de profesor.

Condiciones de Validación:

Si falta la cédula o la contraseña, se solicita completar los datos.

Si la cédula normalizada no tiene 7 u 8 dígitos, se informa que el formato es incorrecto.

Si el usuario no existe, está inactivo o la contraseña es incorrecta, se rechaza el ingreso.

Si el token es inválido o falla la base de datos, se informa el problema y no se autentica al usuario.

Caso de Prueba 02: Cerrar sesión (CU02)

Objetivo: Verificar que Cerrar Sesión finalice el acceso autenticado e impida reutilizar la sesión anterior.

Precondiciones

El usuario tiene una sesión iniciada con rol de profesor, coordinador o administrador.

Pasos:

1. Abrir el panel del usuario autenticado.
2. Pulsar Cerrar Sesión.
3. Comprobar que se muestre la pantalla de ingreso.
4. Volver atrás o intentar abrir una ruta protegida.
5. Comprobar que el sistema solicite iniciar sesión nuevamente.

Datos de Entrada:

Sesión activa de un profesor.

Acción: Cerrar Sesión.

Ruta protegida: panel de profesor.

Resultado Esperado:

Se eliminan los datos de la sesión y vence su cookie.

El usuario vuelve al ingreso y la sesión anterior no permite acceder a funciones protegidas.

Condiciones de Validación:

Si no hay una sesión activa, la acción devuelve igualmente al ingreso.

Si se intenta acceder a una ruta protegida después del cierre, se exige una nueva autenticación.

La prueba se realiza mediante Cerrar Sesión; cerrar una pestaña no equivale a ejecutar esta acción.

Caso de Prueba 03: Consultar usuarios del sistema (CU03)

Objetivo: Verificar que el administrador consulte los usuarios registrados y las cantidades de cuentas activas por rol.

Precondiciones

Hay una sesión de administrador iniciada.

Existen cinco cuentas de prueba: cuatro activas y una inactiva, con los roles indicados.

Pasos:

1. Abrir el panel del administrador.
2. Revisar la lista de usuarios y sus nombres, cédulas y roles.
3. Comparar los indicadores con las cuentas activas preparadas.
4. Localizar al usuario inicial y revisar las acciones disponibles.

Datos de Entrada:

Cuentas activas: dos profesores, un coordinador y el administrador inicial.

Cuenta inactiva: un profesor adicional.

Los cinco usuarios tienen nombres y cédulas diferentes.

Resultado Esperado:

Se presentan los usuarios registrados con sus datos y acciones permitidas.

Los indicadores muestran 2 profesores, 1 coordinador y 1 administrador activos.

El usuario inicial no ofrece acciones de cambio de rol ni eliminación; la consulta no modifica cuentas.

Condiciones de Validación:

La cuenta inactiva no se incluye en las cantidades de usuarios activos.

Si falla la base de datos, se muestra un aviso de conexión.

Si otro rol solicita el panel administrativo, se lo redirige a su propio panel.

Caso de Prueba 04: Crear usuario (CU04)

Objetivo: Verificar que el administrador registre una cuenta activa con una cédula nueva y el rol seleccionado.

Precondiciones

Hay una sesión de administrador iniciada.

La cédula 45678901 no está registrada.

Pasos:

1. Pulsar Nuevo Usuario.
2. Completar nombre, cédula y contraseña.
3. Seleccionar el rol Profesor.
4. Pulsar Crear Usuario.
5. Comprobar que la cuenta aparezca en el panel con el rol elegido.

Datos de Entrada:

Nombre: Usuario de prueba.

Cédula: 45678901.

Contraseña: Prueba123!

Rol: profesor.

Resultado Esperado:

Se crea una cuenta activa con rol profesor y es_inicial=0.

La contraseña se guarda protegida y el panel refleja el nuevo usuario.

Condiciones de Validación:

Si faltan datos, la cédula es incorrecta o el rol es inválido, se rechaza el alta.

Si la cédula ya está registrada, se informa que el usuario existe.

Si el administrador cancela, se cierra el formulario sin crear la cuenta.

Si el token es inválido o falla la base de datos, no se confirma el alta.

Caso de Prueba 05: Cambiar rol de usuario (CU05)

Objetivo: Verificar que el administrador asigne un nuevo rol a una cuenta que no sea la inicial.

Precondiciones

Hay una sesión de administrador iniciada.

Existe el usuario de prueba con cédula 45678901, rol profesor y es_inicial=0.

Se conocen las cantidades de usuarios activos por rol antes del cambio.

Pasos:

1. Localizar al usuario en el panel administrativo.
2. Cambiar su selector de rol de Profesor a Coordinador.
3. Comprobar la confirmación del cambio.
4. Recargar el panel y revisar el rol y los indicadores.

Datos de Entrada:

Usuario destino: cédula 45678901.

Rol anterior: profesor.

Nuevo rol: coordinador.

Resultado Esperado:

El rol del usuario queda actualizado a coordinador en la base de datos.

Al cargar el panel, la cantidad de profesores activos baja en uno y la de coordinadores sube en uno.

Condiciones de Validación:

Si se intenta cambiar el rol del usuario inicial, el sistema lo impide.

Si el rol o el identificador son inválidos, o el token es incorrecto, se rechaza la solicitud.

Si no hay filas modificadas, el controlador muestra el aviso asociado al usuario inicial.

Caso de Prueba 06: Eliminar usuario (CU06)

Objetivo: Verificar que el administrador elimine una cuenta no inicial que no tenga reportes asociados.

Precondiciones

Hay una sesión de administrador iniciada.

Existe el usuario con cédula 45678901 y es_inicial=0.

La cuenta no tiene reportes diarios ni incidencias vinculados.

Pasos:

1. Localizar la cuenta en el panel administrativo.
2. Pulsar Eliminar en su fila.
3. Confirmar la operación.
4. Comprobar el mensaje de eliminación y la lista actualizada.
5. Verificar que los demás usuarios y los reportes existentes se conserven.

Datos de Entrada:

Usuario destino: cédula 45678901.

Confirmación de la eliminación: aceptar.

Resultado Esperado:

La cuenta se elimina y deja de aparecer en la lista.

Los reportes existentes se conservan.

Condiciones de Validación:

Si se cancela la confirmación, la cuenta permanece.

Si la cuenta es la inicial, se rechaza su eliminación.

Si tiene reportes vinculados, la base de datos rechaza el borrado por las claves foráneas.

Si el identificador o el token son inválidos, no se elimina la cuenta.

Caso de Prueba 07: Registrar reporte diario (CU07)

Objetivo: Verificar que el profesor registre el uso de un aula y las asignaciones de computadoras a alumnos.

Precondiciones

Hay una sesión de profesor iniciada.

Laboratorio 1 está activo y disponible en el formulario.

Pasos:

1. Abrir Crear Reporte Diario y comprobar la fecha automática.
2. Seleccionar turno, horario, grupo, asignatura y aula.
3. Agregar dos filas de PC y completar los nombres de los alumnos.
4. Pulsar Guardar Reporte.
5. Comprobar el regreso al panel y que el reporte y sus dos asignaciones estén guardados.

Datos de Entrada:

Fecha: la que completa automáticamente el sistema.

Turno: matutino; horario: 08:00 a 10:00.

Grupo: 3.º Informática; asignatura: Programación; aula: Laboratorio 1.

PC1: Alumno Uno; PC2: Alumno Dos.

Resultado Esperado:

Se guarda un reporte asociado al profesor y al aula, con las dos asignaciones ingresadas.

El reporte y sus asignaciones se almacenan en una misma transacción y se vuelve al panel.

Condiciones de Validación:

Si no se agregan equipos, esta versión permite guardar el reporte sin asignaciones.

Si faltan datos obligatorios, el formulario impide el envío.

Si el aula es inválida o el token es incorrecto, el servidor rechaza la solicitud.

Si falla una inserción, se revierte la transacción; si se cancela, no se crea el reporte.

Caso de Prueba 08: Registrar incidencia (CU08)

Objetivo: Verificar que el profesor comunique un problema de un aula y que la incidencia se registre como pendiente.

Precondiciones

Hay una sesión de profesor iniciada.

Laboratorio 1 está activo y disponible.

Pasos:

1. Abrir Crear Reporte de Incidencia.
2. Comprobar el docente de la sesión y la fecha y hora automáticas.
3. Completar asignatura, grupo, turno y aula; seleccionar Otro como tipo.
4. Especificar el tipo y escribir la descripción del problema.

5. Pulsar Enviar Reporte y comprobar el regreso al panel.

6. Consultar el registro creado y verificar su estado pendiente.

Datos de Entrada:

Asignatura: Programación; grupo: 3.º Informática; turno: matutino.

Aula: Laboratorio 1; tipo de solicitud: Otro.

Especificación del tipo: problema de conectividad.

Descripción: La PC3 no tiene conexión a la red.

Resultado Esperado:

Se registra la incidencia con los datos ingresados, vinculada al profesor y al aula.

El estado inicial es pendiente.

Condiciones de Validación:

Si se elige Otro sin especificar el tipo, el formulario exige completar ese campo.

Si faltan datos obligatorios, se impide el envío o el servidor lo rechaza.

La descripción inicial es opcional en esta versión.

Si se cancela, no se crea la incidencia; si el token es inválido o falla la base, no se confirma el envío.

Caso de Prueba 09: Consultar historial propio (CU09)

Objetivo: Verificar que el profesor consulte únicamente sus reportes y que los filtros distingan diarios e incidencias.

Precondiciones

Hay una sesión del profesor de prueba iniciada.

Ese profesor tiene un diario y tres incidencias: pendiente, en proceso y resuelta.

Otro profesor tiene un diario y una incidencia que deben quedar fuera del historial.

Pasos:

1. Abrir Mis Reportes.
2. Revisar Todos y comprobar que aparezcan los cuatro reportes propios.
3. Seleccionar Diarios y comprobar que aparezca solo el diario propio.
4. Seleccionar Incidencias y revisar las tres incidencias y sus estados.
5. Comprobar las cantidades, las acciones disponibles y la ausencia de reportes ajenos.

Datos de Entrada:

Filtro: Todos, luego Diarios y luego Incidencias.

Reportes propios: 1 diario y 3 incidencias.

Reportes ajenos: 1 diario y 1 incidencia.

Resultado Esperado:

Todos muestra 4 reportes propios; Diarios muestra 1 e Incidencias muestra 3.

Se presentan los estados correspondientes y no se consultan ni modifican registros ajenos.

Condiciones de Validación:

Si el profesor no tiene reportes, se muestra el historial vacío.

Una incidencia en proceso o resuelta no permite editar ni eliminar.

Si un diario supera el plazo del servidor, no puede guardarse su edición, aunque el botón pudiera seguir visible.

Caso de Prueba 10: Editar reporte diario propio (CU10)

Objetivo: Verificar que el profesor corrija los datos generales de un diario propio dentro de los 10 minutos permitidos.

Precondiciones

Hay una sesión de profesor iniciada.

Existe un diario propio creado hace 2 minutos, con asignaciones de PC registradas.

Se conocen sus valores originales y hay un aula válida para la modificación.

Pasos:

1. Abrir Mis Reportes y localizar el diario.

2. Pulsar Editar y revisar los valores cargados.

3. Modificar aula, turno, horario, grupo y asignatura.

4. Pulsar Guardar Cambios antes de superar los 600 segundos desde la creación.

5. Comprobar los cambios en el historial y comparar la fecha y las asignaciones originales.

Datos de Entrada:

Diario propio con antigüedad de 2 minutos al comenzar.

Nuevos datos: Laboratorio 2; turno vespertino; horario 14:00 a 16:00.

Nuevo grupo: 3.º B; nueva asignatura: Base de Datos.

Resultado Esperado:

Se actualizan los datos generales y se vuelve al historial.

La fecha original y las asignaciones de PC se conservan.

Condiciones de Validación:

Si el reporte es inexistente o pertenece a otro profesor, se rechaza la edición.

Si pasaron más de 600 segundos desde su creación, se rechaza por plazo vencido.

Si el identificador, el aula o el token son inválidos, se rechaza la solicitud.

Si se cancela, el reporte conserva sus valores anteriores.

Caso de Prueba 11: Editar incidencia propia pendiente (CU11)

Objetivo: Verificar que el profesor corrija una incidencia propia pendiente y conserve su estado, fecha y hora originales.

Precondiciones

Hay una sesión de profesor iniciada.

Existe una incidencia propia pendiente, con fecha y hora conocidas.

Laboratorio 2 está disponible para seleccionar.

Pasos:

1. Abrir Mis Reportes y localizar la incidencia pendiente.

2. Pulsar Editar y revisar los datos cargados.

3. Modificar el aula, la asignatura, el grupo, el turno, el tipo y la descripción.

4. Pulsar Guardar Cambios.

5. Comprobar los nuevos datos y que se conserven el estado pendiente y la fecha y hora originales.

Datos de Entrada:

Incidencia propia en estado pendiente.

Nuevos datos: Laboratorio 2; Base de Datos; 3.º B; turno vespertino; tipo Falla.

Nueva descripción: La PC4 se reinicia al iniciar el sistema.

Resultado Esperado:

Se guardan los datos corregidos y el estado permanece pendiente.

Se conservan la fecha y la hora originales.

Comportamiento conocido: si la incidencia dejó de estar pendiente antes del envío, la base de datos no modifica el registro, aunque puede mostrarse el mensaje «actualizada correctamente». Ese aviso es un defecto conocido y no acredita una modificación efectiva.

Condiciones de Validación:

Si faltan campos obligatorios, se solicita completarlos.

Si la incidencia es ajena o dejó de estar pendiente, no debe modificarse.

Si el token es incorrecto o falla la base de datos, no se confirma una actualización válida.

Si se cancela, la incidencia conserva sus datos anteriores.

Caso de Prueba 12: Eliminar incidencia propia pendiente (CU12)

Objetivo: Verificar que el profesor elimine una incidencia propia pendiente y que deje de aparecer en las consultas.

Precondiciones

Hay una sesión de profesor iniciada.

Existe una incidencia propia en estado pendiente.

Se conoce su identificador y hay acceso autorizado a la consulta general para comprobar su ausencia.

Pasos:

1. Abrir Mis Reportes y localizar la incidencia pendiente.
2. Pulsar Eliminar.
3. Aceptar la confirmación.
4. Comprobar el regreso al historial y la ausencia de la incidencia.
5. Con una sesión de coordinador o administrador, comprobar su ausencia en la consulta general.

Datos de Entrada:

Incidencia de prueba identificada, propia y pendiente.

Confirmación de eliminación: aceptar.

Resultado Esperado:

La incidencia se elimina.

Deja de aparecer tanto en el historial del profesor como en la consulta general.

Condiciones de Validación:

Si se cancela la confirmación, se conserva la incidencia.

Si es inexistente o ajena, se informa la falta de permisos.

Si está en proceso o resuelta, se rechaza la eliminación.

Si el identificador o el token son inválidos, no se elimina.

Caso de Prueba 13: Solicitar laboratorio (CU19)

Objetivo: Verificar que el profesor registre una solicitud para un aula activa en una fecha y horario futuros y que quede pendiente.

Precondiciones

La aplicación y la base de datos están disponibles y hay una sesión de profesor iniciada.

Laboratorio 1 está activo. Para la fecha de mañana, según la fecha del sistema al ejecutar la prueba, el intervalo de 14:00 a 15:30 está libre y no tiene solicitudes aprobadas superpuestas.

Se conoce la cantidad inicial de solicitudes del profesor para comprobar que se agrega una sola.

Pasos:

1. Abrir Solicitar laboratorio.

2. Seleccionar Laboratorio 1 y la fecha de mañana; revisar que la disponibilidad del día indique libre el intervalo de 14:00 a 15:30.

3. Completar las horas de inicio y fin y el motivo o asignatura.

4. Pulsar el botón de envío de la solicitud y revisar la confirmación.

5. Abrir Mis solicitudes y localizar la nueva solicitud por aula, fecha y horario.

6. Comprobar sus datos, autor, estado pendiente y que la cantidad de solicitudes aumentó en una.

Datos de Entrada:

Aula: Laboratorio 1, activa; fecha: mañana, calculada al ejecutar la prueba.

Hora de inicio: 14:00; hora de fin: 15:30.

Motivo o asignatura: Práctica de Programación del grupo 3.º Informática.

Resultado Esperado:

Se muestra un mensaje de éxito y se guarda una única solicitud asociada al profesor de la sesión.

La solicitud aparece en Mis solicitudes con el aula, la fecha, el horario y el motivo ingresados, en estado pendiente.

Condiciones de Validación:

La fecha y el horario deben ser futuros al momento del envío y el aula debe continuar activa y disponible.

Si el token CSRF es inválido o falla la base de datos, no debe confirmarse un registro válido.

Caso de Prueba 14: Solicitar laboratorio con horario superpuesto (CU19)

Objetivo: Verificar que el sistema rechace una solicitud cuyo horario se superpone con una solicitud ya aprobada de la misma aula y fecha.

Precondiciones

Hay una sesión de profesor iniciada y Laboratorio 1 está activo.

Como preparación de la prueba, existe una solicitud aprobada de otro profesor para Laboratorio 1, mañana según la fecha del sistema, de 14:00 a 15:30.

Se conocen el identificador y los datos de la solicitud aprobada, y la cantidad inicial de solicitudes del profesor que ejecuta la prueba.

Pasos:

1. Abrir Solicitar laboratorio y seleccionar Laboratorio 1 y la fecha de mañana.
2. Revisar la disponibilidad del día y comprobar el intervalo ya ocupado de 14:00 a 15:30.
3. Completar una nueva solicitud de 15:00 a 16:00 con el motivo indicado.
4. Intentar enviar la solicitud y revisar el aviso de horario no disponible.
5. Abrir Mis solicitudes y verificar que no se agregó la solicitud rechazada.
6. Comprobar que la solicitud aprobada original conserva su estado y sus datos.

Datos de Entrada:

Aula y fecha para ambas solicitudes: Laboratorio 1; mañana, calculada al ejecutar la prueba.

Reserva existente aprobada: 14:00 a 15:30. Nueva solicitud: 15:00 a 16:00; superposición: 15:00 a 15:30.

Motivo o asignatura de la nueva solicitud: Práctica de Base de Datos.

Resultado Esperado:

El sistema informa que el horario no está disponible y no guarda la nueva solicitud.

La cantidad de solicitudes del profesor permanece igual y la solicitud aprobada existente no se modifica.

Condiciones de Validación:

La comprobación se realiza sobre la misma aula y fecha, con intervalos efectivamente superpuestos.

La validación del servidor debe impedir el alta aunque el envío llegue al servidor; ocultar el horario en la pantalla no sustituye esa validación.

Caso de Prueba 15: Solicitar con fecha pasada o fin antes del inicio (CU19)

Objetivo: Verificar, en dos variantes independientes, que no se guarden solicitudes con fecha pasada ni con un intervalo cuyo fin sea anterior al inicio.

Precondiciones

Hay una sesión de profesor iniciada y Laboratorio 1 está activo.

Se toma como referencia la fecha del sistema al ejecutar la prueba. El aula no tiene solicitudes aprobadas superpuestas con los datos de prueba.

Se registra la cantidad inicial de solicitudes y se abre un formulario limpio para cada variante, de modo que cada intento tenga un único dato inválido.

Pasos:

1. Variante A: abrir Solicitar laboratorio y seleccionar Laboratorio 1.
2. Ingresar la fecha de ayer, un intervalo válido de 14:00 a 15:30 y el motivo indicado.
3. Intentar enviar y comprobar la validación por fecha u horario pasados.
4. Revisar Mis solicitudes y verificar que no se agregó ningún registro.
5. Variante B: volver a abrir un formulario limpio y seleccionar Laboratorio 1 y la fecha de mañana.
6. Ingresar inicio 15:30, fin 14:00 y el mismo motivo.
7. Intentar enviar y comprobar la validación del intervalo incorrecto.
8. Revisar Mis solicitudes y verificar que tampoco se agregó un registro por la variante B.

Datos de Entrada:

Variante A: fecha de ayer, calculada al ejecutar; inicio 14:00; fin 15:30.

Variante B: fecha de mañana, calculada al ejecutar; inicio 15:30; fin 14:00.

Datos comunes: aula Laboratorio 1; motivo o asignatura Práctica de Programación.

Resultado Esperado:

En la variante A se informa que la fecha u hora ya pasó. En la variante B se informa que el intervalo es inválido.

Ninguno de los dos intentos crea una solicitud; la cantidad de registros permanece igual después de cada variante.

Condiciones de Validación:

Ejecutar ambas variantes por separado: la variante B usa una fecha futura para no mezclar el error de fecha con el de intervalo.

Si el formulario impide el envío, registrar la validación mostrada; el servidor también debe rechazar esos datos si recibe la solicitud.

Caso de Prueba 16: Consultar mis solicitudes (CU20)

Objetivo: Verificar que el profesor consulte solo sus solicitudes, con los estados pendiente, aprobada o rechazada y la respuesta del revisor cuando exista.

Precondiciones

Hay una sesión del profesor de prueba iniciada y la aplicación y la base de datos están disponibles.

El profesor tiene tres solicitudes identificables: una pendiente sin respuesta, una aprobada con respuesta y una rechazada con respuesta.

Otro profesor tiene una solicitud que debe quedar fuera de la consulta. Se conocen los datos originales de los cuatro registros.

Pasos:

1. Abrir Mis solicitudes con la sesión del profesor de prueba.
2. Localizar las tres solicitudes propias por sus fechas, horarios, aula y motivo.

3. Comprobar que la primera aparece pendiente y sin respuesta del revisor.

4. Comprobar que la segunda aparece aprobada y muestra la respuesta de aprobación preparada.

5. Comprobar que la tercera aparece rechazada y muestra la respuesta de rechazo preparada.

6. Verificar que la solicitud del otro profesor no aparece.

7. Recargar la consulta y comprobar que los registros conservan sus datos y estados originales.

Datos de Entrada:

Solicitudes propias en Laboratorio 1, de 14:00 a 15:30: mañana, pendiente; dentro de 2 días, aprobada; dentro de 3 días, rechazada. Fechas calculadas al preparar la prueba. Respuesta de la aprobada: «Aprobada para la práctica de Programación». Respuesta de la rechazada: «Solicitud rechazada por mantenimiento del aula».

Solicitud ajena: otro profesor, Laboratorio 1, dentro de 4 días, de 14:00 a 15:30.

Motivos distintos permiten identificar cada registro.

Resultado Esperado:

Se muestran exactamente las tres solicitudes propias con su estado y, en las decididas, la respuesta correspondiente del revisor.

No aparece la solicitud ajena y la consulta no modifica datos, estados ni respuestas.

Condiciones de Validación:

Repetir con un profesor sin solicitudes: se muestra una lista vacía sin registros de otros profesores.

Si falla la base de datos, se informa el problema y no se presentan datos ajenos ni se modifican solicitudes.

Gestion de riesgos

Categoría	Descripción del Riesgo	Factor Contextual / Causa	Probabilidad	Efecto
Personal / Proyecto	Limitación de tiempo por compromisos laborales	Emidio Machado y Lucas Borges combinan trabajo y estudio.	Alta	Grave
Proyecto / Formación	Inasistencia docente / Retraso en temas clave del proyecto	Falta de clases para avanzar en temas académicos requeridos para el SGRSI.	Moderada	Grave
Personal / Proyecto	Enfermedad o indisponibilidad de personal clave	Ausencia de un integrante en momentos críticos de entrega.	Moderada	Grave
Tecnológico / Producto	Rendimiento insuficiente de la base de datos de SGRSI	Fallas en el diseño o volumen de transacciones por segundo no soportado.	Moderada	Grave
Requerimientos	Cambios o ajustes en la especificación del sistema	Modificaciones en el alcance del proyecto SGRSI que demanden rediseño.	Moderada	Grave
Estimación	Subestimación del tiempo de desarrollo o tamaño del software	Cálculo optimista de la carga de trabajo de análisis, diseño o codificación.	Alta	Grave
Herramientas	Bajo rendimiento o falta de integración en herramientas CASE / IDE	Problemas de compatibilidad entre entornos de diseño UI y la base de datos.	Alta	Tolerable
Personal	Falta de capacitación previa en tecnologías específicas	Requerimiento de herramientas de desarrollo no dominadas previamente.	Moderada	Tolerable
Herramientas	Ineficiencia en código generado automáticamente	Uso de generadores de código con estándares bajos de calidad.	Moderada	Insignificante

Planificación y Estrategias de Mitigación:

Para cada riesgo clave se definen acciones específicas destinadas a minimizar su impacto en el desarrollo del SGRSI:

Riesgos de Disponibilidad Laboral y Académica

- **Limitación horaria por trabajo (E. Machado / L. Borges):** Establecer un cronograma flexible con hitos semanales claros. Redistribuir tareas de análisis (L. Borges) y diseño UI/UX (E. Machado) de forma asíncrona para maximizar el avance en horarios fuera de la jornada laboral.
- **Inasistencia docente / Retraso curricular:** Implementar sesiones de autoestudio guiado y aprendizaje autónomo entre los integrantes para cubrir los temas técnicos necesarios sin depender exclusivamente de las clases presenciales.

Riesgos de Personal y Estructura

- **Enfermedad o contingencia de un integrante:** Fomentar la polivalencia técnica mediante la documentación continua de los avances. Lucas Borges (Analista) y Facundo Pereyra (DBA) mantendrán coordinación constante sobre la estructura de la base de datos y los requerimientos para respaldar las labores de desarrollo en caso de contingencia.

Riesgos Tecnológicos y de Base de Datos

- **Rendimiento de la Base de Datos:** Aplicar técnicas rigurosas de normalización en las etapas iniciales de diseño de la base de datos del SGRSI e investigar componentes de motores de bases de datos de mayor rendimiento antes de la implementación final.

- **Integración de Herramientas:** Validar la compatibilidad del entorno de desarrollo y los plugins de modelado de datos/UI en las fases iniciales del proyecto.

Riesgos de Estimación y Requerimientos

- **Subestimación del tiempo o tamaño:** Priorizar el desarrollo de un Producto Mínimo Viable (PMV) del SGRSI con las funcionalidades indispensables. Recurrir a librerías y componentes preconstruidos de fiabilidad comprobada para optimizar el tiempo de desarrollo.
- **Cambio de requerimientos:** Aplicar modularidad y ocultamiento de información en la arquitectura del software para que los cambios en un módulo no afecten el resto del sistema.

Evaluación de funcionamiento grupal y mejoras

A lo largo del desarrollo de la segunda entrega para el proyecto del S.G.R.S.I, el equipo experimentó un avance significativo en su dinámica de trabajo en comparación con la primera etapa. El aprendizaje adquirido en su dinámica inicial permitió identificar falencias operativas y transformar el proceso metodológico en una experiencia de colaboración mucho más sólida, organizada y eficiente.

El Factor determinante para este cambio cualitativo fué el fortalecimiento de la comunicación interna. A diferencia de la primera fase, se proporcionaron espacios de reunión mucho más abiertos y participativos donde todos los integrantes tuvieron la oportunidad de proponer ideas , debatir alternativas técnicas y aportar al diseño del software. Este enfoque deliberativo no sólo enriqueció la toma de decisiones para el correcto avance, sino que también generó un consenso claro sobre los objetivos comunes y las propiedades de desarrollo.

Esta mejora en la comunicación impactó de manera directa en la organización y en el nivel de compromiso individual y colectivo:

- **Mayor organización:** La claridad alcanzada en los acuerdos de equipo facilitó una mejor distribución del trabajo, previniendo la duplicación de tareas y asegurando un flujo de avance constante en los módulos del sistema.
- **Compromiso renovado:** La participación activa de cada miembro en la fase de discusión incrementó el sentido de pertenencia con el proyecto, lo que se tradujo en un cumplimiento más riguroso de los plazos y en entregables de mayor calidad.

En conclusión, la evolución del grupo respecto a la primera entrega demuestra una adaptabilidad madura ante el trabajo colaborativo en ingeniería de software. La capacidad de transformar el espacio de reunión en un entorno receptivo y participativo fue la clave para

consolidar la estructura del proyecto SGRSI y asegurar un ritmo de trabajo sostenible y eficaz.

Estándares de codificación

El archivo docs/Regament de relevamiento de software.md establece normas para HTML, CSS, JavaScript, PHP y persistencia. La siguiente enumeración describe esas normas y su correspondencia con la implementación, sin asumir cumplimiento absoluto.

1. Nomenclatura. El reglamento prescribe kebab-case para archivos y carpetas, camelCase para variables y funciones JavaScript, PascalCase para clases y UPPER_SNAKE_CASE para constantes. En PHP existen archivos PascalCase y camelCase, como Conexion.php y guardarReporteDiario.php; por tanto, la regla universal de archivos requiere armonización con la práctica real.

2. Organización por capas. public contiene los puntos de entrada y recursos estáticos; app agrupa configuración, controladores, modelos, utilidades y vistas. La raíz pública debe configurarse en proyecto/public.

3. Tipado y formato PHP. Se utiliza declare(strict_types=1), clases y métodos con tipos. Un ejemplo es Conexion::obtener(): PDO. El reglamento exige tipado completo; no se presenta una auditoría automática de todo el repositorio como evidencia de cumplimiento.

4. HTML y accesibilidad. El reglamento solicita etiquetas semánticas, idioma español, textos alternativos y atributos apropiados para interacción. Estas reglas describen el estándar del equipo; no equivalen a una certificación de accesibilidad.

5. CSS. Se prescribe BEM y el uso de rem para tipografía y espaciados. La organización observada está separada por pantalla; se conserva la distinción entre norma escrita y aplicación en cada selector.

6. Comentarios. Los comentarios identifican responsabilidades y decisiones, por ejemplo la precedencia de configuración o la compatibilidad entre rutas. Se recomienda explicar el motivo de una decisión y mantener el comentario sincronizado con el comportamiento.

7. Acceso a datos. Las operaciones de persistencia utilizan PDO y parámetros enlazados. Los identificadores de tabla o columna no se deben tomar directamente de entradas del usuario.

8. Validación y errores. PeticionJson valida contenido JSON, campos, identificadores, textos, fechas y horas. Los controladores JSON utilizan RespuestaJson para éxitos y errores; los controladores de formularios recurren a mensajes y redirecciones. Hay excepciones de integración que se detallan en el capítulo 12.

9. Sesión, roles y CSRF. Las utilidades centralizan la sesión y la autorización; las operaciones JSON de recursos suelen exigir token CSRF. Los endpoints de sesión y la ruta de aulas presentan diferencias que se explican en la API.

10. Salida de datos. El reglamento exige escapar valores dinámicos; el proyecto dispone de escapar() en PHP y un auxiliar equivalente en api.js. Esta medida debe aplicarse al incorporar datos en HTML.

Repositorio utilizado

El código fuente, scripts de despliegue, documentación y modelos relacionales del sistema Ipsum están íntegramente centralizados y sincronizados en la plataforma GitHub Cloud, bajo la organización oficial del equipo. Esto asegura un registro histórico inmutable de toda la evolución del proyecto.

Ficha Técnica del Repositorio

Plataforma y Motor SCM	GitHub Cloud (SaaS) / Git Distributed Version Control System
Organización / Owner	ipsum-maker (ipsum.emp@gmail.com)
Nombre del Repositorio	Ipsum-ProyectoEgreso
URL Remota HTTPS	https://github.com/ipsum-maker/Ipsum-ProyectoEgreso
URL Remota SSH	git@github.com:ipsum-maker/Ipsum-ProyectoEgreso.git
Rama de Producción por Defecto	main

Características, Ventajas y Desventajas

Características Principales:

- **Control Distribuido:** Arquitectura distribuida donde cada programador dispone de una copia fiel con historial completo en su disco local.
- **GitHub Issues y Projects:** Sistema integrado para gestión de requerimientos, reporte de bugs y asignación de tareas a miembros del equipo.
- **Branch Protection Rules:** Capacidad de restringir commits directos sobre la rama main obligando a la creación de Pull Requests con code review.

- **Webhooks & API:** Disparadores automáticos configurables para notificaciones de eventos e integraciones futuras.

Ventajas Técnicas Evidenciadas:

- **Trabajo Desconectado:** Permite avanzar en la programación de vistas y lógica incluso durante desconexiones a internet en el instituto.
- **Trazabilidad Total:** Identificación infalible del autor, fecha, hora y contexto funcional de cada carácter de código mediante git blame.
- **Automatización del Flujo:** La vinculación nativa de palabras clave (Fixed #ID) automatiza el cierre y avance de tareas sin carga administrativa extra.
- **Seguridad y Copias de Seguridad:** Respaldo geográficamente redundante de alta disponibilidad que anula riesgos por fallos de hardware en equipos personales.

Desventajas y Mitigaciones:

- **Dependencia de Plataforma Externa:** Al tratarse de una solución propietaria en la nube, el equipo depende de la conectividad de los servidores de GitHub. Mitigación: Se aprovecha la naturaleza distribuida de Git para mantener repositorios locales completos y backups en discos externos.
- **Límites en Cuentas Gratuitas:** Los repositorios privados gratuitos imponen cuotas de almacenamiento (2 GB) y minutos de compilación. Mitigación: Uso estricto de .gitignore para no subir volcados de bases de datos pesados (.sql.bak) ni dependencias innecesarias.

Comparativa de Mercado y Justificación

Frente a otras alternativas destacadas del mercado, se evaluaron las siguientes dimensiones:

Plataforma	Fortalezas Principales	Debilidades	Veredicto del Equipo
GitHub	Máxima popularidad mundial, excelente integración con herramientas IDE y tableros Issues/Projects simples y potentes.	Plataforma SaaS cerrada, funciones avanzadas pagas.	**SELECCIONADA: Curva de aprendizaje óptima y estándar de la industria.**
GitLab	Suite completa con DevOps nativo muy potente y opción autoalojada (Self-Hosted).	Mayor consumo de recursos, interfaz más densa y compleja para equipos pequeños.	**Descartada: Excesiva complejidad para el alcance académico actual.**
Bitbucket	Fuerte integración histórica con Jira y productos de la suite Atlassian.	Comunidad abierta menor, menor ecosistema de extensiones y soporte en editores.	**Descartada: Menor agilidad para revisión de código estudiantil.**

Justificación Final: Se adoptó formalmente GitHub por su inmejorable balance entre usabilidad, estabilidad, integración inmediata con Visual Studio Code y por constituir el estándar indiscutido en la industria tecnológica uruguaya e internacional, permitiendo al docente Leandro López acceder, evaluar e interactuar de forma inmediata con el trabajo del equipo.

Gestion de ramas

Para coordinar el desarrollo simultáneo de los tres integrantes de Ipsum sin riesgo de sobrescribir código funcional ni romper la estabilidad del sistema, se implementó una metodología de ramificación basada en Git Flow combinada con Feature Branching (ramas por funcionalidad).

Modelo de Ramificación (Git Flow / Feature Branching)

El repositorio estructura formalmente tres ramas troncales permanentes y un conjunto dinámico de ramas de trabajo:

- **1. main:** Rama de Producción / Release Oficial. Aloja el código que ha superado todas las auditorías de seguridad, pruebas de integración y validaciones cruzadas. Cada entrega académica (como esta Tercera Entrega) se marca sobre main mediante un tag de versión semántica.
- **2. testing:** Rama de Integración Continua. Espacio de concurrencia donde convergen las ramas de características para verificar que no existan colisiones entre controladores, base de datos ni plantillas visuales antes del pase definitivo.
- **3. production:** Rama Espejo de Despliegue. Réplica congelada de la versión operativa en los servidores de demostración institucional, asegurando un entorno de aislamiento idéntico al que utiliza el usuario final.
- **4. feature/* y fix/*:** Ramas Efímeras de Funcionalidad. Ramas aisladas creadas puntualmente para desarrollar una característica o solucionar una falla específica asociada a un issue planificado.

Ciclo de Vida de Integración y Políticas de Merge

El flujo de trabajo acordado por el equipo sigue un ciclo riguroso de cuatro etapas:

1. **Paso 1: Bifurcación Aislada -** A partir de la rama testing debidamente actualizada, el desarrollador crea una rama de trabajo con nomenclatura descriptiva: git checkout -b feature/modulo-solicitudes o git checkout -b fix/alerta-error.
2. **Paso 2: Desarrollo y Verificación Local -** El programador implementa la lógica, estilos y persistencia en su estación local con XAMPP, realizando commits atómicos periódicos según las convenciones acordadas.
3. **Paso 3: Checklist Pre-Integración -** Previo a solicitar la integración, se realizan chequeos obligatorios: (a) Ausencia de errores de sintaxis en PHP con tipado estricto; (b) Verificación de integridad referencial en MySQL; (c) Comprobación de diseño responsivo y selectores BEM; (d) Inclusión obligatoria de tokens CSRF en formularios POST.
4. **Paso 4: Pull Request y Fusión -** Se publica la rama en GitHub mediante git push origin feature/... y se abre un Pull Request hacia testing. Otro miembro del equipo realiza la revisión de código (Code Review), evalúa posibles colisiones y autoriza el merge.

Resolución de Conflictos: En caso de que dos desarrolladores modifiquen simultáneamente un archivo común (por ejemplo, public/index.php o conexion.php), la política prohíbe el uso de comandos destructivos (como git push --force). Los conflictos se resuelven manualmente comparando los diffs en el editor, consensuando con los autores originales, y re-ejecutando las pruebas locales antes de consolidar el commit de resolución.

Ciclo Posterior al Merge: Una vez que el trabajo de la rama se fusionó con éxito y las pruebas de regresión son satisfactorias, la rama feature se elimina tanto local como remotamente (git branch -d y git push origin --delete) para mantener el árbol de trabajo limpio y libre de ramas huérfanas obsoletas.

Convenciones de commit y resolución de issues

Con el propósito de mantener un registro de cambios legible, uniforme y susceptible de ser auditado automáticamente, Ipsum adoptó una adaptación formal del estándar Conventional Commits combinada con el motor de vinculación automática de GitHub Issues.

Prefijos de Conventional Commits y Ejemplos Reales

Cada mensaje de confirmación se estructura formalmente como: tipo(ámbito opcional): descripción imperativa en minúsculas. A continuación, se presenta la tabla de prefijos normados con ejemplos extraídos directamente del historial real de commits de Ipsum:

Prefijo / Tipo	**Significado Técnico**	**Ejemplo Real en el Repositorio Ipsum**
feat	Incorporación de una nueva funcionalidad al sistema.	feat(mvc): integración completa de base de datos MySQL y seguridad CSRF
feat	Ampliación de funcionalidades transaccionales.	integracion de modulo de solicitudes de espacios y endpoints API REST
fix	Corrección de un fallo o error en el sistema.	actualización de la alerta y bug solucionado
docs	Creación o actualización de documentación técnica.	Actualización de Reglamento de relevamiento de software
refactor	Reorganización de código o carpetas sin alterar la lógica externa.	Reacomodo de carpetas y arquitectura de archivos
style	Ajustes de maquetación, selectores CSS o diseño BEM.	Diseño y funcionalidades aplicadas a seccion de Mis Reportes

Vinculación y Cierre Automático de Issues

El equipo utilizó sistemáticamente las palabras clave de cierre (Closing Keywords) soportadas por GitHub para articular el tablero de requerimientos con el avance de código:

- **Fixed #ID / Closes #ID:** Al fusionar el commit a la rama principal, GitHub cierra automáticamente el ticket referenciado, asociando la solución a los cambios de código. Evidencia real en commits: 'Hecho Estructura del Panel... Fixed #16', 'issues resuelto funcionalidad... Fixed #15', 'Estructura de Mis Reportes hecha. Fixed #18', 'Diseño Formulario Reporte Diario. Fixed #11'.
- **In progress #ID:** Permite relacionar avances intermedios sin clausurar el ticket, dejando constancia de trazabilidad para revisión docente. Evidencia real en commits: 'In progress: diseño y funcionalidad de panel administrador y panel coordinador #17'.

Criterios de Agrupación y Calidad del Historial

¿Cuándo se considera relevante realizar un commit? Un commit se realiza exclusivamente cuando conforma una unidad de trabajo lógica, atómica y autocompleta (por ejemplo, la creación de una tabla en la base de datos con su correspondiente modelo DAO, o la adición de una vista con su hoja de estilo). Se prohíben commits monolíticos que mezclen correcciones de estilos con migraciones de datos.

Criterio de Redacción: Los mensajes inician con un verbo imperativo en presente ('agrega', 'corrige', 'actualiza'), limitando la primera línea a menos de 72 caracteres y detallando con viñetas en el cuerpo del commit las modificaciones estructurales cuando el cambio reviste complejidad técnica.

Versionado

El proyecto adopta formalmente el estándar Semantic Versioning (SemVer 2.0.0) bajo el formato universal MAJOR.MINOR.PATCH (X.Y.Z), garantizando que la numeración refleje objetivamente el impacto arquitectónico de las modificaciones introducidas.

Estándar Semantic Versioning (SemVer 2.0.0)

- **MAJOR (X.0.0):** Cambios de ruptura o modificaciones estructurales mayores de la arquitectura. En Ipsum, representó la transición fundacional desde un prototipo estático dependiente de LocalStorage/SessionStorage hacia una arquitectura profesional Modelo-Vista-Controlador conectada a MySQL.
- **MINOR (X.Y.0):** Incorporación de nuevas características o módulos funcionales compatibles con las versiones anteriores. Ejemplos en Ipsum: incorporación de turnos y franjas horarias, módulo de préstamos y solicitudes de aulas, y panel de métricas analíticas.
- **PATCH (X.Y.Z):** Corrección de errores, parches de seguridad y optimizaciones menores que no alteran la interfaz pública ni rompen compatibilidad. Ejemplos en Ipsum: calibración de zona horaria America/Montevideo, afinamiento de mensajes visuales de alertas y corrección de validadores.

Cronología y Trazabilidad de Releases en Ipsum

El repositorio registra una evolución cronológica auditada mediante etiquetas formales de Git (tags). La siguiente tabla documenta la trazabilidad completa desde la génesis del sistema hasta la versión final de esta Tercera Entrega:

Versión (Tag)	Fecha / Hito	Descripción Técnica y Alcance del Release
v1.0.0	Hito Inicial	Estructura base de maquetación HTML5/CSS3 con paneles de Profesor, Coordinador y Administrador.
v1.1.0	Hito Funcional	Implementación del subsistema de control de errores y validaciones en interfaz de usuario.
v1.1.1	Parche	Ajustes cosméticos y correcciones menores en la vista de reporte-diario.
v1.2.1	Base de Datos	Avance de persistencia relacional y primeras conexiones en el flujo de inicio de sesión.
v1.3.1	Refactor	Reorganización de la jerarquía de carpetas y normalización de rutas de archivos.
v1.4.1	Infraestructura	Integración de base de datos vía phpMyAdmin con PHP XAMPP y control de fallas de conexión en login.
v1.5.1	Coordinación	Activación del alta de usuarios, consolidación de botones y renombre a Panel de Asistente.
v1.6.1	Integración	Implementación unificada de controladores PHP y manipuladores JS para formularios.

v1.7.1	Arquitectura MVC	Salto a arquitectura MVC pura con MySQL, eliminación definitiva de localStorage y blindaje CSRF.
v1.8.1	Lógica de Negocio	Incorporación de turnos, horarios, equipos editables, ventana de 10 min de edición y modal de solución.
v1.8.2	Esquema SQL	Actualización del esquema DDL con DROP TABLE ordenado y feedback visual reactivo.
v1.8.3	Parche Alertas	Depuración de bugs menores en visualización de notificaciones modales.
v1.8.4	Normalización 3FN	Reestructuración y saneamiento integral de la base de datos conforme a la Tercera Forma Normal.
v1.9.4	Analítica	Módulo de métricas de gestión institucional para el rol de administrador.
v1.10.4	Tercera Entrega	Release definitivo: API REST completa (/api/), módulo de solicitudes de laboratorios, disponibilidad y DAO.

Criterio de Tagging: Cada release se fija en el repositorio mediante etiquetas anotadas: git tag -a v1.10.4 -m "Sistema con APIs, modulo de prestamos y metricas", sincronizándose con GitHub mediante git push origin --tags.

Estructura de archivos y directorios del sistema

La organización jerárquica del código fuente de Ipsum adopta una arquitectura desacoplada basada en el patrón Modelo-Vista-Controlador (MVC), combinada con el

aislamiento estricto de la raíz pública del servidor web (DocumentRoot) para blindar el código de negocio frente a accesos no autorizados.

Árbol Jerárquico del Proyecto

A continuación, se detalla la disposición completa y actualizada de carpetas y ficheros de la aplicación:

```
Ipsum-ProyectoEgreso/
├── INICIAR-PROYECTO.bat            # Script de arranque automatico en entorno local
├── docs/                           # Documentacion tecnica, normativa y modelos
│   ├── Reglamento de relevamiento...  # Estandares de codificacion y normas del equipo
│   └── persistencia/               # Modelos relacionales (DER, MER y Normalizacion 3FN)
│       ├── der_diagrama.png        # Diagrama Entidad-Relacion fisico
│       ├── mer_diagrama.png        # Modelo Entidad-Relacion logico conceptual
│       └── NORMALIZACION.pdf       # Memoria formal de descomposicion a 3FN
└── proyecto/                       # Codigo fuente integral de la aplicacion
    ├── app/                        # LOGICA PRIVADA (Protegida fuera del alcance web directo)
    │   ├── config/
    │   │   └── Conexion.php        # Patron Singleton de conexion PDO a MySQL
    │   ├── controlador/            # Capa de Controladores de Negocio
    │   │   ├── administrador/      # Controladores de administracion (usuarios, metricas)
    │   │   ├── api/                # Controladores REST (Incidencias, Diarios, Solicitudes)
    │   │   ├── autenticacion/      # Controladores de inicio, cierre y redireccion
    │   │   ├── coordinacion/       # Controladores de revision, resolucion y estado
    │   │   └── profesor/           # Controladores de partes diarios, incidencias y solicitudes
    │   ├── modelo/                 # Capa de Modelos de Dominio y DAOs
    │   │   ├── AulaModelo.php      # Persistencia de catalogo de espacios fisicos
    │   │   ├── IncidenciaDAO.php   # Data Access Object para reportes de incidencias
    │   │   ├── MetricasModelo.php  # Calculo y agregacion estadistica de incidencias y aulas
    │   │   ├── ReporteDiarioDAO.php  # Data Access Object para sesiones de clase y puestos PC
    │   │   ├── SolicitudLaboratorioDAO.php # DAO de reservas y chequeo de disponibilidad
    │   │   └── UsuarioModelo.php   # Gestion y autenticacion de credenciales docentes
    │   ├── utilidades/             # Funciones auxiliares y seguridad transversal
```

```
│   │   ├── cedula.php          # Algoritmo de validacion Modulo 10 para cedula uruguaya
│   │   ├── PeticionJson.php    # Helper de lectura, decodificacion y auth para API REST
│   │   └── sesion.php          # Control RBAC de roles, timeouts y tokens CSRF
│   └── vista/                  # Plantillas de Renderizado y Respuestas
│       ├── RespuestaJson.php   # Formateador canonico de respuestas JSON para la API
│       ├── login.php           # Formulario de acceso autenticado
│       ├── administrador/      # Vistas de gestion de usuarios y tablero de metricas
│       ├── coordinacion/       # Vistas de revision de incidentes y solicitudes de aula
│       └── profesor/           # Vistas de llenado de partes, historial y solicitudes
├── database/
│   └── ipsum_reportes.sql      # Script DDL normalizado con tablas, FKs e indices
└── public/                     # RAIZ PUBLICA DEL SERVIDOR (DocumentRoot)
    ├── index.php               # Front Controller y enrutador principal del sistema web
    ├── login.php               # Punto de entrada de autenticacion
    ├── api/                    # Enrutador de la API REST
    │   ├── index.php           # Despachador unico de recursos REST (?r=...)
    │   └── *.php               # Pasarelas de compatibilidad (aulas, incidencias...)
    └── assets/                 # Recursos estaticos del lado del cliente
        ├── css/                # Hojas de estilo modulares bajo metodologia BEM
        └── js/                 # Scripts JavaScript nativos (api.js, validadores...)
```

Justificación de la Arquitectura y DocumentRoot

Aislamiento de Seguridad: En los servidores de producción y en la configuración local de Apache, la raíz del servidor (DocumentRoot) se fija exclusivamente en proyecto/public/. De este modo, un atacante o usuario externo jamás podrá descargar ni acceder por navegador a los archivos PHP de lógica sensible (/app), configuraciones de bases de datos (/config) ni scripts SQL (/database).

Patrón Front Controller: Toda navegación web se canaliza a través de public/index.php con el parámetro ?ruta=..., actuando como enrutador y verificador único de sesiones y privilegios antes de invocar al controlador correspondiente.

Delimitación Cliente / Servidor y Descripción por Capas

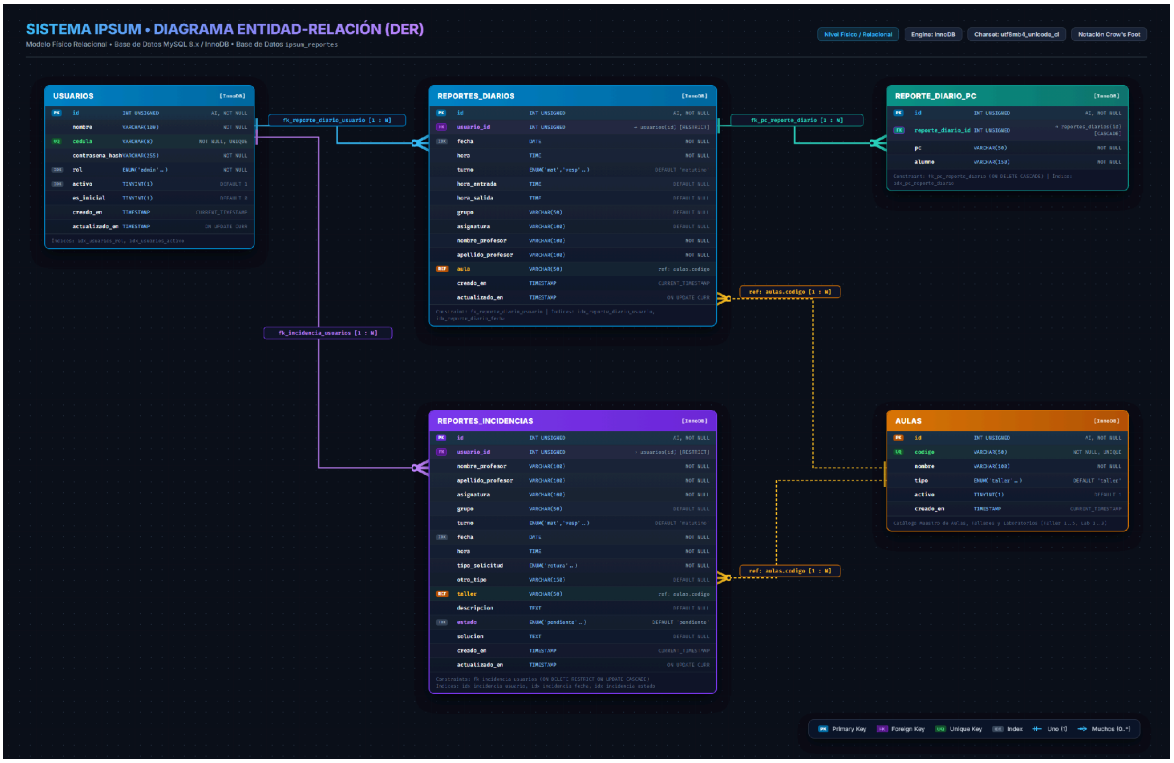
- **Código del Cliente (Frontend):** Reside exclusivamente en el navegador y en proyecto/public/assets/. Incluye las hojas de estilo CSS3 BEM y los scripts JavaScript nativos (api.js, control-errores.js). Se encarga de capturar eventos del usuario, ejecutar validaciones inmediatas en formularios (como la cédula uruguaya) y consumir asíncronamente la API mediante Fetch API.
- **Código del Servidor (Backend):** Reside en proyecto/app/ y proyecto/public/index.php. Se procesa enteramente en el servidor PHP 8.3 sin que el cliente tenga acceso a su código fuente ni a la conexión con MySQL.
- **Ubicación de Rutas:** Se centralizan en public/index.php (para vistas web) y public/api/index.php (para la API REST). Mapean las URLs hacia los controladores adecuados.
- **Lógica de Aplicación:** Encapsulada estrictamente en los controladores especializados de proyecto/app/controlador/ y controladores REST de proyecto/app/controlador/api/.

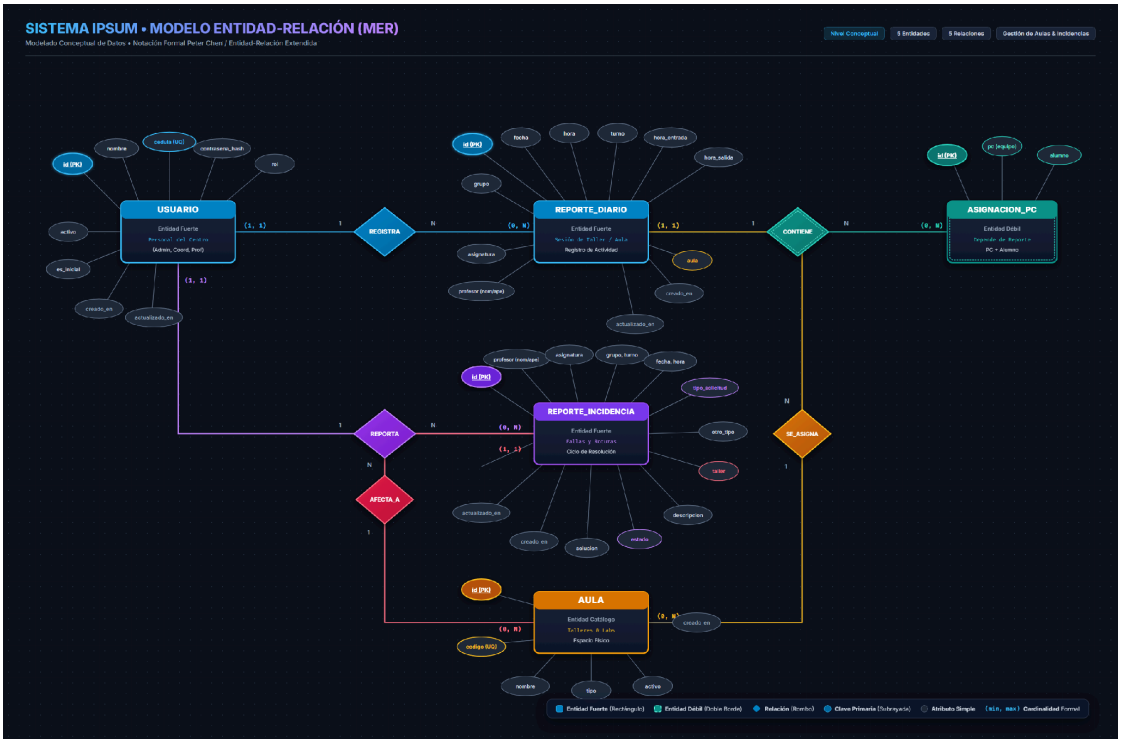
- **Acceso a Datos y Persistencia:** Aislada por completo en las clases DAO y Modelos de proyecto/app/modelo/, que gestionan sentencias SQL parametrizadas a través del conector PDO provisto por Conexion.php.

Persistencia

El subsistema de persistencia de Ipsum fue creado para garantizar integridad transaccional absoluta, rendimiento optimizado en consultas analíticas y eliminación total de redundancias no controladas.

Diagramas de Entidad-Relación (DER y MER)





Modelo Relacional Inicial (0FN) y Diagnóstico de Anomalías

|El punto de partida del diseño correspondió a la recolección de formularios administrativos planos utilizados históricamente en las libretas de partes y planillas de soporte del instituto, originando una tabla desnormalizada en Forma No Normalizada (0FN):

REPORTE_GENERAL_0FN (

id_reporte, cedula_profesor, nombre_profesor, rol_usuario, contrasena_hash, usuario_activo,

fecha, hora, turno, hora_entrada, hora_salida, grupo, asignatura, codigo_aula, nombre_aula, tipo_aula,

{ numero_pc, nombre_alumno } [MULTIVALUADO],

tipo_incidencia, otro_tipo_incidencia, descr_incidencia, estado_incidencia, solucion_incidencia

)

id_rep	cedula	profesor	aula	pcs_alumnos (Multivaluado)	estado
1	12345678	Elvis (Admin)	Taller 1	[PC1: Juan, PC2: Maria, PC3: Pedro]	NULL
2	23456789	Pepita (Coord)	Lab 1	[PC5: Lucas]	pendiente
3	34567890	Pepito (Prof)	Taller 2	[PC1: Ana, PC2: Carlos, PC4: Sofia]	NULL

- **1. Violación de Atomicidad:** El atributo pcs_alumnos agrupa colecciones de valores en una sola celda, violando el principio de atomicidad de Codd.
- **2. Anomalía de Inserción:** Es imposible dar de alta a un docente o crear un laboratorio sin que exista simultáneamente una clase impartida o una avería reportada.
- **3. Anomalía de Modificación:** Si un docente rectifica su nombre o un taller cambia de denominación, se requiere actualizar potencialmente cientos de registros históricos redundantes, arriesgando inconsistencia.
- **4. Anomalía de Eliminación:** Si se borra un parte diario antiguo, se elimina la única referencia sobre la existencia del docente o del espacio físico.

Proceso Formal de Normalización (1FN → 2FN → 3FN)

Paso a Primera Forma Normal (1FN):

Reglas: (1) Atomicidad estricta en cada columna; (2) Eliminación de grupos repetitivos descomponiéndose en filas independientes; (3) Definición de una clave primaria compuesta unívoca: (id_reporte, numero_pc).

Resultado 1FN: La lista de puestos se aplanó en registros independientes. Sin embargo, persisten Dependencias Parciales: los atributos del docente (nombre, rol) y de la sesión (fecha, hora, aula) dependen únicamente de id_reporte o cedula, y no de la clave primaria completa (id_reporte, numero_pc).

Paso a Segunda Forma Normal (2FN):

Reglas: El esquema se encuentra en 1FN y cada atributo no clave depende funcionalmente de la TOTALIDAD de la clave primaria, desacoplando entidades independientes mediante dependencias funcionales:

- **DF1 (Usuarios):** cedula → nombre, contrasena_hash, rol, activo, es_inicial.
- **DF2 (Reportes Diarios):** id_reporte → usuario_id, fecha, hora, turno, hora_entrada, hora_salida, grupo, asignatura, aula.
- **DF3 (Asignaciones PC):** (reporte_diario_id, numero_pc) → alumno.
- **DF4 (Incidencias):** id_incidencia → usuario_id, asignatura, grupo, turno, fecha, hora, tipo_solicitud, taller, descripcion, estado, solucion.

Diagnóstico 2FN: Aunque se eliminaron las dependencias parciales, en reportes diarios e incidencias los datos del aula dependen del código de aula (aula_id), originando Dependencias Transitivas: id_reporte → aula_id → (nombre_aula, tipo_aula).

Paso a Tercera Forma Normal (3FN):

Reglas: El esquema se encuentra en 2FN y ningún atributo no clave depende transitivamente de otro atributo no clave (para toda dependencia funcional no trivial X → Y, X es superclave).

Solución 3FN: Se desacopla la entidad maestra independiente aulas (id, codigo, nombre, tipo, activo). En las tablas transaccionales reportes_diarios, reportes_incidencias y solicitudes_laboratorio, el espacio se referencia mediante la clave foránea aula_id vinculada a aulas(id).

Esquema Relacional Final Normalizado en 3FN

A continuación, se define el esquema relacional definitivo en 3FN indicando claves primarias (PK), claves foráneas (FK) y restricciones de integridad referencial:

1. **1.** usuarios (id [PK], nombre, cedula [UQ], contrasena_hash, rol, activo, es_inicial, creado_en, actualizado_en)

   Restricciones: cedula es valor único no nulo; rol limitado a ENUM('administrador', 'coordinador', 'profesor').
2. **2.** aulas (id [PK], codigo [UQ], nombre, tipo, activo, creado_en)

   Restricciones: codigo es identificador único alfanumérico; tipo limitado a ENUM('taller', 'laboratorio', 'aula').
3. **3.** reportes_diarios (id [PK], usuario_id [FK], aula_id [FK], fecha, hora, turno, hora_entrada, hora_salida, grupo, asignatura, creado_en, actualizado_en)

   Restricciones: usuario_id REFERENCES usuarios(id) ON DELETE RESTRICT ON UPDATE CASCADE; aula_id REFERENCES aulas(id) ON DELETE RESTRICT ON UPDATE CASCADE.

4. **4.** reporte_diario_pc (id [PK], reporte_diario_id [FK], pc, alumno)

   Restricciones: reporte_diario_id REFERENCES reportes_diarios(id) ON DELETE CASCADE ON UPDATE CASCADE. Asegura que la eliminación de un parte limpie automáticamente sus puestos asignados.

5. **5.** reportes_incidencias (id [PK], usuario_id [FK], aula_id [FK], asignatura, grupo, turno, fecha, hora, tipo_solicitud, otro_tipo, descripcion, codigo_equipo, estado, solucion, creado_en, actualizado_en)

   Restricciones: usuario_id REFERENCES usuarios(id) ON DELETE RESTRICT; aula_id REFERENCES aulas(id) ON DELETE RESTRICT; estado ENUM('pendiente', 'en_proceso', 'resuelto').

6. **6.** solicitudes_laboratorio (id [PK], usuario_id [FK], aula_id [FK], fecha, hora_inicio, hora_fin, motivo, estado, respuesta, revisor_id [FK], creado_en, decidido_en)

   Restricciones: usuario_id y revisor_id REFERENCES usuarios(id) ON DELETE RESTRICT; aula_id REFERENCES aulas(id) ON DELETE RESTRICT; estado ENUM('pendiente', 'aprobada', 'rechazada').

Matriz de Prevención de Anomalías

Operación CRUD	Riesgo en Estado No Normalizado (0FN)	Garantía en Esquema Normalizado (3FN)
Alta de Docente	Imposible ingresar un profesor sin registrar forzosamente una clase o un reporte de fallo técnico.	Resuelto: Se inserta de forma autónoma e independiente en la tabla usuarios con rol y clave hash.
Alta de Aula o Lab	Imposible crear un nuevo laboratorio sin que exista una sesión previa o avería asignada en él.	Resuelto: Se crea directamente en la tabla catálogo aulas con su código único, tipo y estado.
Modificación de Datos	Modificar el nombre de un profesor o aula obligaba a actualizar cientos de tuplas con riesgo de inconsistencia.	Resuelto: Se actualiza un único registro atómico en usuarios.nombre o aulas.nombre.
Baja de Reporte Diario	Borrar un parte de clase provocaba la pérdida catastrófica de los datos del docente y del aula física.	Resuelto: El borrado elimina en cascada únicamente las tuplas hijas en reporte_diario_pc, preservando usuarios y aulas.
Gestión de Equipos	El arreglo [PC1: Juan, PC2: Maria] impedía realizar consultas indexadas o estadísticas por máquina.	Resuelto: Se ejecutan consultas SQL estándar (COUNT, WHERE pc = 'PC1') con rendimiento indexado.

Punteo de Operaciones en Álgebra Relacional

A continuación, se formalizan cinco consultas fundamentales del sistema mediante la notación canónica del Álgebra Relacional:

- **Consulta 1 - Docentes Habilitados:** Obtener la cédula y nombre de todos los docentes que se encuentren activos en el sistema.

  Operación Algebraica: $\pi_{cedula, nombre} (\sigma_{rol = 'profesor' \wedge activo = 1} (usuarios))$

- **Consulta 2 - Incidencias Activas:** Listar el código de espacio, tipo de avería, fecha y estado de las incidencias que aún no han sido resueltas.

  Operación Algebraica: π_{a.codigo, i.tipo_solicitud, i.fecha, i.estado} (σ_{i.estado ≠ 'resuelto'} (reportes_incidencias i ⋈_{i.aula_id = a.id} aulas a))

- **Consulta 3 - Partes Diarios en Taller 1:** Consultar las sesiones de clase del turno matutino en el Taller 1, detallando docente, fecha y asignatura.

  Operación Algebraica: π_{u.nombre, r.fecha, r.grupo, r.asignatura} (σ_{a.codigo = 'taller1' ∧ r.turno = 'matutino'} ((reportes_diarios r ⋈_{r.aula_id = a.id} aulas a) ⋈_{r.usuario_id = u.id} usuarios u))

- **Consulta 4 - Ocupación de Equipos:** Recuperar las computadoras y alumnos asignados en el reporte de clase con identificador #15.

  Operación Algebraica: π_{pc, alumno} (σ_{reporte_diario_id = 15} (reporte_diario_pc))

- **Consulta 5 - Solicitudes Pendientes:** Listar las solicitudes de laboratorio pendientes de revisión junto al nombre del profesor solicitante y aula.

  Operación Algebraica: π_{u.nombre, a.nombre, s.fecha, s.hora_inicio, s.hora_fin, s.motivo} (σ_{s.estado = 'pendiente'} ((solicitudes_laboratorio s ⋈_{s.usuario_id = u.id} usuarios u) ⋈_{s.aula_id = a.id} aulas a))

Consultas DQL (Data Query Language)

```
-- DQL 1: Docentes Habilitados
SELECT cedula, nombre
FROM usuarios
```

```
WHERE rol = 'profesor' AND activo = 1
ORDER BY nombre ASC;


-- DQL 2: Incidencias no resueltas con detalle de espacio fisico
SELECT a.codigo AS aula_codigo, i.tipo_solicitud, i.codigo_equipo, i.fecha, i.estado
FROM reportes_incidencias i
INNER JOIN aulas a ON i.aula_id = a.id
WHERE i.estado != 'resuelto'
ORDER BY i.fecha DESC, i.hora DESC;


-- DQL 3: Partes diarios del turno matutino en Taller 1
SELECT u.nombre AS profesor, r.fecha, r.hora_entrada, r.hora_salida, r.grupo, r.asignatura
FROM reportes_diarios r
INNER JOIN aulas a ON r.aula_id = a.id
INNER JOIN usuarios u ON r.usuario_id = u.id
WHERE a.codigo = 'taller1' AND r.turno = 'matutino'
ORDER BY r.fecha DESC;


-- DQL 4: Alumnos y PCs asignadas para el reporte #15
SELECT pc, alumno
FROM reporte_diario_pc
WHERE reporte_diario_id = 15
ORDER BY pc ASC;


-- DQL 5: Solicitudes de laboratorio pendientes de aprobacion
SELECT s.id, u.nombre AS docente, a.nombre AS laboratorio, s.fecha, s.hora_inicio, s.hora_fin, s.motivo
FROM solicitudes_laboratorio s
INNER JOIN usuarios u ON s.usuario_id = u.id
INNER JOIN aulas a ON s.aula_id = a.id
WHERE s.estado = 'pendiente'
ORDER BY s.fecha ASC, s.hora_inicio ASC;
```

Sentencias DML (Data Manipulation Language)

```
-- DML 1 (INSERT): Registro de nuevo parte de clase en Taller 2
INSERT INTO reportes_diarios
(usuario_id, aula_id, fecha, hora, turno, hora_entrada, hora_salida, grupo, asignatura)
VALUES (3, 2, '2026-10-23', '08:30:00', 'matutino', '08:30:00', '10:00:00', '3°MC', 'Programación Full Stack');

-- DML 2 (INSERT): Asignacion de equipos en cascada para el parte recién creado (ID 20)
INSERT INTO reporte_diario_pc (reporte_diario_id, pc, alumno) VALUES
(20, 'PC1', 'Lucas Borges'),
(20, 'PC2', 'Facundo Pereyra');

-- DML 3 (UPDATE): Resolucion de incidencia tecnica por el asistente de coordinacion
UPDATE reportes_incidencias
SET estado = 'resuelto', solucion = 'Se reemplazo el cable de red RJ-45 dañado. Conectividad 100% restablecida.'
WHERE id = 8 AND estado = 'en_proceso';

-- DML 4 (DELETE): Cancelacion controlada de un parte diario
-- La restriccion ON DELETE CASCADE se encarga de eliminar de forma atomica sus registros en
reporte_diario_pc
DELETE FROM reportes_diarios WHERE id = 12;
```

Script DDL (Data Definition Language)

El siguiente extracto reproduce el script DDL consolidado de proyecto/database/ipsum_reportes.sql, definiendo la totalidad de tablas, tipos enumerados, índices y restricciones transaccionales:

```
CREATE DATABASE IF NOT EXISTS ipsum CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE ipsum;

DROP TABLE IF EXISTS solicitudes_laboratorio;
DROP TABLE IF EXISTS reporte_diario_pc;
DROP TABLE IF EXISTS reportes_diarios;
DROP TABLE IF EXISTS reportes_incidencias;
DROP TABLE IF EXISTS aulas;
DROP TABLE IF EXISTS usuarios;

CREATE TABLE usuarios (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    cedula VARCHAR(8) NOT NULL UNIQUE,
    contrasena_hash VARCHAR(255) NOT NULL,
    rol ENUM('administrador', 'coordinador', 'profesor') NOT NULL,
    activo TINYINT(1) NOT NULL DEFAULT 1,
    es_inicial TINYINT(1) NOT NULL DEFAULT 0,
    creado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    actualizado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    INDEX idx_usuarios_rol (rol),
    INDEX idx_usuarios_activo (activo)
) ENGINE=InnoDB;

CREATE TABLE aulas (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    codigo VARCHAR(50) NOT NULL UNIQUE,
    nombre VARCHAR(100) NOT NULL,
```

```
    tipo ENUM('taller', 'laboratorio', 'aula') NOT NULL DEFAULT 'taller',
    activo TINYINT(1) NOT NULL DEFAULT 1,
    creado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE reportes_diarios (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    usuario_id INT UNSIGNED NOT NULL,
    aula_id INT UNSIGNED NOT NULL,
    fecha DATE NOT NULL,
    hora TIME NOT NULL,
    turno ENUM('matutino', 'vespertino', 'nocturno') NOT NULL DEFAULT 'matutino',
    hora_entrada TIME DEFAULT NULL,
    hora_salida TIME DEFAULT NULL,
    grupo VARCHAR(50) DEFAULT NULL,
    asignatura VARCHAR(100) DEFAULT NULL,
    creado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    actualizado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    CONSTRAINT fk_reporte_diario_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON
DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_reporte_diario_aula FOREIGN KEY (aula_id) REFERENCES aulas(id) ON DELETE
RESTRICT ON UPDATE CASCADE,
    INDEX idx_reporte_diario_fecha (fecha)
) ENGINE=InnoDB;

CREATE TABLE reporte_diario_pc (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    reporte_diario_id INT UNSIGNED NOT NULL,
    pc VARCHAR(50) NOT NULL,
    alumno VARCHAR(150) NOT NULL,
    CONSTRAINT fk_pc_reporte_diario FOREIGN KEY (reporte_diario_id) REFERENCES reportes_diarios(id)
ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

```
CREATE TABLE reportes_incidencias (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    usuario_id INT UNSIGNED NOT NULL,
    aula_id INT UNSIGNED NOT NULL,
    asignatura VARCHAR(100) NOT NULL,
    grupo VARCHAR(50) DEFAULT NULL,
    turno ENUM('matutino', 'vespertino', 'nocturno') NOT NULL DEFAULT 'matutino',
    fecha DATE NOT NULL,
    hora TIME NOT NULL,
    tipo_solicitud ENUM('rotura-equipo', 'falla-equipo', 'otro') NOT NULL,
    otro_tipo VARCHAR(150) DEFAULT NULL,
    descripcion TEXT DEFAULT NULL,
    codigo_equipo VARCHAR(50) DEFAULT NULL,
    estado ENUM('pendiente', 'en_proceso', 'resuelto') NOT NULL DEFAULT 'pendiente',
    solucion TEXT DEFAULT NULL,
    creado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    actualizado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    CONSTRAINT fk_incidencia_usuarios FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE
RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_incidencia_aula FOREIGN KEY (aula_id) REFERENCES aulas(id) ON DELETE RESTRICT
ON UPDATE CASCADE,
    INDEX idx_incidencia_estado (estado),
    INDEX idx_incidencia_equipo (codigo_equipo)
) ENGINE=InnoDB;

CREATE TABLE solicitudes_laboratorio (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    usuario_id INT UNSIGNED NOT NULL,
    aula_id INT UNSIGNED NOT NULL,
    fecha DATE NOT NULL,
    hora_inicio TIME NOT NULL,
    hora_fin TIME NOT NULL,
```

```
    motivo TEXT NOT NULL,
    estado ENUM('pendiente', 'aprobada', 'rechazada') NOT NULL DEFAULT 'pendiente',
    respuesta TEXT DEFAULT NULL,
    revisor_id INT UNSIGNED DEFAULT NULL,
    creado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    decidido_en TIMESTAMP NULL DEFAULT NULL,
    CONSTRAINT fk_solicitud_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE
RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_solicitud_aula FOREIGN KEY (aula_id) REFERENCES aulas(id) ON DELETE RESTRICT
ON UPDATE CASCADE,
    CONSTRAINT fk_solicitud_revisor FOREIGN KEY (revisor_id) REFERENCES usuarios(id) ON DELETE
RESTRICT ON UPDATE CASCADE,
    INDEX idx_solicitud_disponibilidad (aula_id, fecha, estado, hora_inicio, hora_fin)
) ENGINE=InnoDB;
```

API REST

La API REST de Ipsum constituye la espina dorsal para la comunicación asíncrona entre el cliente web (frontend) y la lógica de negocio (backend). Se diseñó siguiendo los principios arquitectónicos de Roy Fielding, empleando JSON como formato canónico y respetando la semántica de verbos HTTP.

Propósito, Arquitectura y Protocolo

- **Propósito Principal:** Permitir que el frontend interactúe en tiempo real (consultar disponibilidad de salones, cambiar estados de incidencias a 'en proceso', actualizar gráficos de métricas) sin necesidad de recargar la página completa.
- **Desacoplamiento Total:** El cliente no interactúa directamente con la base de datos ni con los modelos. Se comunica exclusivamente a través de solicitudes HTTP enviadas a public/api/index.php, procesadas por controladores REST que devuelven respuestas JSON uniformes.
- **Protocolo y Seguridad:** Toda petición que no sea GET viaja con cabeceras Content-Type: application/json y el token de seguridad X-CSRF-Token, impidiendo ataques de falsificación de peticiones en sitios cruzados.

Catálogo de Recursos, Endpoints y Métodos HTTP

Recurso / Endpoint	Métodos HTTP	Descripción de la Operación	Control de Acceso
/api/sesion	**GET, POST, DELETE**	Verificación del usuario activo, autenticación y cierre de sesión seguro.	Público / Autenticado

/api/usuarios	**GET, POST, PUT, PATCH, DELETE**	Listado, alta, edición, suspensión y restablecimiento de credenciales de docentes.	Rol Administrador
/api/aulas	**GET**	Catálogo completo de talleres, laboratorios y salones activos del centro.	Todos los roles autenticados
/api/incidencias	**GET, POST, PUT, PATCH, DELETE**	Reporte de averías por docentes, cambio de estado ('en proceso') y registro de solución.	Profesor / Coordinación / Admin
/api/reportes-diarios	**GET, POST, PUT, PATCH**	Registro y consulta de partes diarios de clase con detalle individual de computadoras.	Profesor / Coordinación / Admin
/api/solicitudes-laboratorio	**GET, POST, PUT, PATCH**	Gestión de reservas de salas especiales, registro de peticiones y dictamen de coordinación.	Profesor / Coordinación
/api/solicitudes-laboratorio/disponibilidad	**GET**	Algoritmo de cálculo de superposición horaria para verificar disponibilidad de salones.	Todos los roles autenticados
/api/metricas	**GET**	Cálculo analítico consolidado: porcentaje de resolución, averías por aula y uso de talleres.	Rol Administrador

Catálogo de Recursos, Endpoints y Métodos HTTP

Recurso / Endpoint	**Métodos HTTP**	**Descripción de la Operación**	**Control de Acceso**

/api/sesion	**GET, POST, DELETE**	Verificación del usuario activo, autenticación y cierre de sesión seguro.	Público / Autenticado
/api/usuarios	**GET, POST, PUT, PATCH, DELETE**	Listado, alta, edición, suspensión y restablecimiento de credenciales de docentes.	Rol Administrador
/api/aulas	**GET**	Catálogo completo de talleres, laboratorios y salones activos del centro.	Todos los roles autenticados
/api/incidencias	**GET, POST, PUT, PATCH, DELETE**	Reporte de averías por docentes, cambio de estado ('en proceso') y registro de solución.	Profesor / Coordinación / Admin
/api/reportes-diarios	**GET, POST, PUT, PATCH**	Registro y consulta de partes diarios de clase con detalle individual de computadoras.	Profesor / Coordinación / Admin
/api/solicitudes-laboratorio	**GET, POST, PUT, PATCH**	Gestión de reservas de salas especiales, registro de peticiones y dictamen de coordinación.	Profesor / Coordinación
/api/solicitudes-laboratorio/disponibilidad	**GET**	Algoritmo de cálculo de superposición horaria para verificar disponibilidad de salones.	Todos los roles autenticados
/api/metricas	**GET**	Cálculo analítico consolidado: porcentaje de resolución, averías por aula y uso de talleres.	Rol Administrador

Introducción al servidor

El objetivo es centralizar la administración del servidor mediante automatización, control de acceso y copias de seguridad. La infraestructura se apoya en Linux con Apache, PHP-FPM y MariaDB. El sistema cuenta con scripts optimizados, un panel de control y un plan de contingencia con respaldos externos para garantizar la continuidad del servicio.

Organización de la tercera entrega

La implementación se divide en los siguientes componentes principales:

- **Bloque 1: Script principal del servidor.** Centraliza mediante un diseño modular las tareas de administración de usuarios, grupos, respaldos, redes, bases de datos, cortafuegos (*firewall*) y registros del sistema (*logs*).
- **Bloque 2: Despliegue de aplicaciones.** Detalla el entorno de ejecución implementado actualmente y señala las diferencias principales frente al enfoque basado en contenedores exigido en la letra del proyecto.
- Bloque 3: Servidor de respaldo. Especifica la estrategia de almacenamiento alternativo, el cifrado o transferencia segura de archivos y los pasos para la recuperación del sistema. La instalación definitiva del servidor secundario se registrará una vez que se lleve a cabo.

Objetivos y alcance

Objetivo general

Implementar una infraestructura de administración para el servidor SGRSI que facilite el mantenimiento, la supervisión de servicios, la seguridad, la automatización de tareas y la recuperación de información.

Objetivos específicos

- Centralizar las operaciones de administración mediante un script principal.
- Organizar las funciones en módulos independientes y reutilizables.
- Gestionar usuarios, grupos y permisos de acceso.
- Facilitar la ejecución y el seguimiento de las rutinas de respaldo.
- Incorporar herramientas de consulta y verificación de red.
- Gestionar de forma automatizada la base de datos MariaDB y sus procesos de *dump* y restauración.
- Consultar y gestionar las reglas del firewall.
- Inspeccionar los registros del sistema y los errores de los servicios.
- Ejecutar SGRSI mediante Apache, PHP-FPM y MariaDB.
- Establecer una estrategia de contingencia para conservar copias fuera del servidor principal.

Alcance de la implementación

La infraestructura actual se encuentra instalada directamente sobre Ubuntu Server.

La aplicación se encuentra en /var/www/html/sgrsi/.

El directorio utilizado por apache es /var/www/html/sgrsi/public/.

Los scripts de administración se encuentran en /scripts/.

El respaldo integral se encuentra en /home/adminipsum/**respaldo.sh**

El directorio de destino de los respaldos es /home/adminipsum/respaldos/

El informe distingue con claridad entre las funciones y servicios que ya han sido verificados y aquellos procedimientos que aún precisan implementación o validaciones adicionales.

Entorno de instalación y arquitectura

Sistema operativo

El servidor utiliza Ubuntu Server como plataforma de administración y ejecución de SGRSI.

Para consultar la información del sistema operativo se utilizó cat /etc/os-release.

```
adminipsum@iti-ipsum-2026:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04 (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
```

Para consultar la versión del kernel se utilizó uname -r.

```
adminipsum@iti-ipsum-2026:~$ uname -r
7.0.0-30-generic
```

Para consultar la versión del servidor web se utilizó apache2 -v.

```
adminipsum@iti-ipsum-2026:~$ apache2 -v
Server version: Apache/2.4.66 (Ubuntu)
Server built:   2026-10-08T19:55:44
```

Para consultar las versiones de PHP y MariaDB se utilizó php -v && mariadb --version

```
adminipsum@iti-ipsum-2026:~$ php -v && mariadb --version
PHP 8.5.4 (cli) (built: Oct 19 2026 14:35:18) (NTS)
Copyright (c) The PHP Group
Built by Ubuntu
Zend Engine v4.5.4, Copyright (c) Zend Technologies
    with Zend OPcache v8.5.4, Copyright (c), by Zend Technologies
mariadb from 11.8.6-MariaDB, client 15.2 for debian-linux-gnu (x86_64) using  EditLine wrapper
```

Las versiones registradas durante las verificaciones fueron:

Ubuntu Server	**26.04 LTS**
Apache HTTP Server	**2.4.66**
PHP	**8.5.4**
MariaDB	**11.8.6**

Arquitectura del servidor

La arquitectura actual se compone de los siguientes elementos:

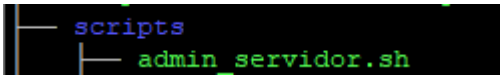
- **Ubuntu Server:** sistema operativo principal.
- **Apache HTTP Server:** Gestiona y atiende las peticiones web entrantes.
- **PHP-FPM:** ejecución de los scripts PHP.
- **MariaDB:** Ofrece el soporte para el almacenamiento, gestión y consulta de datos relacionales.
- **Scripts administrativos:** automatización de las tareas habituales de mantenimiento.
- **Directorio de respaldos:** Almacena de forma interna las copias de seguridad generadas.
- **Cron:** Programa y automatiza la ejecución periódica del plan de respaldo integral.

Script final de administración del servidor

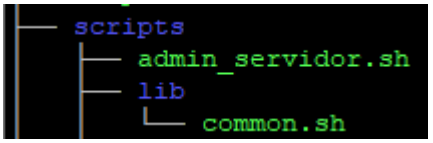
Estructura modular

Con el fin de centralizar la administración del sistema, se ha dispuesto un menú principal que facilita el acceso modular a cada una de sus funciones.

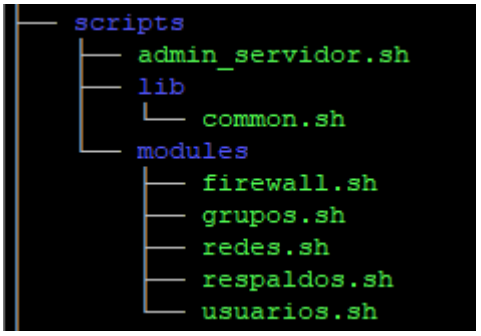
El script principal se encuentra en /scripts/admin_servidor.sh



La biblioteca común está ubicada en /scripts/lib/**common.sh**



Los módulos independientes se encuentran en /scripts/modules/



Esta distribución permite separar las tareas administrativas, evitando concentrar todas las operaciones en un único archivo.

Utilizar módulos facilita el mantenimiento y permite corregir o ampliar un módulo sin tener que modificar toda la estructura del menú principal.

Consulta la estructura de los scripts

Para mostrar los archivos de administración instalados se utilizó sudo find /scripts -maxdepth 3 -type f -print | sort

```
adminipsum@iti-ipsum-2026:~$ sudo find /scripts -maxdepth 3 -type f -print | sort
/scripts/admin_servidor.sh
/scripts/lib/common.sh
/scripts/modules/base_datos.sh
/scripts/modules/firewall.sh
/scripts/modules/grupos.sh
/scripts/modules/logs.sh
/scripts/modules/redes.sh
/scripts/modules/respaldos.sh
/scripts/modules/respaldos.sh.bak_20261021_010403
/scripts/modules/respaldos.sh.bak_20261021_030652
/scripts/modules/usuarios.sh
adminipsum@iti-ipsum-2026:~$
```

Para consultar los permisos del directorio se utilizó sudo ls -la /scripts/

```
adminipsum@iti-ipsum-2026:~$ sudo ls -la /scripts/
total 20
drwxr-xr-x  4 root root 4096 Oct 21 16:36 .
drwxr-xr-x 23 root root 4096 Sep 30 13:03 ..
-rwxr-xr-x  1 root root 1235 Oct 21 16:36 admin_servidor.sh
drwxr-xr-x  2 root root 4096 Sep 29 23:26 lib
drwxr-xr-x  2 root root 4096 Oct 21 18:01 modules
adminipsum@iti-ipsum-2026:~$
```

Para consultar el contenido del menú principal se utilizó sudo sed -n '1,220p' /scripts/admin_servidor.sh

```
adminipsum@iti-ipsum-2026:~$ sudo sed -n '1,220p' /scripts/admin_servidor.sh
#!/bin/bash

source /scripts/lib/common.sh

while true
do
        clear
        mostrar_titulo "Administración del servidor"
        echo "1- Gestion de usuarios"
        echo "2- Gestion de grupos"
        echo "3- Gestion de respaldos"
        echo "4- Gestion de redes"
        echo "5- Gestion de base de datos"
        echo "6- Gestion de firewall"
        echo "7- Gestion de logs"
        echo "0- Salir"

        echo
        read -p "Seleccione una opcion del 0-7: " opcion

        case $opcion in
                1)
                  /scripts/modules/usuarios.sh
                  ;;
                2)
                  /scripts/modules/grupos.sh
                  ;;
                3)
                  /scripts/modules/respaldos.sh
                  ;;
                4)
                  /scripts/modules/redes.sh
                  ;;
                5)
                  /scripts/modules/base_datos.sh
                  ;;
                6)
                  /scripts/modules/firewall.sh
                  pausa
                  ;;
                7)
                  /scripts/modules/logs.sh
                  pausa
                  ;;
                0)
                  echo
                  mostrar_ok "Saliendo del sistema de administracion"
                  exit 0
                  ;;
                *)
                  mostrar_error "Opcion no valida"
                  pausa
                  ;;
        esac
```

Función de la biblioteca common

La biblioteca common.sh centraliza las funciones auxiliares que emplean los distintos módulos administrativos, reduciendo la duplicación de código y estandarizando las operaciones recurrentes.

Entre sus principales responsabilidades destacan:

- Presentación de títulos y separadores de menú.
- Mensajes de información, advertencia, éxito y error.
- Validaciones comunes de entrada.
- Funciones de espera o pausa entre operaciones.
- Comprobaciones relacionadas con los privilegios necesarios.

Los módulos vinculan la biblioteca de manera dinámica a través del comando source, calculando su ruta relativa en tiempo de ejecución. Este mecanismo evita la duplicación de código y optimiza el mantenimiento de las funciones compartidas.

Archivo common.sh:

```
adminipsum@iti-ipsum-2026:~$ sudo cat scripts/lib/common.sh
#!/bin/bash

mostrar_titulo() {
        echo "========================================="
        echo " $1"
        echo "========================================="
}

mostrar_ok() {
        echo "[OK] $1"
}

mostrar_error() {
        echo "[ERROR] $1"
}

mostrar_aviso() {
        echo "[AVISO] $1"
}

pausa() {
        echo
        read -p "Presione ENTER para continuar"
}

comprobar_root() {
        if [[ $EUID -ne 0 ]]; then
            mostrar_error "Este módulo debe ejecutarse como root."
            echo "Ejecute el script utilizando sudo."
            return 1
        fi
        return 0
}

confirmar() {
        local mensaje="$1"
        read -p "$mensaje [s/N]: " respuesta
        if [[ "$respuesta" == "s" || "$respuesta" == "S" ]]; then
                return 0
        fi
        return 1
}
adminipsum@iti-ipsum-2026:~$
```

Demostración sobre cómo se utilizaron las funciones establecidas en **common.sh**, en los scripts:

```
#!/bin/bash

source /scripts/lib/common.sh

crear_usuario() {

        echo
        read -p "Ingrese el nombre del nuevo usuario: " nuevo_usuario

        if [[ -z "$nuevo_usuario" ]]; then
           mostrar_error "El nombre de usuario no puede estar vacío."
           pausa
           return
        fi

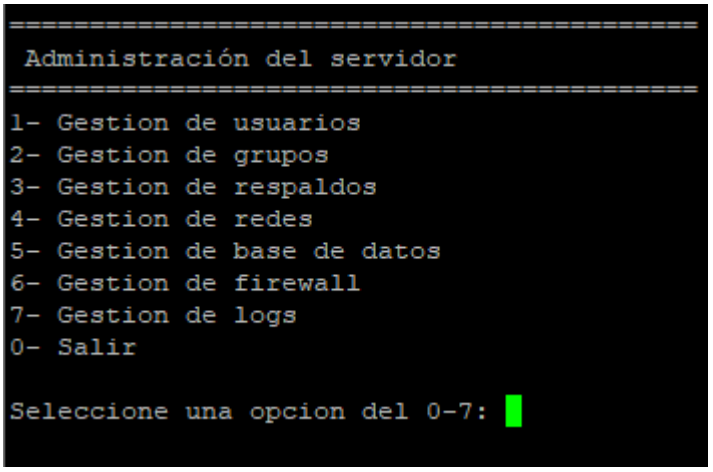
        if id "$nuevo_usuario" &>/dev/null; then
           mostrar_error "El usuario '$nuevo_usuario' ya existe."
           pausa
           return
        fi

        if useradd -m "$nuevo_usuario"; then
           mostrar_ok "Usuario '$nuevo_usuario' creado correctamente."
        else
           mostrar_error "No se pudo crear el usuario '$nuevo_usuario'."
           pausa
           return
        fi
           passwd "$nuevo_usuario"

           pausa
}
```

Ejecución del menú principal

El menú principal se ejecuta desde la terminal sudo /scripts/admin_servidor.sh



La interfaz actúa como el punto de acceso central a las operaciones del sistema, facilitando la navegación entre las herramientas administrativas y la ejecución de las funciones de cada módulo.

Manejo de errores

La gestión de errores resulta fundamental para evitar que una operación administrativa falle sin informar al usuario.

Los scripts deben comprobar los resultados de las operaciones relevantes y presentar mensajes claros cuando no puedan completarse.

También es importante que las operaciones delicadas, como la eliminación de cuentas o la restauración de bases de datos, requieran las comprobaciones y confirmaciones correspondientes.

Para inspeccionar el tratamiento de errores y los mensajes del código se utilizo sudo grep -nE 'mostrar_error|mostrar_exito|log_error|return 1|exit 1' \ /scripts/lib/common.sh /scripts/modules/*.sh

```
adminipsum@iti-ipsum-2026:~$ sudo grep -nE 'mostrar_error|mostrar_exito|log_error|return 1|exit 1' \
  /scripts/lib/common.sh /scripts/modules/*.sh
/scripts/lib/common.sh:13:mostrar_error() {
/scripts/lib/common.sh:28:          mostrar_error "Este módulo debe ejecutarse como root."
/scripts/lib/common.sh:30:          return 1
/scripts/lib/common.sh:41:      return 1
/scripts/modules/base_datos.sh:16:              mostrar_error "MySQL o MariaDB no esta activo."
/scripts/modules/base_datos.sh:27:              mostrar_error "El cliente MySQL no esta instalado."
/scripts/modules/base_datos.sh:48:                      mostrar_error "No se pudo consultar la base de datos."
/scripts/modules/base_datos.sh:60:              mostrar_error "El cliente mysqladmin no esta instalado."
/scripts/modules/base_datos.sh:71:              mostrar_error "Debe ingresar un usuario."
/scripts/modules/base_datos.sh:79:              mostrar_error "No se pudo conectar con el servidor de base de datos."
/scripts/modules/base_datos.sh:119:             mostrar_error "Opcion no valida"
/scripts/modules/firewall.sh:7:         exit 1
/scripts/modules/firewall.sh:28:        mostrar_error "El puerto no es valido."
/scripts/modules/firewall.sh:36:        mostrar_error "Protocolo no valido."
/scripts/modules/firewall.sh:46:            mostrar_error "No se pudo agregar la regla."
/scripts/modules/firewall.sh:61:        mostrar_error "Numero de regla no valido."
/scripts/modules/firewall.sh:72:            mostrar_error "No se pudo eliminar la regla."
/scripts/modules/firewall.sh:118:                   mostrar_error "No se pudo activar el firewall."
/scripts/modules/firewall.sh:129:                   mostrar_error "No se pudo desactivar el firewall."
/scripts/modules/firewall.sh:138:           mostrar_error "Opcion no valida."
/scripts/modules/grupos.sh:7:    exit 1
/scripts/modules/grupos.sh:15:  mostrar_error "El nombre del grupo no puede estar vacio"
/scripts/modules/grupos.sh:21:  mostrar_error "El nombre del grupo '$grupo' ya existe"
/scripts/modules/grupos.sh:29:  mostrar_error "No se pudo crear el grupo '$grupo'"
/scripts/modules/grupos.sh:40:  mostrar_error "El grupo '$grupo' no existe"
/scripts/modules/grupos.sh:49:  mostrar_error "El nuevo nombre no puede estar vacio"
/scripts/modules/grupos.sh:55:      mostrar_error "El grupo '$nuevo_grupo' ya existe"
/scripts/modules/grupos.sh:63:  mostrar_error "No se pudo modificar el grupo"
/scripts/modules/grupos.sh:74:  mostrar_error "El grupo '$grupo' no existe"
/scripts/modules/grupos.sh:91:  mostrar_error "No se pudo eliminar el grupo '$grupo'"
/scripts/modules/grupos.sh:102:         mostrar_error "El grupo '$grupo' no existe"
/scripts/modules/grupos.sh:148:             mostrar_error "Opcion no valida"
/scripts/modules/logs.sh:7:     exit 1
/scripts/modules/logs.sh:17:        mostrar_error "Cantidad no valida."
/scripts/modules/logs.sh:34:        mostrar_error "Nombre de servicio no valido."
/scripts/modules/logs.sh:56:        mostrar_error "No se pudieron guardar los registros."
/scripts/modules/logs.sh:94:            mostrar_error "Opcion no valida."
/scripts/modules/redes.sh:7:    exit 1
/scripts/modules/redes.sh:35:           mostrar_error "No se encontro el archivo Netplan."
/scripts/modules/redes.sh:57:           mostrar_error "No se pudo crear la copia de seguridad."
/scripts/modules/redes.sh:71:           mostrar_error "Configuracion incorrecta. Restaurando copia."
/scripts/modules/redes.sh:88:           mostrar_error "No se confirmaron los cambios"
/scripts/modules/redes.sh:95:                   mostrar_error "Revise manualmente la configuracion de red"
/scripts/modules/redes.sh:113:          mostrar_error "No se pudo establecer la conexion"
/scripts/modules/redes.sh:152:              mostrar_error "Opcion no valida"
/scripts/modules/respaldos.sh:7:    exit 1
/scripts/modules/respaldos.sh:16:                       mostrar_error "No se pudo preparar el directorio de respaldos
."
/scripts/modules/respaldos.sh:17:                       return 1
/scripts/modules/respaldos.sh:32:               mostrar_error "La ruta no existe o no es un directorio."
/scripts/modules/respaldos.sh:45:                       mostrar_error "Ya existe un archivo con ese nombre."
/scripts/modules/respaldos.sh:56:               mostrar_error "No se pudo crear el respaldo."
/scripts/modules/respaldos.sh:98:               mostrar_error "Ingrese solamente el nombre de un archivo .tar.gz."
/scripts/modules/respaldos.sh:106:              mostrar_error "No se encontro ese archivo de respaldo."
/scripts/modules/respaldos.sh:112:              mostrar_error "El archivo no es un respaldo TAR.GZ valido."
/scripts/modules/respaldos.sh:118:                mostrar_error "El respaldo contiene rutas no permitidas."
/scripts/modules/respaldos.sh:126:              mostrar_error "La carpeta de destino no existe."
/scripts/modules/respaldos.sh:140:                              mostrar_error "Ocurrio un error al restaurar el respa
```

Esta instrucción facilita la localización de rutinas orientadas al control de errores y al registro del sistema. La interpretación de la salida depende de las funciones que hayan sido implementadas concretamente en los archivos.

Administración de usuarios y grupos

Géstion de usuarios

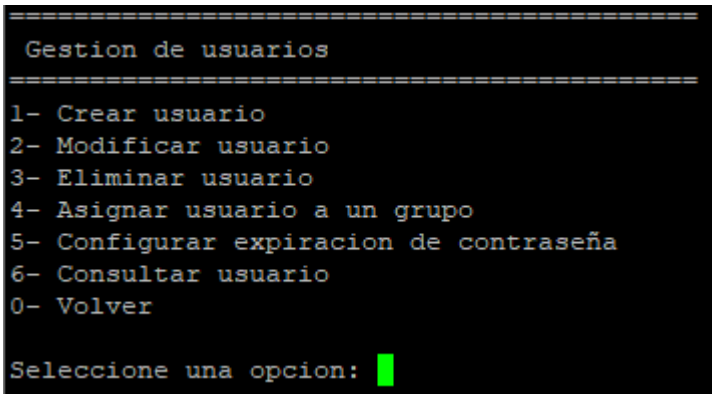
Este módulo sirve para administrar las cuentas de usuario en el servidor. Ayuda a controlar quién entra al sistema y evita que varias personas usen la misma cuenta.

Desde aquí se pueden realizar tareas como:

- Ver la lista de usuarios.
- Crear, cambiar o borrar cuentas.
- Asignar usuarios a diferentes grupos.
- Bloquear accesos cuando sea necesario.

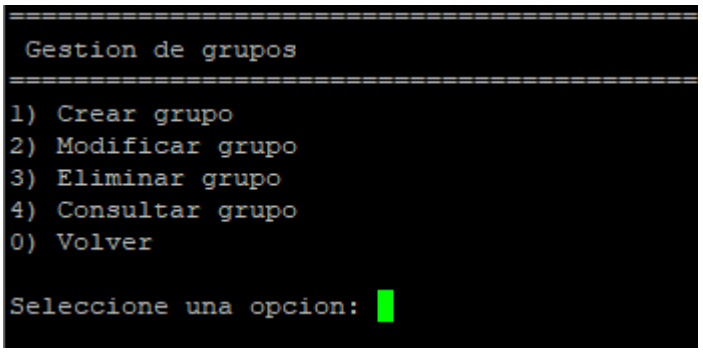
Por seguridad, solo los usuarios autorizados y con permisos de administrador pueden usar estas opciones.

Para ejecutar el módulo a través del menú principal se utilizó sudo /scripts/admin_servidor.sh, una vez desplegado el menú de admin_servidor.sh, ingresamos mediante el número 1 al menú de gestión de usuarios.



Gestion de Grupos

Los grupos sirven para juntar varias cuentas que comparten las mismas tareas o permisos. Esto facilita mucho el trabajo, ya que permite dar acceso a varios usuarios al mismo tiempo sin tener que configurar la cuenta de cada uno por separado.



Seguridad en la administración de cuentas

Para mantener el servidor seguro, es muy importante asignar a cada usuario solo los permisos indispensables para que pueda realizar sus tareas. Además, las acciones delicadas, deben hacerse con cuidado, verificando siempre a qué usuario se está modificando. Por último, hay que cuidar la privacidad y no incluir contraseñas ni datos confidenciales en el texto o las imágenes del proyecto.

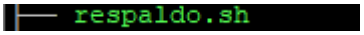
Gestión de respaldos

Objetivo de la estrategia de respaldo

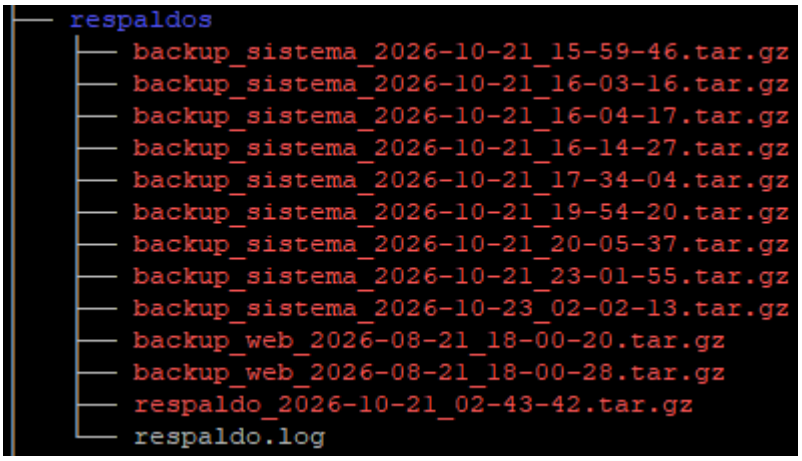
Los respaldos protegen la información del servidor ante posibles fallas. Para asegurar una recuperación real, es importante generar estas copias, como verificar que se puedan leer y contar con un plan de restauración paso a paso.

Script de respaldo integral

El script de respaldo del servidor se encuentra en /home/adminipsum/**respaldo.sh**



El directorio de almacenamiento es /home/adminipsum/respaldos/



El procedimiento crea un archivo comprimido con fecha y hora, registra los resultados de la ejecución y genera un volcado de las bases de datos de MariaDB.

- **/etc:** Guardar la configuración del sistema.
- **/scripts:** Conservar los scripts de administración.
- **/var/www/html/sgrsi:** Guardar los archivos de la aplicación o sitio web.
- **Volcado SQL de MariaDB:** Resguarda la información contenida en la base de datos.
- **Archivo de log:** Guardar el resultado del proceso para revisar si hubo errores.

Ejecución de respaldo

El script puede ejecutarse manualmente mediante sudo /home/adminipsum/respaldo.sh

Durante las pruebas, el proceso de respaldo se ejecutó correctamente y generó la copia completa de la información. El reporte generado permite verificar de forma sencilla cuándo comenzó la tarea, si terminó con éxito, qué nombre tiene el archivo y cuánto espacio ocupa.

Verificación del archivo de respaldo

Para revisar el contenido de un archivo comprimido utilizamos sudo tar -tzf /home/adminipsum/respaldos/backup_sistema_2026-10-23_02-02-13.tar.gz

Para comprobar la lectura del archivo sin imprimir todo su contenido se utilizó sudo tar -tzf /home/adminipsum/respaldos/backup_sistema_2026-10-23_02-02-13.tar.gz \

> /dev/null && echo "Respaldo integro" || echo "ERROR: respaldo dañado"

La prueba realizada devolvió:

“Respaldo integro”

```
adminipsum@iti-ipsum-2026:~$ sudo tar -tzf /home/adminipsum/respaldos/backup_sistema_2026-10-23_02-02-13.tar.gz \
  > /dev/null && echo "Respaldo integro" || echo "ERROR: respaldo dañado"
Respaldo integro
adminipsum@iti-ipsum-2026:~$
```

Esta prueba confirma que el archivo de respaldo se puede abrir y consultar sin problemas. Pero no reemplaza la prueba final, que consiste en volver a instalar los archivos y la base de datos para asegurar que funcionen correctamente.

Programación automática

La tarea de respaldo está programada en el crontab de root para que se realice los sábados a la 01:00 am, utilizando 0 1 * * 6 /home/adminipsum/respaldo.sh

Para consultar las tareas programadas utilizamos sudo crontab -l

```
adminipsum@iti-ipsum-2026:~$ sudo crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
0 1 * * 6 /home/adminipsum/respaldo.sh
adminipsum@iti-ipsum-2026:~$
```

Para verificar el estado del servicio utilizamos sudo systemctl is-active cron

```
adminipsum@iti-ipsum-2026:~$ sudo systemctl is-active cron
active
adminipsum@iti-ipsum-2026:~$
```

La comprobación demuestra que la tarea está registrada y que el servicio cron está activo.

Conservación de respaldos

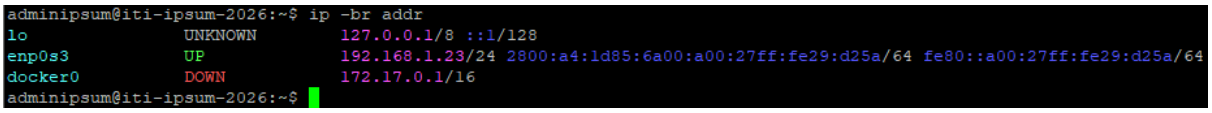
Guardar copias de seguridad antiguas permite solucionar errores o volver a versiones anteriores previas a algún fallo o problema. Para que el sistema sea confiable, hay que definir cada cuánto se hacen las copias, cuánto tiempo se conservan y cómo se guardan fuera del servidor.

Gestión de red

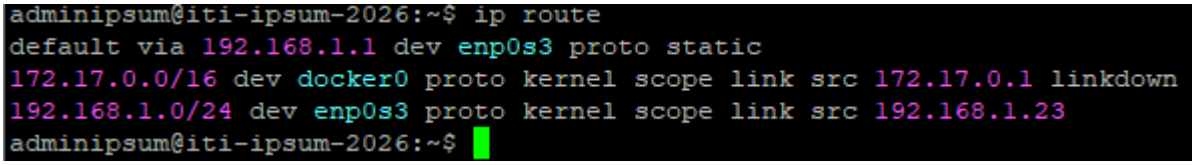
Verificación de interfaces de red

La interfaz de red es la que permite que el servidor pueda comunicarse con otros equipos de la misma red. Durante las pruebas del proyecto, se utilizó la dirección IP 192.168.1.23 para comprobar que el servidor respondía correctamente y era accesible desde otra computadora.

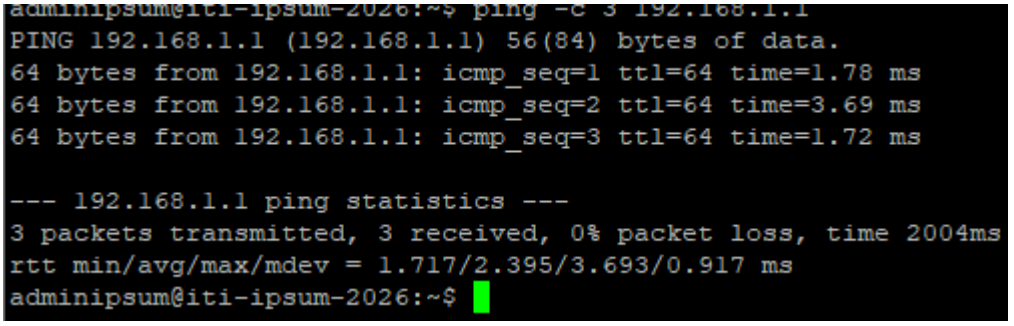
Ver las interfaces de red y sus direcciones IP utilizamos ip -br addr



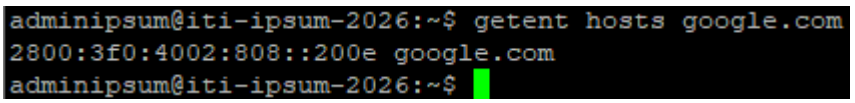
Consultar las rutas de red utilizamos ip route



Para probar la conexión con la puerta de enlace, verificando la IP 192.168.1.1 con 3 intentos utilizamos ping -c 3 192.168.1.1



Para verificar que el sistema traduce direcciones web correctamente utilizamos getent hosts **google.com**



La salida de los comandos confirma que las tarjetas de red, las rutas de acceso y el servicio DNS están funcionando correctamente.

Administración de bases de datos

Función de MariaDB

SGRSI utiliza MariaDB para almacenar y gestionar la información de la aplicación. Como medida de seguridad, el sistema evita usar la cuenta de administrador general (root) en sus tareas cotidianas y utiliza en su lugar un usuario propio con permisos limitados y específicos.

Consulta del servicio

Para comprobar el estado de MariaDB utilizamos

```
adminipsum@iti-ipsum-2026:~$ sudo systemctl is-active mariadb
[sudo: authenticate] Password:
active
adminipsum@iti-ipsum-2026:~$
```

Para consultar la versión instalada utilizamos mariadb --version

```
adminipsum@iti-ipsum-2026:~$ mariadb --version
mariadb from 11.8.6-MariaDB, client 15.2 for debian-linux-gnu (x86_64) using  EditLine wrapper
adminipsum@iti-ipsum-2026:~$
```

Para obtener un listado de las bases de datos, mediante una conexión administrativa local utilizamos sudo mariadb -e "SHOW DATABASES;"

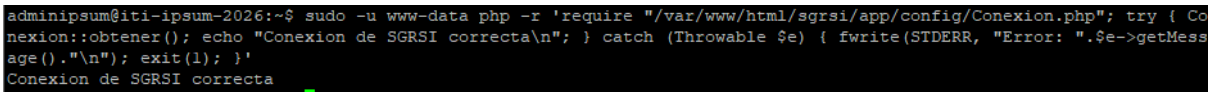
```
adminipsum@iti-ipsum-2026:~$ sudo mariadb -e "SHOW DATABASES;"
+--------------------+
| Database           |
+--------------------+
| information_schema |
| ipsum_reportes     |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
adminipsum@iti-ipsum-2026:~$
```

Durante las comprobaciones de SGRSI se identificó la base de datos ipsum_reportes

Conexión desde PHP

- La conexión de la aplicación se encuentra en /var/www/html/sgrsi/app/config/Conexion.php.
- La configuración local se mantiene en /var/www/html/sgrsi/app/config/local.php.
    - La conectividad desde la aplicación se verificó mediante la clase de conexión

sudo -u www-data php -r 'require "/var/www/html/sgrsi/app/config/Conexion.php"; try { Conexion::obtener(); echo "Conexion de SGRSI correcta\n"; } catch (Throwable $e) { fwrite(STDERR, "Error: ".$e->getMessage()."\n"); exit(1); }'.



Pudimos verificar que PHP establece la conexión con la base de datos utilizando su configuración local.

Copias y recuperación de bases de datos

La rutina genera respaldos de la base de datos con mariadb-dump para asegurar la recuperación de la información. Antes de restaurar una copia —proceso que requiere permisos y un entorno controlado—, es obligatorio contar con un respaldo del estado actual.

Administración del firewall

Objetivo de firewall

El firewall regula el tráfico de red permitiendo únicamente el acceso a los servicios necesarios, lo que evita la exposición innecesaria de puertos y protege el servidor.

- Para consultar el estado del firewall: sudo ufw status verbose

```
adminipsum@iti-ipsum-2026:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
443/tcp (v6)               ALLOW IN    Anywhere (v6)

adminipsum@iti-ipsum-2026:~$
```

- Para consultar las reglas numeradas: sudo ufw status numbered

```
adminipsum@iti-ipsum-2026:~$ sudo ufw status numbered
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 22/tcp                     ALLOW IN    Anywhere
[ 2] 80/tcp                     ALLOW IN    Anywhere
[ 3] 443/tcp                    ALLOW IN    Anywhere
[ 4] 22/tcp (v6)                ALLOW IN    Anywhere (v6)
[ 5] 80/tcp (v6)                ALLOW IN    Anywhere (v6)
[ 6] 443/tcp (v6)               ALLOW IN    Anywhere (v6)
```

Estos comandos muestran el estado actual del firewall y el listado de reglas activas en el servidor.

Puertos requeridos

Los puertos son:

- TCP 22: Para el servicio SSH
- TCP 80: Para el servicio HTTP
- TCP 443: Para el servicio HTTPS

Abrir un puerto requiere que el servicio correspondiente esté realmente en uso y cumpla la política de seguridad. Al modificar las reglas, es indispensable asegurar un acceso administrativo de respaldo para evitar perder la comunicación remota con el servidor.

Gestión de registros y logs del sistema

Importancia de los registros

Los registros del sistema permiten investigar fallas, errores y eventos de acceso. El módulo de logs facilita la consulta de esta información, la cual debe revisarse y conservarse para analizar posibles incidentes futuros.

Registros del sistema

Cuando están disponibles en la instalación, se pueden consultar los siguientes archivos: /var/log/syslog y /var/log/auth.log.

Para consultar las últimas entradas de los registros:

- /var/log/syslog
- /var/log/auth.log

Disponibilidad de archivos: La existencia de los archivos de registro cambia según la configuración del sistema operativo. Si no se encuentran en la ruta habitual, se debe consultar el gestor de logs propio.

También pueden consultarse los eventos de systemd mediante:

- sudo journalctl -p err -b --no-pager, este comando permite consultar mensajes de error registrados.

Registros de Apache

Apache dispone de registros separados para errores y solicitudes de acceso.

Los archivos del sitio SGRSI son:

- /var/log/apache2/sgrsi_error.log
- /var/log/apache2/sgrsi_access.log

Para consultar los últimos errores:

- sudo tail -n 30 /var/log/apache2/sgrsi_error.log

Para consultar las últimas solicitudes registradas:

- sudo tail -n 30 /var/log/apache2/sgrsi_access.log

La consulta del registro de SGRSI no mostró entradas de error recientes. Esto confirma la ausencia de fallas en la muestra revisada, aunque no asegura la ausencia total de errores en el sistema.

Despliegue de SGRSI con Apache y PHP-FPM

Ubicación de la aplicación

La aplicación se encuentra desplegada bajo:

- /var/www/html/sgrsi/

El directorio público utilizado por Apache es:

- /var/www/html/sgrsi/public/

Separar el directorio público de los archivos internos evita que configuraciones y componentes sensibles del sistema queden expuestos directamente a través del servidor web.

Para consultar la estructura:

- sudo ls -la /var/www/html/sgrsi/

```
adminipsum@iti-ipsum-2026:~$ sudo ls -la /var/www/html/sgrsi/
[sudo: authenticate] Password:
total 20
drwxr-x--- 5 root www-data 4096 Oct 21 18:10 .
drwxr-xr-x 3 root root     4096 Oct 23 01:24 ..
drwxr-x--- 7 root www-data 4096 Oct 21 18:10 app
drwxr-x--- 2 root www-data 4096 Oct 21 18:10 database
drwxr-x--- 4 root www-data 4096 Oct 21 18:10 public
adminipsum@iti-ipsum-2026:~$
```

- sudo ls -la /var/www/html/sgrsi/public/

```
adminipsum@iti-ipsum-2026:~$ sudo ls -la /var/www/html/sgrsi/public/
total 28
drwxr-x--- 4 root www-data 4096 Oct 21 18:10 .
drwxr-x--- 5 root www-data 4096 Oct 21 18:10 ..
drwxr-x--- 2 root www-data 4096 Oct 21 18:10 api
drwxr-x--- 4 root www-data 4096 Oct 21 18:10 assets
-rw-r----- 1 root www-data 4045 Oct 21 18:10 index.php
-rw-r----- 1 root www-data   66 Oct 21 18:10 login.php
-rw-r----- 1 root www-data   80 Oct 21 18:10 procesar-login.php
```

Configuración del sitio de Apache

La configuración específica de SGRSI se encuentra en:

- /etc/apache2/sites-available/sgrsi.conf

El contenido utilizado es el siguiente:

```
<VirtualHost *:80>
    ServerName localhost
    DocumentRoot /var/www/html/sgrsi/public

    <Directory /var/www/html/sgrsi/public>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    <FilesMatch "\.php$">
        SetHandler "proxy:unix:/run/php/php8.5-fpm.sock|fcgi://localhost/"
    </FilesMatch>

    ErrorLog ${APACHE_LOG_DIR}/sgrsi_error.log
    CustomLog ${APACHE_LOG_DIR}/sgrsi_access.log combined
</VirtualHost>
```

La configuración de Apache se divide en varios parámetros clave: DocumentRoot define la carpeta pública de la aplicación, el bloque Directory aplica reglas de seguridad como prohibir el listado de archivos (-Indexes) y habilitar .htaccess, FilesMatch procesa PHP con PHP-FPM, y los registros (ErrorLog / CustomLog) almacenan los errores y la actividad del servidor.

Funcionamiento de PHP-FPM

PHP-FPM se encarga de procesar las solicitudes que requieren ejecutar código PHP.

La comunicación con Apache se configura a través del socket:

- /run/php/php8.5-fpm.sock

Para comprobar el servicio:

- sudo systemctl is-active php8.5-fpm

```
adminipsum@iti-ipsum-2026:~$ sudo systemctl is-active php8.5-fpm
active
adminipsum@iti-ipsum-2026:~$
```

Para consultar la versión de PHP:

- php -v

```
adminipsum@iti-ipsum-2026:~$ php -v
PHP 8.5.4 (cli) (built: Oct 19 2026 14:35:18) (NTS)
Copyright (c) The PHP Group
Built by Ubuntu
Zend Engine v4.5.4, Copyright (c) Zend Technologies
    with Zend OPcache v8.5.4, Copyright (c), by Zend Technologies
adminipsum@iti-ipsum-2026:~$
```

Para comprobar las extensiones relacionadas con la conexión a bases de datos:

- php -m | grep -E 'mysqli|PDO|pdo_mysql'

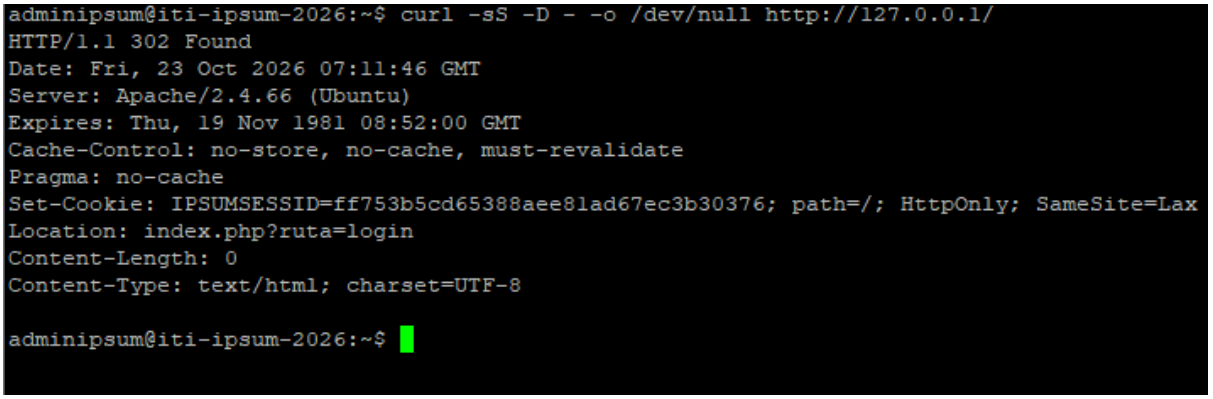
```
adminipsum@iti-ipsum-2026:~$ php -m | grep -E 'mysqli|PDO|pdo_mysql'
mysqli
PDO
pdo_mysql
adminipsum@iti-ipsum-2026:~$
```

Durante las verificaciones se confirmó que PHP-FPM se encontraba activo y que estaban disponibles las extensiones mysqli, PDO y pdo_mysql.

Comprobacion HTTP y acceso a la aplicación

Para consultar la respuesta HTTP del servidor local:

- curl -sS -D - -o /dev/null http://127.0.0.1/



Se confirmó la respuesta correcta del servidor (200 OK) y el inicio de sesión de usuarios desde un equipo cliente. La prueba final de los módulos del sistema se completará según los casos de uso definidos para SGRSI.

Configuración y verificacion de MariaDB para SGRSI

SGRSI utiliza MariaDB para su gestión de datos, centralizando la conexión en Conexion.php. El sistema emplea la base de datos ipsum_reportes y la cuenta de usuario sgrsi_app, evitando así el uso de la cuenta administrativa global para las tareas diarias de la aplicación.

Para verificar conjuntamente el estado de los servicios principales:

- sudo systemctl is-active apache2 php8.5-fpm mariadb

```
adminipsum@iti-ipsum-2026:~$ sudo systemctl is-active apache2 php8.5-fpm mariadb
active
active
active
adminipsum@iti-ipsum-2026:~$
```

Seguridad del servidor y acceso remoto mediante SSH

Acceso remoto

SSH proporciona un mecanismo cifrado para administrar el servidor de manera remota. Su funcionamiento debe comprobarse junto con las reglas del firewall y la configuración de las cuentas administrativas.

Para consultar el estado del servicio en Ubuntu:

- sudo systemctl is-active ssh

```
adminipsum@iti-ipsum-2026:~$ sudo systemctl is-active ssh
inactive
adminipsum@iti-ipsum-2026:~$
```

Para consultar si se encuentra habilitado al inicio:

- sudo systemctl is-enabled ssh

```
adminipsum@iti-ipsum-2026:~$ sudo systemctl is-enabled ssh
enabled
adminipsum@iti-ipsum-2026:~$
```

Principio de mínimo privilegio

Los servicios web y los scripts administrativos deben ejecutarse con los permisos necesarios para realizar sus tareas, sin conceder privilegios adicionales de manera indiscriminada.

La aplicación utiliza una cuenta específica de base de datos y mantiene sus parámetros locales separados del código de acceso a datos.

Control de acceso al directorio de respaldos

El directorio de respaldos se encuentra en:

- /home/adminipsum/respaldos/

Para comprobar sus permisos:

- sudo ls -ld /home/adminipsum/respaldos/

```
adminipsum@iti-ipsum-2026:~$ sudo ls -ld /home/adminipsum/respaldos/
drwx------ 2 adminipsum adminipsum 4096 Oct 23 05:38 /home/adminipsum/respaldos/
adminipsum@iti-ipsum-2026:~$
```

Para consultar los permisos de un archivo de respaldo:

- sudo ls -lh /home/adminipsum/respaldos/

```
adminipsum@iti-ipsum-2026:~$ sudo ls -lh /home/adminipsum/respaldos/
total 9.8M
-rw------- 1 root       root       600K Oct 21 15:59 backup_sistema_2026-10-21_15-59-46.tar.gz
-rw------- 1 root       root       600K Oct 21 16:03 backup_sistema_2026-10-21_16-03-16.tar.gz
-rw------- 1 root       root       600K Oct 21 16:04 backup_sistema_2026-10-21_16-04-17.tar.gz
-rw------- 1 root       root       600K Oct 21 16:14 backup_sistema_2026-10-21_16-14-27.tar.gz
-rw------- 1 root       root       1.2M Oct 21 17:34 backup_sistema_2026-10-21_17-34-04.tar.gz
-rw------- 1 root       root       1.3M Oct 21 19:54 backup_sistema_2026-10-21_19-54-20.tar.gz
-rw------- 1 root       root       1.3M Oct 21 20:05 backup_sistema_2026-10-21_20-05-37.tar.gz
-rw------- 1 root       root       1.3M Oct 21 23:01 backup_sistema_2026-10-21_23-01-55.tar.gz
-rw------- 1 root       root       1.3M Oct 23 02:02 backup_sistema_2026-10-23_02-02-13.tar.gz
-rw------- 1 root       root       1.3M Oct 23 05:38 backup_sistema_2026-10-23_05-38-29.tar.gz
-rw-rw-r-- 1 adminipsum adminipsum  549 Aug 21 18:00 backup_web_2026-08-21_18-00-20.tar.gz
-rw-rw-r-- 1 adminipsum adminipsum  549 Aug 21 18:00 backup_web_2026-08-21_18-00-28.tar.gz
-rw-rw-r-- 1 adminipsum adminipsum 4.9K Oct 23 05:38 respaldo.log
-rw------- 1 adminipsum adminipsum  188 Oct 21 02:43 respaldo_2026-10-21_02-43-42.tar.gz
adminipsum@iti-ipsum-2026:~$
```

Durante la ejecución del script se utilizaron permisos restrictivos para proteger el directorio y los archivos generados.

Gestión de contenedores y alcance del despliegue

La rúbrica incluye el despliegue mediante contenedores; sin embargo, la arquitectura implementada en este servidor utiliza una instalación nativa sobre Ubuntu Server con

servicios tradicionales (Apache, PHP-FPM y MariaDB). Este apartado registra dicha diferencia respecto al criterio original.

Arquitectura actualmente utilizada

La aplicación se publica desde:

- /var/www/html/sgrsi/public/

El flujo del sistema se divide entre Apache para las peticiones web, PHP-FPM para el procesamiento de código y MariaDB para el almacenamiento de datos, utilizando configuraciones locales independientes.

Los servicios se administran con systemd:

- sudo systemctl is-active apache2 php8.5-fpm mariadb

Servidor de respaldo y contingencia

Objetivo del servidor secundario

El respaldo integral protege la información del sistema frente a fallos o pérdidas de datos. Incluye archivos clave del servidor, scripts, la aplicación web y la base de datos MariaDB. Cada ejecución genera un paquete comprimido y un archivo de registro para su verificación.

Ejecución de respaldo

El script se encuentra en:

- /home/adminipsum/**respaldo.sh**

Para ejecutarlo manualmente:

- sudo /home/adminipsum/**respaldo.sh**

Para consultar el registro:

- sudo tail -n 30 /home/adminipsum/respaldos/respaldo.log

Se verificó la ejecución manual del procedimiento, obteniendo el archivo comprimido de respaldo y confirmando su registro en el log del sistema.

Verificación del archivo

Para comprobar que un archivo puede leerse correctamente:

- sudo tar -tzf /home/adminipsum/respaldos/<archivo_de_respaldo>.tar.gz \
- > /dev/null && echo "Respaldo integro" || echo "ERROR: respaldo dañado"

La prueba realizada leyó el paquete correctamente y arrojó el resultado Respaldo integro. Esta acción confirma la lectura del comprimido, pero no garantiza la restauración completa de los componentes.

Recuperación de los archivos

Busque la copia de seguridad necesaria y ábrala primero en una carpeta de prueba para no borrar la información que está en uso. Antes de hacer la recuperación final, confirme que los archivos son los correctos y que funcionan con la versión actual del sistema.

Recuperación de MariaDB

El archivo SQL permite recuperar los datos del sistema. Antes de importarlo, identifique la base de datos de destino, respalde los datos actuales y confirme que el archivo sea compatible. Evite sobrescribir directamente sobre una base de datos que contenga información que aún se necesite conservar.

Pruebas de recuperación

Se comprobó con éxito la lectura del archivo comprimido. La restauración completa de la aplicación y la base de datos queda pendiente para ejecutarse y documentarse en un

entorno de prueba controlado. La prueba se considerará validada cuando se confirme el acceso a la aplicación, la conexión a MariaDB y el correcto funcionamiento del sistema.

Conclusiones del servidor

El trabajo realizado permitió poner en marcha el sistema SGRSI en un servidor Ubuntu, utilizando Apache para atender las páginas web, PHP-FPM para procesar el código y MariaDB para almacenar la información de manera segura.

La aplicación quedó instalada en la ruta /var/www/html/sgrsi/, utilizando la carpeta public como acceso principal. Durante el proceso, se comprobó que todos los servicios funcionan correctamente, que las páginas responden de forma adecuada, que la conexión entre PHP y la base de datos es estable y que se puede acceder a la plataforma sin problemas. Además, se cuenta con una organización modular que facilita la administración de las tareas del servidor.

En cuanto a la seguridad de la información, se creó una rutina de respaldo completa que ya fue probada de forma manual, revisando sus registros y confirmando que los archivos comprimidos se pueden leer sin errores. Asimismo, se dejó programada una copia automática semanal utilizando el servicio cron del sistema.

Riesgos y pruebas de seguridad

El análisis de riesgos nos permite identificar qué información puede verse afectada y qué medida debemos aplicar. Las prioridades siguientes son una propuesta técnica basada en el código; no representan una calificación docente ni resultados de pruebas realizadas.

Riesgo	**Prioridad**	**Medida necesaria**
XSS almacenado en detalles	Alta	textContent y JSON protegido en script.
Permisos antiguos en sesión	Alta	Comprobar usuario activo y rol vigente.
Pérdida de datos MySQL	Alta	Backup independiente y restauración probada.
Entradas inválidas y estados	Alta	Validación común y transiciones permitidas.
Intentos masivos de login	Media	Límites por cuenta y origen.
Sesión prolongada y logout CSRF	Media	Vencimientos y logout POST con token.

Plan de pruebas pendiente de ejecución

- **Acceso:** login correcto, incorrecto e inactivo; cambio del identificador de sesión al autenticar y destrucción al salir.
- **Permisos:** peticiones directas como anónimo y cada rol; dos profesores intentando acceder a reportes ajenos.
- **Revocación**: mantener una sesión abierta y comprobar que el cambio de rol o baja impide continuar con permisos antiguos.
- **CSRF y XSS:** tokens ausentes o de otra sesión; texto con etiquetas en reportes, modales y datos del usuario.
- Datos y errores: valores fuera de lista, fechas inválidas, arrays inesperados, SQLi controlada y caída de MySQL.
- **Infraestructura:** HTTPS, cookies, raíz pública, ejecución cron y restauración completa.

Para cada caso se debe registrar fecha, versión, pasos, resultado esperado, resultado obtenido y evidencia. No se deben incluir contraseñas o datos personales reales en capturas de prueba. El anexo ofrece una ficha para completar.

Plan de mejoras y cierre

La base de seguridad del sistema está implementada, pero necesitamos completar los siguientes puntos antes de considerarlo preparado para uso con datos reales. Las mejoras se ordenan según su impacto sobre cuentas, información y recuperación.

Correcciones de prioridad alta

- Eliminar la interpretación HTML de datos en modales y proteger el JSON dentro de script.
- Actualizar autorización por rol y usuario activo en cada petición, revocando acceso cuando cambien permisos.
- Completar validaciones de servidor, controlar estados permitidos y comprobar filas afectadas antes de mostrar éxito.
- Retirar credenciales de reserva de producción, corregir rutas para Linux y controlar errores de manera global.
- Agregar MySQL a los respaldos, copia independiente y prueba de recuperación.

Mejoras complementarias

- Definir política de contraseñas, límites de intentos, actualización de hash y vencimiento de sesiones.
- Completar logout POST con token y verificar HTTPS, cookies y acceso SSH.
- Registrar acciones sensibles con actor y resultado y aplicar retención y protección de logs.

Continuidad en la entrega final

Full Stack exige una API con métodos GET, POST, PUT y DELETE y respuestas JSON consistentes. La versión revisada utiliza controladores que generan HTML o redirigen. Cuando se implemente la API, cada endpoint deberá conservar los mismos controles de sesión, rol, propiedad, validación y CSRF si utiliza cookies.

También debemos completar el script modular de administración del servidor y reunir en la carpeta integrada la documentación y los anexos. Los manuales enlazados deberán incorporarse íntegramente y no sustituirse por enlaces externos.

Conclusión

La versión 1.8.4 protege contraseñas, consultas y varias operaciones mediante controles del servidor. El trabajo pendiente se concentra en XSS, vigencia de permisos, validaciones y recuperación de información. La seguridad debe demostrarse con el código corregido y resultados reales de pruebas, manteniendo esta documentación actualizada con cada cambio.

Referencias Bibliográficas

Libros y textos académicos

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.
- Kotler, P., & Keller, K. L. (2016). Marketing management (15.ª ed.). Pearson.
- McConnell, S. (2006). Software estimation: Demystifying the black art. Microsoft Press.
- Medeiros, M. (s. f.). Ingeniería de software. [Material de estudio]. Nota: falta confirmar la fuente original, el año y el tipo de publicación.
- Moore, G. A. (2014). Crossing the chasm: Marketing and selling disruptive products to mainstream customers (3.ª ed.). HarperBusiness.
- Ries, E. (2011). The lean startup: How today's entrepreneurs use continuous innovation to create radically successful businesses. Crown Business.
- Rogers, E. M. (2003). Diffusion of innovations (5.ª ed.). Free Press.
- Sommerville, I. (2010). Software engineering (9.ª ed.). Addison-Wesley.

Fuentes normativas e institucionales

- Congreso de la República Oriental del Uruguay. (1937). Ley N.º 9.739: Derechos de autor. IMPO. https://www.impo.com.uy/bases/leyes/9739-1937
- Congreso de la República Oriental del Uruguay. (2008). Ley N.º 18.331: Protección de datos personales y acción de habeas data. IMPO. https://www.impo.com.uy/bases/leyes/18331-2008
- Instituto Tecnológico de Informática. (2026). Pautas y letra del Proyecto de Egreso BT Informática 2026 (Rev. 1.0). CETP-UTU.

Estándares internacionales y documentación técnica

- Apache Software Foundation. (s. f.). Apache HTTP Server version 2.4 documentation. https://httpd.apache.org/docs/2.4/
- Canonical. (s. f.). Ubuntu Server documentation. https://documentation.ubuntu.com/server/
- Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/en/v2
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Tesis doctoral, University of California, Irvine]. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm
- PHP-FIG. (s. f.). PSR-12: Extended coding style guide. https://www.php-fig.org/psr/psr-12/
- PHP Group. (s. f.). PHP manual. https://www.php.net/docs.php
- Preston-Werner, T. (s. f.). Semantic Versioning 2.0.0. https://semver.org/
- The MariaDB Foundation. (s. f.). MariaDB documentation. https://mariadb.com/docs/
- World Wide Web Consortium. (s. f.). HTML: The markup language. https://www.w3.org/TR/html/
- Yandex. (s. f.). BEM: Block, Element, Modifier. https://en.bem.info/

Material de clase

- Medeiros, M. (s. f.). Ingeniería de software [Presentación de diapositivas]. Google Slides. https://docs.google.com/presentation/d/1sbgGYkt3LE0GU2G4zogwOdB--lKII0CL/edit

Anexo - Actas

01_acta15_06_2026 (1).pdf

02_acta20_04_2026.pdf

03_acta 12_5_2026.pdf

04_acta 19_05_2026.pdf

05_acta 10_06_2026.pdf

06_acta 15_06_2026.pdf

07_acta23_06_2026.pdf

08_acta30_06_2026.pdf

09_acta07_07_2026.pdf

10_acta14_07_2026.pdf

11_acta21_07_2026.pdf

12_acta28_07_2026.pdf

13_acta04_08_2026.pdf

14_acta19_08_2026.pdf

Anexo

- Reglamento Interno
- Prototipo del sistema en figma
- Trello
    - Tablero primera entrega
    - Tablero segunda entrega
    - Tablero tercera entrega
- Video demostrativo del sistema
- Repositorio de github
- Documento de la entrevista con el cliente
- **Historial Cronológico de Commits:**
    - https://github.com/ipsum-maker/Ipsum-ProyectoEgreso/commits/main/
- **Reglamento Interno de Codificación de Ipsum:**
    - https://github.com/ipsum-maker/Ipsum-ProyectoEgreso/blob/main/docs/Regament%20de%20relevamiento%20de%20software.md
- **Documentación Formal de Persistencia (DER, MER y 3FN):**
    - https://github.com/ipsum-maker/Ipsum-ProyectoEgreso/tree/main/docs/persistencia